Aether por dentro

El libro completo del proyecto

24 de octubre de 2026

Índice

Parte A · Panorama

Prólogo: cómo leer este libro 4

Capítulo 1: El mapa de Aether 8

Parte B · El cuerpo físico

Capítulo 2: Electricidad y física para mecatrónicos 14

Capítulo 3: Estructura y mecánica del robot y del carrito 21

Capítulo 4: Alimentación, baterías, protecciones y paro de emergencia 29

Capítulo 5: Motores, drivers y el ESP32 36

Capítulo 6: Sensores y cámaras 43

Capítulo 7: La celda de manufactura 49

Capítulo 8: La Raspberry Pi 4 y la seguridad eléctrica del GPIO 57

Capítulo 9: El Jetson Orin Nano Super 63

Parte C · El software

Capítulo 10: Redes desde cero 70

Capítulo 11: La web por dentro 76

Capítulo 12: La terminal, Linux y los procesos 83

Capítulo 13: Python, FastAPI y la configuración 88

Capítulo 14: Visión en el borde 94

Capítulo 15: La voz de Lumina 99

Capítulo 16: Sistemas distribuidos 106

Capítulo 17: Del recuadro al inventario 110

Capítulo 18: Todos los idiomas de Aether 114

Capítulo 19: El panel de Aether 119

Capítulo 20: El sitio público, el dominio y Cloudflare 123

Capítulo 21: Seguridad 127

Parte D · Ingeniería y entrega

Capítulo 22: Pruebas y calidad 132

Capítulo 23: Git, documentación y auditoría 137

Capítulo 24: El video y la presentación en vivo 142

Capítulo 25: Manual para cuando todo falla 146

Capítulo 26: Hacia dónde puede crecer Aether 152

Capítulo 27: Lo que enseñan las empresas reales 156

Apéndices

Apéndice A: Hoja de arranque y apagado 159

Apéndice B: Lista para la defensa 163

Glosario y pendientes

Glosario de siglas y términos 166

Pendientes de confirmar con el equipo 173

Prólogo: cómo leer este libro

Para quién es, qué cuenta, cómo está escrito y cómo distinguir lo que se comprobó de lo que se supone.

Un libro para alguien que llega de afuera

Este libro explica **Aether**, un proyecto integrador de ingeniería mecatrónica desarrollado por un equipo de seis personas en Chihuahua, México: un **robot móvil de transporte** y una **celda de manufactura** que inspecciona piezas, con un sistema de software que une todo. Está escrito para una persona que **no conoce al equipo ni lo que se hizo**: un profesor que evaluará el proyecto, alguien que tendrá que retomarlo, o un futuro integrante que llega el primer día. No se da por sabido ningún nombre, ninguna pieza ni ninguna sigla.

La promesa del libro es modesta y práctica: al terminarlo, quien lo lea puede **explicar con sus palabras por qué cada pieza está donde está**, **arrancar y apagar el sistema en el orden correcto**, **diagnosticar las fallas típicas** y **defender el proyecto sin afirmar nada que no esté comprobado**. No es un curso general de electrónica, redes o Python: la teoría aparece solo cuando hace falta para entender algo que ocurre en Aether, con ejemplos y errores reales del proyecto.

Una verdad incómoda

Aether junta, en un solo sistema, materias que normalmente se estudian por separado: electricidad y mecánica, microcontroladores, redes de computadoras, sistemas operativos, desarrollo web, visión por computadora e inteligencia artificial. Por eso, cuando algo falla, parece magia negra. No lo es: casi todo lo que ocurrió en los meses de trabajo se explica con un puñado de ideas que se aprenden en una tarde. Los capítulos están escritos para dar esas ideas **en el momento en que se necesitan**.

Una promesa sobre las siglas

En computación y en electrónica se usan muchas siglas y la mayoría de los libros las sueltan como si todos las supieran. Aquí **cada sigla y cada término técnico se explica la primera vez que aparece**, con su nombre completo (casi siempre en inglés, porque así se escribieron) y una frase que dice qué es en la vida real. Al final del libro hay un **glosario unificado** con todas, por si alguna se olvida.

Cinco palabras que se usan en todo el libro y conviene tener claras desde ya:

Palabra	Qué significa aquí
Programa	Un conjunto de instrucciones guardado en un archivo que todavía no está haciendo nada
Proceso	Un programa que ya está corriendo
Servidor	Un programa que se queda esperando a que otro le pida algo
Cliente	El programa que pide
Red	Los cables o el WiFi por los que dos computadoras se mandan mensajes

Cuatro etiquetas de certeza

Este libro separa lo que sabe de lo que supone. Cada afirmación importante lleva una de estas marcas:

- **VERIFICADO:** se leyó en el código, se comprobó con una prueba automática o se vio en una salida real pegada por el equipo.
- **MEDIDO:** es una cifra obtenida con una prueba o un instrumento; el texto dice dónde y con qué.
- **NO VERIFICADO:** se supone, o viene de un documento que nadie comprobó.
- **PENDIENTE:** falta un dato que solo el equipo puede dar. Lleva un código (con la forma **P-xx**) que remite a la lista del final del libro.

Cuando algo es una regla general de ingeniería (no una medición de Aether), el texto lo dice con las palabras *regla general* o *conocimiento general*.

Regla de resolución de contradicciones. La documentación del repositorio envejece más rápido que el código. Cuando un documento contradice al código, **gana el código**; cuando la guía de construcción del 9 de octubre de 2026 contradice a la bitácora de hardware del Jetson del 21 de octubre, **gana la bitácora**. Las decisiones firmes que el equipo comunicó el 24 de octubre prevalecen sobre cualquier documento previo.

Cómo está organizado

Parte	Capítulos	De qué trata
A. Panorama	1	Qué es Aether, quién hace qué, cómo encajan las piezas

Parte	Capítulos	De qué trata
B. El cuerpo físico	2 a 9	Electricidad, mecánica, alimentación y paro de emergencia, motores y ESP32, sensores, la celda de manufactura, la Raspberry Pi y el Jetson
C. El software	10 a 21	Redes, web, terminal y Linux, Python y FastAPI, visión, voz, sistemas distribuidos, inventario, comunicaciones de la línea, el dashboard, el sitio público y la seguridad
D. Ingeniería y entrega	22 a 27	Pruebas, Git y documentación, el video de presentación, el manual para cuando todo falla, el crecimiento futuro y los casos de la industria
Apéndices	A, B, glosario, pendientes	Hoja de arranque y apagado, lista para la defensa, glosario unificado y lista de pendientes

Cada capítulo empieza con una historia o un hallazgo real del proyecto, explica el concepto desde cero, dice en qué archivo o comando se encuentra y qué síntomas da cuando falla, y termina con preguntas de repaso. Los subtítulos cambian de un capítulo a otro porque cada uno trata cosas distintas.

Cómo se escribieron los comandos

Todo comando indica **en qué máquina y en qué ventana** se ejecuta, entre corchetes, por ejemplo `[Laptop · PowerShell]` o `[Jetson · SSH]`. Los comandos de Linux (Jetson y Raspberry Pi) se escriben en una terminal Bash; los de la laptop (que usa Windows) se escriben en PowerShell. **Nunca aparecen claves ni contraseñas**: el repositorio es público y el libro también debe poder serlo. Donde hace falta una clave se nombra la variable de entorno que la guarda, no su valor.

Si solo hay un día

Leer completos los capítulos 1, 2 (primera mitad), 7 y 25, y el Apéndice A. Con eso se puede arrancar el sistema, entender la celda y diagnosticar lo básico. El Apéndice B sirve para preparar una defensa ante un jurado.

Una nota sobre el estado del proyecto

El libro está escrito en octubre de 2026 y describe el proyecto **tal como estaba el 24 de octubre**. Las partes de software funcionan y están probadas en laptop; varias partes de hardware (el carrito, la celda en la Raspberry Pi, el micrófono real) **todavía no se han**

probado en el equipo físico, y el libro lo dice cada vez. Esa franqueza es parte del método del proyecto: *no vender lo que no se ha comprobado*.

Capítulo 1: El mapa de Aether

Qué es el proyecto, quién lo hace, qué máquinas lo componen y por qué el video nunca viaja por el WiFi.

Qué es Aether, en una frase y en un párrafo

Aether es un sistema que junta dos cosas en un solo alcance: **Aether Inventory**, un robot móvil que recorre un espacio, mira productos y convierte lo que ve en información de inventario; y **una línea de manufactura**, una **celda** con banda transportadora donde una cámara decide si una pieza automotriz es una pieza registrada (PASS) o no (FAIL) y un pistón expulsa lo rechazado. El producto de la línea son **piezas automovilísticas**; las que se usan en las pruebas son tres modelos de **ECU** (*Engine Control Unit*, la computadora de motor de un coche).

El proyecto nació el **30 de julio de 2026** como proyecto integrador de ingeniería, con una ambición de **prototipo comercialmente defendible**: no una demostración desechable. La ventana de desarrollo es de unos **4 meses** con aproximadamente **2 horas diarias** de dedicación por persona, unas **240 horas** totales, según los documentos de contexto; toda decisión de alcance se mide contra ese presupuesto de tiempo. El requisito académico es **integrar manipulación y actuación física real, con hardware real**, en la línea de manufactura: no basta una simulación.

El equipo: seis personas, seis áreas

Persona	Área
Mario	Todo el software del robot y la visión por computadora; es además quien lleva este repositorio
Isaac	Diseño mecánico, CAD (diseño asistido por computadora) e impresión 3D; diseñó la banda transportadora y trabaja los fixtures
Pedro	El **pistón de scrap**: el mecanismo que expulsa las piezas rechazadas
Uriel	La comunicación de la Raspberry Pi con el controlador de la celda y con la visión
David	Ensamble mecánico y electrónica del robot y de la celda
Emmanuel	La **HMI/GUI** (*Human-Machine Interface*, pantalla de operación) aparte

Cuando en un capítulo de hardware falta un dato que pertenece al área de una de estas personas, el libro **no lo inventa**: lo marca **PENDIENTE** y nombra a quien debe confirmarlo.

El problema que Aether resuelve

El diagnóstico del equipo (informe integrador, capítulo 2) se resume en cuatro causas. **Personal:** depender de personas para trasladar piezas entre estaciones y contar inventario introduce variabilidad. **Método:** sin un protocolo de enlace entre estaciones, el operador humano se vuelve el mecanismo de coordinación por defecto y aparecen tiempos muertos. **Detección:** con reflejos de luz, un marcador de posición puede leerse como otro identificador válido sin dar señal de error. **Entorno:** la iluminación variable y la falta de puntos de referencia fijos aumentan el error de detección.

El mensaje comercial que se defiende es concreto: "*el robot revisó el Anaquel 4 y encontró 23 unidades del producto X*", no detalles de implementación. El proyecto se obliga a **no prometer más de lo que ha verificado** (el principio de honestidad de producto, más abajo).

El mapa de máquinas

Aether no corre en una sola computadora. Estas son las piezas, quién corre qué y por qué puerto habla (los **puertos** son las "puertas" numeradas de una computadora por donde entran las conexiones; el capítulo 10 los explica):

Máquina	Qué corre	Puerto	Qué le llega o qué manda
Laptop (Windows)	El **servidor central**: base de datos, panel web, asistente Lumina (modelo de lenguaje local Ollama), servicio de voz con el micrófono	8000 (servidor), 11434 (Ollama)	Páginas y datos JSON; texto de voz
Jetson Orin Nano Super (en el robot)	**Agente de percepción**: cámara, YOLO, lectura de códigos y ArUco, video anotado	8001	Manda a la laptop solo observaciones JSON; sirve su video a la página
ESP32 (en el robot)	Control de bajo nivel: ruedas, elevador, empujador, uña, sensores cercanos y watchdog	USB al Jetson	Recibe órdenes del Jetson por USB
Raspberry Pi 4 (en la celda)	**Agente de la celda**: cámara de línea, visión, relevadores, `celda.html`	8000	Manda a la laptop eventos de pieza; sirve su propio video y su pantalla
LOGO (en la celda)	Mini controlador lógico que mueve la banda	Entrada I4 desde un relevador	Recibe un pulso de la Pi

Máquina	Qué corre	Puerto	Qué le llega o qué manda
Robot carrito	Estructura, ruedas, torre elevadora (capítulos 3 y 4)	—	—

(El "robot móvil" y el "carrito" son la misma máquina: el robot de transporte con elevador. El capítulo 3 lo describe.)

Mover datos chicos, no datos grandes

La decisión de arquitectura más importante del último mes se resume en una frase: **cada cámara se procesa y se muestra en su propio equipo; por la red solo viajan datos pequeños.** Un video de 640 × 480 a 30 cuadros por segundo son decenas de megabits por segundo; una **observación** ("vi una `computadora2`, con 0.87 de confianza, a las 15:04:05") son unos cientos de bytes. Por eso:

- El **Jetson** abre su cámara, corre YOLO y **sirve su video anotado él mismo**. A la laptop solo le manda JSON.
- La **Pi 4** hace lo mismo con la cámara de la celda.
- La **laptop** sigue siendo el servidor central: guarda el inventario, sirve las páginas, aloja a Lumina.
- La página se abre en el navegador de cada equipo, y la página lee del servidor **la dirección del video**; si quien mira es el mismo equipo de la cámara, la dirección sale como `127.0.0.1` y el video no sale de ese equipo.

Esa decisión está documentada en la **ADR 0012** (una *ADR*, *Architecture Decision Record*, es un documento corto que registra una decisión de arquitectura, sus alternativas y su estado). **Estado:** propuesta implementada en la rama `jetson-agente` de Git; **todavía no está aceptada formalmente** por Mario (**PENDIENTE (P-32)**), y se probó en laptop y, parcialmente, en el Jetson (capítulo 9).

Las decisiones de arquitectura, una por una

#	Decisión	Qué resolvió
1	Percepción	Enfoque híbrido: **YOLO** (detección y conteo) + código de barras/QR (identificación). Clasificar un producto solo con la imagen queda como evolución futura
2	Localización	**ArUco** como verificación de posición; no es un sistema de navegación

#	Decisión	Qué resolvió
3	Navegación	Waypoints con odometría por encoders más **detección reactiva de obstáculos** (no "evasión inteligente")
4	Cómputo	**Jetson Orin Nano Super** en el robot; ESP32 para el bajo nivel. (La contingencia con Raspberry Pi 5 se descartó el 30 de septiembre)
5	Esquema de datos	Dos niveles: **Observations** (crudo) y **Declared Inventory** (declarado) con tres estados de cobertura
6	Comunicaciones	MQTT para telemetría (aún sin implementar), REST para el panel; el paro de seguridad no depende de la red
7	Alcance del panel	Inventario declarado más historial y trazabilidad (Opción B)
8 → 10	Línea de manufactura	Primero dos brazos industriales (KUKA y UR5); el **16 de septiembre** se reemplazaron por **banda + fixtures + pistón** (ADR 0010)
9	Auto-refresh	El inventario se refresca solo cada cierto tiempo (versión reducida del monitoreo en vivo)
11	Reparto de roles	Borrador no decidido
12	Agentes de cámara y voz local	La decisión de "datos chicos" (arriba)

El cambio de la línea merece una frase aparte: el equipo **eliminó los dos brazos robóticos** porque la coordinación de tres robots con tiempos críticos era demasiado riesgosa para el presupuesto de horas y la banda más pistón **sigue cumpliendo el requisito de hardware real**. Las secciones de los capítulos 2, 3 y 4 del informe integrador de septiembre que aún mencionan los brazos están superadas por este libro.

Principios no negociables

El proyecto fijó cinco reglas de conducta técnica que explican muchas decisiones:

1. **Honestidad de producto.** Nunca prometer capacidades no verificadas, ni en documentación ni en la interfaz.
2. **Terminología precisa.** "Conteo visible detectado" **no** es "conteo real de inventario". ArUco **no** es navegación. "Cobertura completa" significa que el procedimiento terminó, no que se vio todo. El actuador de la línea es lógica programada de punto fijo, **no** "aprendizaje".
3. **Verificación aritmética.** Las cifras que entrega alguien (cotizaciones, mediciones) se verifican y se señalan las discrepancias.

4. **Módulos independientes y probables.** Cada módulo se puede probar antes de integrarlo.
5. **No fabricar dependencias.** No se instala una librería, un campo ni un endpoint "por si acaso".

Qué existe de verdad (a 24 de octubre de 2026)

Pieza	Estado
Percepción híbrida (YOLO + QR + ArUco) en laptop	**VERIFICADO**, corre; pruebas automáticas
Agente del Jetson	Corrió en el Jetson real: 29.0 FPS (**MEDIDO**, reportado)
Servidor, base de datos, panel, historial	**VERIFICADO**
Lumina (cara, voz, asistente local)	**VERIFICADO** en laptop
Servicio de voz (Vosk)	Probado con audio simulado; micrófono real **NO VERIFICADO**
Celda en la Pi 4	Construida y probada en laptop; Pi real **NO VERIFICADO**
Carrito (estructura, torre, electrónica)	Diseño detallado; armado **NO VERIFICADO**
ESP32 (firmware)	No existe todavía
MQTT	Sin implementar
Paro de emergencia físico (robot y celda)	Diseño del robot; celda sin documentar
Suite de pruebas	**MEDIDO** hoy: 345 pasan, 2 fallan (las mismas dos de Lumina de siempre)

La última fila es una cifra de ejecución real: el 24 de octubre se corrió la suite de pruebas del servidor y dio **345 pruebas aprobadas y 2 fallidas** en 34.9 s; las dos fallidas son de Lumina (`test_robot_ask_conserva_varias_oraciones_reales_tras_filtrar` y `test_robot_ask_lista_numerada_tambien_se_detecta`), ya fallaban antes de los cambios recientes y no se tocaron. Las pruebas de la parte de borde del repositorio (`tests/edge`) dieron **7 aprobadas**.

La historia en una línea de tiempo

Fecha (2026)	Qué pasó
30 jul	Arranca el proyecto
30 ago	Estructura inicial: arquitectura, ADR, entornos virtuales

Fecha (2026)	Qué pasó
1–2 sep	Percepción combinada, motor de inventario, API, panel y robot Lumina; primer informe integrador
16 sep	Cambio de la línea: banda + fixtures + pistón (ADR 0010)
30 sep	Loader global y vista por capas del robot en el panel
8 oct	Documentación de punta a punta
16–17 oct	Sitio público, dominio `aethermx.online` y endurecimiento de seguridad
21 oct	Celda, Lumina de escritorio y carrito 3D al repositorio; primeras sesiones en el Jetson
22 oct	ADR 0012 e implementación de agentes, voz y arranque
23 oct	Primera prueba real del agente en el Jetson; correcciones de CSP; palabra de activación

Cómo se conectan los capítulos

El capítulo 2 enseña la electricidad necesaria; el 3 y el 4 describen el cuerpo del robot y su energía; el 5 y el 6, sus músculos y sentidos; el 7 y el 8, la celda y la Raspberry Pi; el 9, el Jetson. A partir del capítulo 10 el libro sube a la capa de software: redes, web, Linux, Python, visión, voz, datos y seguridad. La Parte D cierra con calidad, documentación, presentación, diagnóstico y futuro.

Preguntas de repaso: visión de conjunto

1. ¿Cuál es la diferencia entre el robot móvil y la celda, y qué tienen en común?
2. ¿Por qué el video de cada cámara no viaja por la red hasta la laptop?
3. ¿Qué cambió el 16 de septiembre y por qué?
4. Explicar la diferencia entre "conteo visible detectado" y "conteo real de inventario".
5. Nombrar tres piezas que **todavía no están probadas en hardware real**.
6. ¿Quién del equipo confirmaría un dato sobre el pistón? ¿Y sobre la comunicación de la Raspberry?

Capítulo 2: Electricidad y física para mecatrónicos

Voltios, amperios, vatios, tierra común, torque y fricción: lo mínimo que hay que entender para que un robot se mueva sin quemar nada ni lastimar a nadie.

Dos frases de un manual que valen cuatro semestres

En la guía de construcción de la celda de manufactura de Aether hay una línea que parece un detalle y es una clase completa de electrónica: "*Un GPIO de la Pi es de 3,3 V: 24 V lo quema.*" (Un **GPIO** es un pin de la computadora Raspberry Pi que puede leer o encender una señal; el capítulo 8 lo explica.) Unas líneas más abajo aparece otra: "*Un módulo infrarrojo de 5 V de Arduino no sirve en esta entrada.*"

Las dos dicen lo mismo con distinto número: **cada componente eléctrico aguanta un voltaje; si recibe más, se rompe; si recibe menos, no entiende lo que se le dice.** En Aether conviven tres mundos de voltaje: **3.3 V** (los pines de la Raspberry Pi y del microcontrolador ESP32), **5 V** (la alimentación de varias placas) y **24 V** (la parte industrial de la celda). Este capítulo enseña a moverse entre ellos y después añade la mitad mecánica: qué fuerza hace falta para mover un robot de hasta 40 kg.

Aclaración de método. Este capítulo es teoría con ejemplos del proyecto. Las cifras que sirven solo para entender (un LED, una rampa) están marcadas como ejemplo; los datos reales del hardware que todavía no están confirmados (peso final, diámetro de llantas) están marcados **PENDIENTE** y no se inventan. Cuando el libro dice **VERIFICADO**, significa que se leyó en el código, en una prueba o en una salida real pegada por el equipo; **MEDIDO**, que es una cifra obtenida con una prueba; **NO VERIFICADO**, que se supone sin comprobar.

Voltios, amperios y vatios con una manguera

Una manguera de jardín sirve de modelo.

- El **voltaje** (símbolo V, unidad: voltio) es la **presión** del agua: lo que "empuja" a la electricidad por el cable. Una batería de 12 V empuja con 12 unidades de presión.
- La **corriente** (símbolo I, unidad: amperio, A) es **cuánta agua pasa por segundo**. Una corriente grande es una manguera ancha a todo chorro.

- La **potencia** (símbolo P, unidad: vatio, W) es **cuánto trabajo hace** ese chorro: **P = V × I**.

Dos ejemplos del proyecto. La computadora del robot, un **Jetson Orin Nano Super** (capítulo 9), tiene modos de energía de 7 W, 15 W, 25 W y MAXN_SUPER (el máximo); la bitácora de hardware del equipo registra que **hoy corre en el modo de 25 W** (**VERIFICADO**, salida real del comando `nvpmodel`). Y la fuente oficial de una Raspberry Pi 5 de 27 W entrega hasta unos 5.4 amperios a 5 V.

Importa porque **los cables, fusibles, conectores y fuentes se eligen por amperios**, no por voltios: un cable delgado que aguanta 1 A se calienta como resistencia de tostador si recibe 10 A.

La ley de Ohm y el LED que no se quema

La **ley de Ohm** dice que **V = I × R**, donde R es la **resistencia** (unidad: ohmio, símbolo Ω), algo que se opone al paso de la corriente, como un estrechamiento en la manguera.

Ejemplo clásico, y justo lo que el plan de seguridad de la celda pide hacer antes de conectar una carga real a la Raspberry (capítulo 8): encender un **LED** (diodo emisor de luz, *Light-Emitting Diode*). Un LED es traicionero: casi no se opone a la corriente, así que conectado directo a un pin se "come" toda la que pueda y se quema. Se le pone una resistencia en serie.

> **Cuenta de ejemplo (regla general, no un dato del proyecto):** pin de 3.3 V, LED rojo que consume unos 2.0 V al encender y que se quiere a 10 mA (0.010 A). La resistencia debe "gastar" 3.3 − 2.0 = 1.3 V. R = 1.3 / 0.010 = **130 Ω**. Se usa la comercial más cercana por arriba, 150 Ω, que deja pasar unos 8.7 mA.

Con una fórmula se calculó algo que protege un componente de pocos centavos. La misma lógica, a otra escala, explica por qué entre el pin de una Pi y un motor hay un **driver** o un **relé** (capítulos 5 y 8).

De la pared a la batería: corriente continua y alterna

Existen dos tipos de electricidad. La **corriente continua** (CC o DC, *Direct Current*) fluye siempre en el mismo sentido: la de las baterías, las fuentes de las computadoras y todo lo que hay dentro del robot y de la parte de control de la celda (12 V de batería, 5 V, 3.3 V, 24 V del tablero). La **corriente alterna** (CA o AC, *Alternating Current*) cambia de sentido

muchas veces por segundo: la del enchufe de la pared. **La alterna aparece solo en el enchufe** que alimenta una fuente. Regla general: nadie debe abrir ni cablear nada conectado directo a la pared sin que lo revise la persona del equipo que sabe de eso.

Tierra común: el cable que nadie dibuja

Este tema es el que más confunde. La guía de construcción lo resume en una frase: *"Los 0 V de la batería de motores y el GND de la Pi se unen en un solo punto."*

GND (*ground*, "tierra" o referencia de 0 V) es el punto contra el que se miden todos los voltajes. Un voltaje no existe solo: siempre es una **diferencia entre dos puntos**. Cuando una placa le manda a otra una señal de "3.3 V", realmente dice "estoy 3.3 V por encima de mi referencia". Si las dos placas no comparten referencia, cada una tiene su propia idea de dónde está el cero y la señal llega como ruido.

Por eso el ESP32, los drivers de motores y las demás placas **comparten GND**. Y por eso importa *dónde* se unen: **en un solo punto**. Si los negativos se unen en varios lugares al azar, las corrientes grandes de los motores pueden meterse por los cables delgados de señal y producir fallas raras e intermitentes, de las que "solo pasan los jueves".

Niveles lógicos: 3.3 V, 5 V y 24 V no se entienden entre sí

Una señal digital es un cable que está "alto" (casi el voltaje de la placa) o "bajo" (casi 0 V). Eso se llama **nivel lógico**.

Dispositivo	Nivel de sus pines	Qué pasa si recibe más
Raspberry Pi (GPIO)	3.3 V	Más de 3.3 V puede dañar el pin o el chip
ESP32	3.3 V	Igual; aunque se alimente por USB a 5 V, sus pines son de 3.3 V
Sensores industriales de la celda	24 V	No se conectan a pines de la Pi
Módulos "de Arduino" de 5 V	5 V	No sirven en una entrada de 24 V ni deben entrar a un pin de 3.3 V

(La tabla combina lo que dicen la guía de construcción y la hoja de conexión de la celda con la regla general de cada familia de placas.)

La solución industrial es el **aislamiento**: un **relé** (interruptor mecánico que se cierra con una bobina) o un **optoacoplador** (un LED y un fototransistor dentro de un chip, que pasa la señal con luz y no con cobre). Así el mundo de 3.3 V *manda* sin *tocar* el de 24 V. Esa es la razón de que, en la celda, la Raspberry no se cablee directo a los sensores ni al pistón y haya un relevador de por medio (capítulo 7).

PNP y NPN: el problema de los tres hilos

Los **infrarrojos industriales** de la celda tienen tres hilos: **marrón** (+24 V), **azul** (0 V) y **negro** (la señal). Pero el hilo negro puede comportarse de dos maneras:

- **PNP**: al detectar, su salida **entrega** +24 V.
- **NPN**: al detectar, su salida **conecta a 0 V**.

La entrada del controlador (un PLC o un LOGO, capítulo 7) debe ser del tipo compatible. Si no coinciden, **el LED de la entrada nunca se enciende aunque el sensor sí detecte**, y no hay manera de arreglarlo con programa. La hoja de conexión lo dice así: "*Eso no se arregla en Python.*" Moraleja de ingeniero: **antes de escribir una línea de código, se comprueba con la mano y con el LED que la entrada ve al sensor.** Marca, modelo y tipo de los infrarrojos reales: **PENDIENTE (P-13)**.

Fusibles: el eslabón que se sacrifica a propósito

Un **fusible** es un hilito calibrado que se funde cuando pasa más corriente de la permitida y corta el circuito. Es barato y existe para que lo que se queme sea él y no el motor, la batería o el cable. Regla general: se elige **más grande que la corriente normal del circuito y más chico que lo que aguantan los cables y componentes**. En el diseño del carrito (capítulo 3) hay dos ramas con fusible, de 15 A y 10 A; el capítulo 4 las explica.

De la electricidad al movimiento: fuerza, torque y rodar

Un robot es la electricidad convertida en empuje.

- **Fuerza** (F, unidad: **newton**, N): lo que empuja o jala. El **peso** es la fuerza con que la Tierra jala una masa: **peso = masa × g**, con g ≈ 9.81 m/s². Un robot de 40 kg pesa unos **392 N**.
- **Torque** o **par** (τ, unidad: **newton-metro**, N·m): la fuerza que hace *girar*. Es la fuerza por la distancia al eje, **τ = F × r**: lo que se siente al apretar una tuerca con una llave larga; la misma fuerza con una llave más larga da más torque.

- **Fricción de rodamiento**: lo que frena a una llanta que rueda. Se aproxima como **F = coeficiente × peso**. Para caucho sobre piso liso, un coeficiente típico (regla general) va de 0.02 a 0.05.
- **Tracción**: la fuerza máxima que la llanta puede transmitir al piso **sin patinar**: **coeficiente de agarre × peso que carga esa llanta**. Para caucho sobre piso liso, del orden de 0.5 a 0.8 (regla general).
- **Pendiente**: subir una rampa de ángulo α añade una fuerza en contra de **peso × sen(α)**.

Un ejemplo con los números de partida del robot

Las estimaciones del equipo (todas **PENDIENTES de confirmar**, **P-02**): peso total máximo **40 kg** y unos **17 kg sobre cada llanta motriz**. El robot es **diferencial**: dos motores y una llanta por motor (capítulo 3). La velocidad máxima de 0.35 m/s y la aceleración de 0.4 m/s^2 salen de la hoja de armado del carrito. El diámetro real de la llanta es **PENDIENTE (P-03)**; para poder hacer la cuenta se usa el de la hoja de armado, 102 mm (radio 0.051 m), **solo como ejemplo**.

> **Cuenta de ejemplo con datos estimados (no son datos confirmados):**
>
> - Peso: 40 × 9.81 ≈ **392 N**.
> - Fricción de rodamiento con coeficiente 0.03 (supuesto): 0.03 × 392 ≈ **11.8 N**.
> - Aceleración a 0.4 m/s^2: 40 × 0.4 = **16 N**.
> - Fuerza total en piso plano: 11.8 + 16 ≈ **27.8 N** entre las dos llantas, o **~13.9 N por llanta**.
> - Torque por llanta con radio 0.051 m: 13.9 × 0.051 ≈ **0.7 N·m**.
> - Rampa de **5°** (supuesto): 392 × sen(5°) ≈ **34 N** extra. Total ≈ 62 N, ~31 N por llanta, torque ≈ **1.6 N·m** por llanta.
> - ¿Patina? Cada llanta carga 17 kg (≈ 167 N). Con agarre 0.6: 0.6 × 167 ≈ **100 N** disponibles por llanta; los 31 N necesarios caben bien.
> - Potencia mecánica al acelerar a 0.35 m/s en piso plano: 27.8 N × 0.35 m/s ≈ **10 W**, sin pérdidas del motor ni de la transmisión. A velocidad constante, solo la fricción: 11.8 × 0.35 ≈ 4 W.

Dos lecciones valen más que los números. **Primera:** la rampa pesa más que todo lo demás; por eso la pendiente más inclinada del lugar donde trabajará el robot es *la* pregunta que dimensiona el motor. **Segunda:** la cuenta del torque se hace antes de comprar motores, no después, y el motor elegido debe entregarlo **con holgura**,

normalmente con un margen de seguridad del doble (regla general). El motor de tracción de Aether ya está decidido (el M0601, capítulo 3); esta cuenta sirve para comprobar que su torque alcanza.

Medir sin miedo: el multímetro

El **multímetro** convierte "creo que" en "mido que". Tres usos:

1. **Continuidad** (modo con símbolo de bocina): comprueba que dos puntos están unidos por cable. La hoja de armado pide, antes de conectar la batería, mirar continuidad entre el positivo y el negativo: **si pita, hay un corto** y no se conecta nada.
2. **Voltaje** (V con una raya recta, para continua): se mide **en paralelo**, con las puntas en los dos puntos que se quieren comparar. Es el modo más seguro.
3. **Corriente** (A): se mide **en serie**, **abriendo el circuito** y metiendo el multímetro como parte del cable. Es el modo peligroso: usado mal funde el fusible del aparato (o algo peor). Conviene aprender primero a medir voltaje.

Cómo lo resuelve un tablero industrial

El estándar de facto para sensores y controladores de un tablero industrial es **24 V de corriente continua**: es lo bastante alto para que el ruido eléctrico no confunda las señales en cables largos y lo bastante bajo para ser menos peligroso que la pared. Los robots móviles de almacén (**AGV**, vehículos guiados automáticamente, y **AMR**, robots móviles autónomos) suelen usar baterías de 24 V o 48 V, y su paro de emergencia es **cableado**, no un botón de pantalla. Aether aplica el mismo principio a escala de laboratorio.

Preguntas de repaso: electricidad y fuerzas

1. Un sensor PNP de 24 V se conecta por error a un pin de 3.3 V de la Raspberry. ¿Qué pasa, y qué pieza intermedia debió ir en medio?
2. Si el ESP32 y el driver de motores no comparten GND, ¿por qué la señal "alto" de uno puede no ser entendida por el otro?
3. Con los datos de ejemplo del capítulo, ¿por qué una rampa cambia más la elección del motor que acelerar más rápido en piso plano?
4. Un fusible de 15 A en un circuito de 12 V: ¿cuántos vatios deja pasar como máximo antes de fundirse?

5. ¿Qué dato falta hoy para dimensionar con seguridad el motor del robot, y quién del equipo tiene que confirmarlo?

Capítulo 3: Estructura y mecánica del robot y del carrito

De qué está hecho el esqueleto, por qué un robot diferencial gira como gira, cómo funcionan la torre y el empujador, y qué dice la hoja de armado 3D (revisión 3.8) pieza por pieza.

Un plano que se gira con el mouse

El equipo de Aether documentó el robot de transporte, al que llama "el carrito", en una **hoja de armado interactiva en 3D**: una página web donde se gira el modelo con el mouse y se avanza por etapas. Cada etapa dice qué se hace, por qué, con qué herramientas, cómo comprobar que quedó bien y cuál es el error típico. Su tabla de medidas lleva fecha del **16 de octubre de 2026** (revisión 3.4) y los ajustes posteriores llegan hasta la **revisión 3.8** y la banda del **18 de octubre**. Son unas cincuenta etapas de estructura, electrónica y funcionamiento, más una etapa por cada uno de los 107 cables de la placa electrónica.

Este capítulo y los de hardware que siguen usan esa hoja como **fuente del diseño**, con dos reglas que la propia hoja enseña:

1. **Todo número que su autor dibujó "a criterio" lleva un "~"** y aparece en una lista de *supuestos*. Aquí se marca igual.
2. La hoja cita como fuentes unos documentos del equipo (`CARRITO_CONSTRUCCION_PASO_A_PASO.md`, `GUIA_ARMADO_PRINCIPIANTES.md`, `COSTOS_CARRITO.md`, `PROYECTO_INTEGRADOR_CONTEXTO.md`) que **no están en el repositorio**, de modo que no se puede contrastar cada cifra con su origen.

Entonces, lo que sigue es **el diseño documentado por el equipo**, no la medición de un robot armado. Cuando el libro dice VERIFICADO se refiere a que **se leyó en la hoja**, no a que alguien midió la pieza con un vernier. El responsable del **diseño mecánico, CAD e impresión 3D** es **Isaac** y el del **ensamble mecánico y la electrónica** es **David** (el capítulo 1 presenta a todo el equipo). Si el carrito ya existe físicamente, las cifras se confirman contra el real: **PENDIENTE (P-04)**.

Qué hace el carrito, en tres frases

La unidad móvil llega a la banda transportadora de la celda y **se acopla** a ella. Va tomando los *fixtures* —las bases que sostienen cada pieza; aquí cada fixture lleva una **ECU**, la computadora de motor de un automóvil, de las marcas Siemens VDO y MaxxECU según el diseño de los fixtures— hasta apilar **6**. Luego viaja a la **estación de descarga** (una mesa inclinada) y, nivel por nivel, un **empujador** los saca por una ventana. Es un robot de **transporte con elevador**, no un brazo: consecuencia de que el proyecto abandonó dos brazos industriales (KUKA y UR5) en favor de banda más pistón (capítulo 1).

El esqueleto: aluminio 3030, escuadras impresas y tuercas T

La estructura es un marco de **perfil de aluminio 3030**: barras de 30 × 30 mm con una ranura de 8 mm. Se une con **tuercas T de martillo**, tornillos Allen **M6** y **escuadras impresas en 3D**, sin soldar. Números del diseño:

Concepto	Valor (revisión 3.8)	Estado
Huella exterior	0.50 m (largo, hacia el frente) × 0.40 m (ancho)	VERIFICADO en la hoja
Postes verticales	4 barras de 0.95 m	VERIFICADO en la hoja
Altura total aproximada	≈ 1.04 m con rodajas	NO VERIFICADO (~)
Largueros y travesaños	4 de 0.44 m y 7 de 0.34 m (más uno de 198 mm para el empujador)	VERIFICADO en la hoja
Cortes de perfil	19 piezas, ≈ 11.4 m en total	VERIFICADO en la hoja
Piezas impresas	28 escuadras + 16 placas de unión, más soportes, cuna de batería, bandeja y guías	VERIFICADO en la hoja
Tuercas T + Allen M6	100 juegos	VERIFICADO en la hoja
Material, impresora y relleno de las piezas impresas	No dichos (solo el consejo "40 % de relleno y 5 perímetros")	PENDIENTE (P-04)

Tres hábitos de oficio que enseña la hoja:

- **Las longitudes cuadran con aritmética.** 0.50 − 2 × 0.030 = 0.44; la hoja marca con una palomita (✓) cada cuenta comprobada.

- **El marco se escuadra con las diagonales.** En un rectángulo bien armado las dos diagonales miden igual: aquí $\sqrt{(0.5^2 + 0.4^2)} = 0.64$ m. La hoja pide igualarlas **a ±2 mm** *antes* de apretar los tornillos, porque "todo lo de arriba hereda este marco".
- **Las tuercas T entran de frente.** Se meten de lado en la ranura y al apretar giran un cuarto de vuelta y se traban. Error típico anotado: apretar sin que hayan girado y se salen.

Robot diferencial: girar sin volante

El robot es **diferencial**: **dos motores y una llanta por motor**, sin volante. Cómo se mueve (**conocimiento general**):

- Las dos llantas giran igual → avanza recto.
- La izquierda más rápido que la derecha → gira a la derecha, y al revés.
- Una avanza y la otra retrocede a igual velocidad → **gira sobre su propio eje**.

Con solo dos llantas el robot se caería hacia adelante o atrás, así que el diseño añade **4 rodajas** (ruedas locas) "twin" de 50 mm en las esquinas, que solo siguen la dirección que marcan las llantas motrices. Cada rodaja mide **9 cm de alto con su espiga**; ese número fija la altura de todo el marco sobre el piso.

El motor de tracción: M0601

El equipo decidió y cotizó como motor de tracción el **M0601** (dato firme, según el documento de costos del equipo "Costos Carrito 9", revisión 3.12b, que no está en el repositorio). Es un **motor-rueda**: una rueda de **Ø 102 mm** (ancho ≈ 45 mm) con el motor dentro. La hoja distingue el modelo **izquierdo (FIT1042)** del **derecho (FIT1038)**, espejos uno del otro, y advierte el error clásico: "*Montar dos del mismo lado: una rueda giraría al revés*". Cada uno trae **su propio driver y su propio encoder** (el capítulo 5 explica qué significa) y se maneja por un bus **RS485**. Funciona a **18 V**.

Lo que **sigue PENDIENTE (P-01)** es lo que ningún documento confirma: si los M0601 y los motores NEMA 17 de la torre (también incluidos en la cotización) **ya llegaron o están instalados**.

Un problema de ingeniería real que la hoja resuelve con un número: con un diámetro de 102 mm la llanta sube hasta 10.2 cm, pero el marco queda a 9 cm del piso, **así que la llanta chocaría con el perfil central**. Por eso el eje de las ruedas se corre **7 cm hacia**

adelante del travesaño central (que se pone "por el costado de atrás": *por el frente pasan los motores de las ruedas*). Un error así se descubre dibujando; descubrirlo con el aluminio cortado cuesta dinero.

Suspensión: 85 newtons que mantienen la llanta pegada al piso

Cada llanta motriz va en una **placa pivotante**: dos baleros **608ZZ** a presión y un tornillo **M8** de pivote, con un **resorte** que la empuja contra el piso. La hoja da la receta exacta: resorte de **75 mm libres**, comprimido **28 mm** (a 47 mm), constante de unos **3 N/mm**, lo que da **≈ 85 N** sobre cada rueda (unos 8.7 kg-fuerza).

Con dos llantas motrices y cuatro rodajas, una irregularidad del piso puede dejar una llanta **sin contacto**; sin contacto no hay tracción y el robot se desvía. El resorte garantiza una fuerza mínima. El balance, escrito por la hoja como error típico: "*Demasiada precarga levanta las rodajas y el carrito se mece; muy poca y las ruedas patinan al arrancar.*" Se ajusta **midiendo con vernier** en el robot real: es el tipo de calibración que nunca sale igual en el mundo físico que en el papel.

Peso y reparto: la cuenta que decide todo

Las estimaciones de partida del equipo (**PENDIENTES de confirmar, P-02**) son un robot de **hasta 40 kg** con unos **17 kg sobre cada llanta motriz**. Con la resta, 40 − 17 − 17 = **6 kg** para las rodajas: casi todo el peso va sobre las llantas que empujan, lo que da agarre (capítulo 2) y obliga a que las rodajas no se levanten.

La **batería** del diseño es una **BR-1224 de 12 V, 24 Ah y 8.3 kg**, de ≈ 16.6 × 17.5 × 12.5 cm (medidas "típicas", marcadas con ~), colocada **al centro y lo más abajo posible**, sujeta con topes impresos y una **correa con hebilla** alrededor del travesaño central. La razón, en palabras de la hoja: "*8.3 kg sueltos en un frenazo rompen algo. Abajo y al centro, baja el centro de gravedad.*" Modelo y capacidad reales de la batería: **PENDIENTE (P-05)**.

Centro de gravedad: el enemigo de las torres altas

El carrito mide poco más de 1 m y carga hasta 6 fixtures arriba. El **centro de gravedad** es el punto donde parece concentrarse todo el peso. Cuanto más alto está, más fácil es volcar al frenar, girar o tomar una pendiente. Regla general: un robot alto y angosto necesita su peso pesado abajo y centrado, y la **torre cargada** sube ese centro justo cuando el robot viaja a la estación de descarga. La altura real del centro de gravedad no existe como dato; calcularla con cifras inventadas es lo que el proyecto evita: **PENDIENTE (P-02)**.

La torre: un elevador que no se cae solo

La plataforma sube y baja con un **elevador de husillo**:

- **Dos husillos TR8** de 60 cm (tornillos de potencia de 8 mm: al girar suben o bajan una tuerca) y **dos varillas lisas de 8 mm** con **6 baleros lineales SC8UU** que guían la plataforma.
- Los mueven **dos motores a pasos NEMA 17 (modelo 17HS4401)** mediante **coples flexibles de 5 a 8 mm**, apoyados en **4 chumaceras KFL08**. Un **motor a pasos** avanza un "paso" fijo por cada pulso, así que "sabe" cuánto giró sin sensor de posición. El husillo tiene **autobloqueo**: sostiene la carga sin gastar corriente (la hoja incluye una etapa de prueba).
- **Tres finales de carrera** (interruptores de límite): dos del elevador (uno al fondo, 39.2 cm) y uno de *home* del empujador. Van como contacto **normalmente cerrado** para que un cable roto se lea como "presionado" y el motor se detenga (idea del capítulo 4).
- Recorrido útil: plataforma de **40.7 a 83.2 cm** del piso, con **6 niveles de 7.0 cm**: 0.407 + 6 × 0.070 = 0.827 m, justo la altura de la banda (83 cm) menos 3 mm.

La medida más importante, que la hoja revisa con palomitas: la **columna de la torre** tiene un hueco de **22.0 × 19.0 cm** y la **quilla** del fixture mide **21.3 × 16.5 cm**; las ECU sobresalen hasta ±11.15 cm del centro. La holgura entre fixture y guías es de **2 a 5 mm por lado**: lo mínimo para que entre sin atorarse ni bailar.

El empujador: cremallera, piñón y una uña con servo

Para **meter** el fixture a la pila y para **sacarlo** en la descarga, el carro lleva un **empujador**: una **cremallera** (barra dentada, 2 tramos de 15 cm = 30 cm) movida por un **piñón** (módulo 1.5, 20 dientes, Ø 30 mm) acoplado a un **tercer NEMA 17**. Recorre **25 cm** (su "home" es la posición 0.8 cm). Una **uña** —un dedo impreso de 7 mm que gira 90° sobre un eje, movido por un **microservo MG90S de 180°** (el de 360° no sirve)— engancha la ranura del fixture para **jalarlo ~7.7 cm** hacia dentro desde la banda.

La carga en la banda: una cinta que no espera

Este es el truco de diseño más fino y la hoja lo explica bien. La **cinta de la banda no se detiene**; el carrito no puede pedirle que pare. Entonces:

1. El carrito llega por **waypoints** (puntos de ruta), un "embudo" lo centra y la **cámara** lee un **ArUco** para confirmar su posición (capítulo 6).
2. Hace *home* del elevador y del empujador y deja la plataforma 3 mm **debajo** de la cinta (a 82.7 cm).
3. El fixture viaja sobre la cinta; cuando su centro pasa la **cresta del rodillo** (de Ø 50 mm en el extremo), su frente baja a la pila, la cinta lo sigue empujando hasta que la cola suelta la cresta y cae plano **≈ 6 cm afuera de la ventana**.
4. La uña baja, el empujador lo jala, sube la uña, regresa a home y la plataforma **baja un nivel (7 cm)**.
5. Se repite **uno a la vez** hasta 6. Un ciclo completo tarda ≈ 15 a 18 s; con la cinta a **0.20 m/s (supuesta)**, eso exige unos 3 m de separación entre fixtures. Por eso quien pone los fixtures espera el aviso "listo" del carrito.

Honestidad de la propia hoja: la velocidad real de la cinta y el punto exacto donde cae el fixture **se miden con la banda real**; el valor "6 cm" depende de que la cinta agarre menos que el PETG sobre PETG (supuesto).

La banda de Isaac y la mesa de descarga

La banda sale del archivo **STEP de SolidWorks** de Isaac (un formato de intercambio de modelos 3D): su modelo es un **tercio** de la banda real, así que el diseño pega **3 tramos de 1030 mm (3.09 m)** con **una sola cinta de 20.35 cm** de ancho. Las patas no vienen en el STEP y se dibujan con perfil 3030. Datos que importan:

- **Altura de la cinta: 83.0 cm** (supuesta; en el STEP la cinta queda **8.5 mm debajo del borde de los rieles**, así que si los rieles están a 83 cm la cinta está a 82.15 cm y "hay que subir la banda o bajar 8.5 mm la altura de carga").
- Guías laterales de 3 mm a 2 mm de la quilla, colgadas de los rieles cada ~0.5 m (**propuesta**: la banda real no trae guías y entre rieles hay 21.6 cm).
- **Mesa de descarga:** una **mesa inclinada de melamina de 1.4 × 0.3 m a ≈ 15°**, con bisagra, cuña, guías y tope de hule espuma (dibujados a criterio). Al empujar el fixture por la ventana, cuando su centro pasa la orilla de la mesa **se vuelca y resbala solo**; en una mesa plana el empujador tendría que arrastrar toda la fila y no le alcanzaría la fuerza.

Datos reales de la banda (motor, velocidad, altura de la cinta): **PENDIENTE (P-15)**.

La cara de enfrente: ventana de MDF

El frente del carrito lleva un **panel de MDF de 3 mm** con una **ventana de 24 × 9.7 cm** (borde inferior a 81.8 cm, 1.2 cm bajo la banda). El panel es además el **tope** de la pila. Las columnas delanteras están separadas **23.2 cm** (a z ±0.131) para que pasen las ECU; las traseras quedan más juntas.

Qué fija la hoja y qué falta confirmar

Tema	Hoja de armado (rev. 3.8)	Estado
Motor de tracción	2 × M0601 (FIT1042 izquierdo, FIT1038 derecho), 18 V, RS485	Decisión firme del equipo; llegada e instalación PENDIENTE (P-01)
Motores de la torre	3 × NEMA 17 (17HS4401) con drivers A4988	Decisión firme; llegada e instalación PENDIENTE (P-01)
Peso total y por llanta	No lo da (batería 8.3 kg)	Estimado del equipo: 40 kg y ~17 kg por llanta; PENDIENTE (P-02)
Cómputo	Jetson Orin Nano Super, "ya comprado"	VERIFICADO (bitácora del Jetson)
Cámara	Logitech 1080p, "ya la tiene" Mario	VERIFICADO (la cámara aparece en el Jetson)
Microcontrolador	ESP32 DevKitC de 38 pines	El equipo ya tiene un ESP32; modelo exacto PENDIENTE (P-07)
Armado físico	Etapas de armado y 107 cables	PENDIENTE (P-04): no hay evidencia de que ya esté armado

Probar la mecánica antes de energizar

La hoja termina cada etapa con una **comprobación física**. Cuatro que cualquier mecatrónico entiende:

1. "*Las 2 ruedas y las 4 rodajas tocan el piso al mismo tiempo.*" Si una no toca, el robot cojea.
2. "*Diagonales del marco de abajo iguales (±2 mm).*" Sin eso, el robot no anda derecho.
3. "*La batería no se mueve aunque sacudan la base; empujada a mano, la base rueda derecho unos 3 m.*"
4. "*Resortes comprimidos 28 mm, medidos con vernier.*"

No hay evidencia en el repositorio de que el carrito ya esté armado. Las pruebas anteriores son las que la hoja prescribe, no las que alguien reportó haber hecho.

El mismo esquema en los almacenes reales

Los robots de almacén que mueven estantes o cajas (los **AGV** y **AMR** ya definidos) usan casi la misma receta: tracción diferencial, ruedas locas de apoyo, batería baja para bajar el centro de gravedad y un mecanismo elevador. Lo que cambia es la certificación: los industriales llevan sensores de seguridad homologados y paros de emergencia certificados. El capítulo 27 revisa casos reales.

Preguntas de repaso: mecánica del carrito

1. Una llanta izquierda a 0.3 m/s y la derecha a 0.1 m/s: ¿hacia dónde gira el robot?
2. ¿Por qué la hoja mueve el eje de las ruedas 7 cm hacia adelante del travesaño central?
3. Con 40 kg y 17 kg por llanta motriz, ¿cuántos kilos quedan para las rodajas, y por qué eso es bueno y malo a la vez?
4. ¿Qué se comprueba con las dos diagonales de un marco rectangular y a qué tolerancia?
5. ¿Por qué la cinta no se detiene durante la carga y qué consecuencia tiene para quien coloca los fixtures?
6. ¿Qué dato falta para saber si el carrito ya existe físicamente?

Capítulo 4: Alimentación, baterías, protecciones y paro de emergencia

De dónde sale la energía del carrito, por qué se reparte en ramas, qué hace cada fusible, convertidor y relé, y por qué el botón rojo no es software.

Un diagrama de bandeja que se lee como un cuento

La hoja de armado del carrito (capítulo 3) cuenta la electrónica de potencia en el orden en que hay que hacerla: primero **se ajustan los voltajes sin conectar nada**, después fusibles y relé y, al final, una **prueba de humo** con la batería pero **sin ESP32 ni motores**. Ese orden es la lección del capítulo: *la electricidad se prueba por capas, de lo que menos cuesta quemar a lo que más.*

Sobre las fuentes. Una guía de construcción del 9 de octubre de 2026 describe un robot con una **Raspberry Pi 5** y un driver pequeño llamado TB6612. Esa Pi 5 fue un plan del 30 de septiembre que se descartó; cuando un documento contradice a la **bitácora de hardware del Jetson** (21 de octubre), el libro sigue a la bitácora. La arquitectura eléctrica de este capítulo sale de la **hoja de armado revisión 3.8**, que ya usa el **Jetson Orin Nano Super**. Es el **diseño del equipo**, no la medición de un carrito encendido: **David**, responsable del ensamble y la electrónica, confirma los datos finales de batería, convertidores y cableado (**PENDIENTE: P-05, P-19**).

Islas de voltaje y un solo cero

Voltaje	De dónde sale	Qué alimenta
12 V (batería)	Batería de plomo de 12 V y 24 Ah, "BR-1224", de 8.3 kg	Todo lo demás, repartido en dos ramas
18 V	**Elevador de voltaje de 400 W** (12 → 18 V)	Los dos motores-rueda M0601
12 V "de pasos"	La rama principal, ya pasada por el relé	Los 3 drivers **A4988** de los motores a pasos
5 V	**Reductor XL4005** (12 → 5 V), ajustado a 5.0–5.1 V	ESP32, módulo MAX485, servo de la uña y tira de LEDs
3.3 V	Regulador del propio ESP32	Lógica de los A4988, sensores VL53L0X, BNO055, PCF8574 y barra QTR-8A

Y **un solo cero**: el capítulo 2 explicó la tierra común. En este diseño todas las tierras se unen **en el borne IN− del elevador de voltaje**, adonde llega el negativo de la batería: "*Todas las tierras unidas: batería −, elevador, XL4005, drivers y ESP32*" (palabras de la hoja). Un solo punto evita que las corrientes grandes de los motores viajen por cables delgados de señal.

Las dos ramas de la batería

Del borne de la batería sale un cable rojo de **14 AWG** (calibre americano; 14 es grueso, para corrientes grandes) a un conector **XT60** (el enchufe amarillo de baterías; hembra del lado de la batería, macho del lado del carrito). Enseguida hay **dos portafusibles**, el corazón de la protección:

Rama principal: fusible de 15 A → relé → motores. El fusible alimenta el contacto **COM** de un **módulo de relé T90 de 30 A** de 2 canales. Del contacto **NO** ("normalmente abierto": cierra solo cuando la bobina recibe energía) salen dos ramas: la entrada del elevador de 18 V (ruedas) y la bornera **J1** de la placa (12 V de los drivers de los motores a pasos). El relé es el interruptor general de la **potencia**.

Rama auxiliar: fusible de 10 A → interruptor KCD1 → lógica. El otro fusible pasa por un **interruptor KCD1** (el de encendido del robot) y llega al **XL4005**, que reparte: 5 V al ESP32 y accesorios; **12 V** al **Jetson** (cable con plug de 5.5 × 2.5 mm, centro positivo); y 12 V a la **bobina** del relé, pasando por el paro de emergencia. Consecuencia: **apagar el KCD1 también suelta el relé**, porque le quita la energía a la bobina; con el robot "apagado" no queda potencia en las ruedas.

¿Por qué dos fusibles? Cada rama tiene un temperamento distinto:

- La de **motores** es potente y brusca: fusible grande para que los arranques normales no lo fundan.
- La **auxiliar** es delicada y sensible a caídas de voltaje: fusible más chico que se funde ante un corto antes de que se caliente un cable fino.
- Si se funde el de motores, el robot se detiene **pero el Jetson sigue vivo y puede reportar qué pasó**. Con un solo fusible se perderían las dos cosas a la vez.

Regla de la hoja sobre dónde colocarlo: *"El fusible va lo más cerca posible de la batería: si algo hace corto, se quema el fusible y no el cable."*

Leer la etiqueta de una batería

Una batería dice tres cosas:

- **12 V**: voltaje nominal.
- **24 Ah** (amperios-hora): capacidad. En teoría, 24 A durante 1 hora o 1 A durante 24 horas.
- **Energía** = 12 × 24 = **288 Wh** (vatios-hora).

Si solo estuviera conectado el Jetson trabajando a 25 W (su modo actual, **VERIFICADO** en la bitácora), 288 Wh darían unas **11 horas** en el mejor caso teórico (**cuenta de ejemplo**, no medida). Los motores, las pérdidas de los convertidores y la regla general de no descargar una batería de plomo a fondo recortan eso mucho. **La autonomía real se mide con el robot completo trabajando.**

Ajustar voltajes antes de conectar nada

La hoja lo dice con claridad: *"La mayoría de las piezas se queman por un voltaje equivocado. Marquen cada módulo con su voltaje en cinta."* Receta de la Etapa 11:

1. Con el módulo conectado a 12 V y **sin nada en la salida**, se gira su potenciómetro (un tornillito) con un desarmador de plástico hasta leer con el multímetro **5.0–5.1 V** en el XL4005 y **18.0 V** en el elevador.
2. Se ajusta el **Vref** de cada driver A4988 **sin motor conectado**: **0.80 V** (si la resistencia de sentido es R100) o **0.54 V** (si es R068), lo que limita cada motor a **1.0 A**. El **Vref** es un voltaje de referencia dentro del driver que fija cuánta corriente máxima deja pasar al motor: demasiado alto y el motor se calienta; demasiado bajo y se salta pasos.
3. Se anotan los valores (la hoja deja renglones para eso).

Dos reglas de seguridad que la hoja repite: **nunca conectar ni desconectar un motor a pasos con el driver encendido** (quema el A4988) y **medir antes de enchufar el plug del Jetson** que el centro sea el positivo (la polaridad al revés lo daña).

La prueba de humo

Antes de montar módulos delicados, la hoja pide una **prueba de humo** (se llama así porque, si algo está mal, sale humo): conectar la batería con el XT60 y el KCD1 encendido, **sin ESP32 y sin motores**. Se miden **5 V** en la placa y **12 V** antes del relé; el relé hace *clic*; se aprieta el paro y debe soltar, dejando el contacto NO en **0 V**. Se esperan 2 minutos y se

tocan los módulos: "*Ninguna pieza caliente ni con olor.*" Si algo falla, se quema un fusible o un regulador de unos pocos pesos, no el ESP32 ni los drivers. Antes de todo eso, con el XT60 **desconectado**, el multímetro en continuidad entre + y − **no debe pitar** (no hay corto).

El paro de emergencia: por qué el botón rojo no es un programa

El código de Aether ya tiene un botón "Paro" en la pantalla de la celda. El propio archivo (`backend/cell_hmi.py`) aclara en su comentario inicial que "*El paro de emergencia no pasa por aquí*". Es deliberado: **un botón de pantalla es software**, y el software puede estar colgado, sin red o mal escrito justo cuando más se necesita. Un **paro de emergencia** (*e-stop*) real es un **interruptor físico que corta la energía de lo peligroso, sin depender de que nada funcione**.

El diseño del carrito lo resuelve así:

1. **Botón LAY37 con contacto normalmente cerrado (NC).** Mientras está suelto deja pasar corriente; al pulsarlo, **abre** el circuito. Si un cable se rompe, también se abre: **falla hacia lo seguro**. Un contacto normalmente abierto haría lo contrario y un cable roto dejaría el paro sin efecto sin avisar.
2. **El paro no corta la potencia directamente: alimenta la bobina del relé.** El cable `12 V aux → contacto NC del paro → VCC del relé` hace que el relé **solo esté cerrado mientras el paro esté suelto**. Al pulsarlo, la bobina se queda sin energía, el relé **cae solo** y corta **a la vez** el elevador de 18 V (ruedas) y los 12 V de los drivers de los motores a pasos (elevador de plataforma y empujador). Frase de la hoja: "*aunque el programa esté colgado*".
3. **La rama auxiliar sigue viva**: el Jetson y la lógica **no se apagan** al pulsar el paro. El robot se detiene pero sigue "pensando" y puede reportar lo ocurrido.
4. **Dónde queda:** "*arriba, en una esquina de atrás*" del carrito, **al alcance de la mano** en cualquier momento del ciclo. Un botón al que hay que rodear el robot para llegar no es un paro.
5. **Prueba que pide la hoja:** "*El paro detiene ruedas, elevador y empujador al instante y la plataforma se queda donde estaba.*"

Una decisión de diseño que sigue abierta

El cableado completo de la placa (107 cables) se leyó para este libro: **el paro solo va al relé; no hay cable del paro hacia un pin del ESP32.** Eso significa que, **hoy, el software no se entera de que se pulsó el paro**: el ESP32 podría seguir mandando órdenes de

movimiento a motores sin energía. Mientras el paro está pulsado no es peligroso, pero **al soltarlo los motores recuperan potencia y podrían moverse de golpe** si el programa conserva la última orden. Es una decisión **abierta entre David y Mario** (responsable del software del robot): ¿se agrega una entrada para que el software detecte el paro y frene antes de que se suelte? **PENDIENTE (P-06).** (La guía vieja del 9 de octubre sí traía un segundo circuito del paro hacia un pin del ESP32; la hoja nueva no lo trae.)

El perro guardián: la segunda red

La hoja dice, del cable USB entre el Jetson y el ESP32: "*Si se desconecta, el watchdog detiene las ruedas en 1 s.*" Un **watchdog** ("perro guardián") es un temporizador que obliga a detener el robot si el programa principal deja de mandar su "latido" a tiempo: protege contra que el cerebro se cuelgue. Ojo: **el programa del ESP32 para este diseño todavía no existe en el repositorio** (`firmware/` solo contiene archivos `.gitkeep`, vacíos que sirven para que git conserve la carpeta). El watchdog es un diseño, no una función que ya corra. El capítulo 5 lo desarrolla.

Medir la batería sin freír un pin

Para que el robot sepa cuánta batería le queda, la hoja usa un **divisor de voltaje**: una resistencia de **100 kΩ** y una de **20 kΩ** en serie, con un capacitor de **100 nF** para limpiar el ruido. Con 14.4 V (batería llena) el punto medio tiene 14.4 × 20 / (100 + 20) = **2.4 V**, un valor seguro para un pin de 3.3 V. Ese punto va al **GPIO 39** del ESP32 (solo entrada analógica). La fórmula de la hoja: **voltaje real ≈ lectura / 4095 × 3.3 × 6**. Regla general: **nunca conectar 12 V directo a un pin de 3.3 V**; se usa un divisor.

Dos caminos de 5 V hacia el mismo ESP32

La hoja alimenta el ESP32 por su pin de **5 V desde el XL4005**, "*no por la computadora*", y a la vez lo conecta por **micro USB al Jetson** para datos. Regla general: cuando una placa recibe 5 V por dos caminos a la vez, las fuentes pueden "pelearse" salvo que haya un diodo o que el cable USB no lleve la línea de alimentación. Cómo se evita aquí no está documentado: **PENDIENTE (P-19)**.

Cables: por qué el grueso importa

La hoja especifica calibres: **14 AWG** para la batería y la rama de motores, **16–18 AWG** hacia los motores-rueda, **18–20 AWG** para drivers y lógica, **22 AWG** para señal. Error que anota: "*Cable de potencia delgado (22 AWG) en la rama de 15 A: se calienta y se derrite.*"

Otra regla: **potencia por un lado y señal por otro**, con un cincho cada ~10 cm, porque "*Cables de señal pegados a los de potencia: los VL53L0X y el RS485 fallan 'al azar'.*"

La celda: 24 V y 5 V separados

La **celda de manufactura** (capítulo 7) tiene su propio mundo eléctrico, según la hoja de conexión y la guía:

- Una **fuente de 24 V del tablero** alimenta el controlador, los dos infrarrojos y la bobina del relé del pistón.
- La **Raspberry Pi 4** se alimenta con **su propia fuente de 5 V**, aparte. "*No hay hilo de 24 V hacia la Pi.*"
- La Pi **no energiza** el pistón: comanda un **relevador** que sí lo hace.

El mismo principio: **cada voltaje en su isla; entre islas se comunica por relé o por red, nunca por cobre directo.** El **paro de emergencia físico de la celda** no aparece en ningún documento del repositorio: **PENDIENTE (P-16)**.

Una lista de errores que queman cosas

Recopilados de la hoja y de la guía:

- Meter 24 V a un pin de la Pi: "*24 V lo quema.*"
- Conectar un motor directo a un pin: un pin entrega unos pocos miliamperios; un motor puede pedir varios amperios.
- Conectar o desconectar un motor a pasos con el driver encendido: quema el A4988.
- Olvidar el capacitor de **100 µF** junto a cada A4988, que amortigua los picos. Con la polaridad al revés "*se infla o revienta*".
- Poner **VDD del driver a 5 V** en vez de 3.3 V: no entiende las señales de 3.3 V del ESP32.
- Alimentar la barra QTR-8A a **5 V** cuando sus salidas van a pines de 3.3 V: "*mandaría 5 V a pines que aguantan 3.6 V.*"
- Alimentar el servo desde el ESP32 en lugar del XL4005: "*al moverse jala corriente y reinicia la placa.*"
- Conectar la batería **al revés** (+ con −): "*quema el ESP32 o el módulo al instante.*"
- Montar un módulo **al revés** en su zócalo.
- Dejar la placa tocando el aluminio: "*un corto en cuanto el carrito vibra.*" (Por eso la bandeja va con separadores M3.)

Normas: qué existe, sin fingir que se cumple

Para máquinas, el paro de emergencia está regulado por normas internacionales: en general **IEC 60204-1** (equipo eléctrico de máquinas) e **ISO 13850** (función de paro de emergencia) (**conocimiento general**, no verificado en el proyecto). Aether es un **prototipo de laboratorio** y no está certificado; lo honesto ante un jurado es decir: *"seguimos los principios (contacto NC, corte de potencia independiente del software), no tenemos certificación."* El propio `cell_hmi.py` usa la misma humildad: su cuenta de pantalla *"no es una certificación del sistema"*.

Seguridad de robots comerciales

En un robot de almacén comercial, el paro de emergencia está **homologado**, hay botones en varios lados, la batería va en una caja con protección contra corto y sobretemperatura, y la potencia (24 V o 48 V) se corta con un **contactor de seguridad** que no depende del cerebro. La lógica es la misma que la del relé gobernado por el paro de Aether, a otra escala y con papeles firmados.

Preguntas de repaso: energía y protecciones

1. ¿Por qué el contacto del paro es "normalmente cerrado" y no "normalmente abierto"?
2. Si el Jetson se cuelga con las ruedas a toda velocidad, ¿qué corta la potencia primero: el programa o el relé del paro? Y si se pulsa el paro, ¿el Jetson se apaga?
3. Una batería de 12 V y 24 Ah: ¿cuántos vatios-hora guarda y por qué la autonomía real será menor?
4. ¿Por qué hay un fusible de 15 A y otro de 10 A, y por qué el de 10 A pasa por un interruptor?
5. En un divisor de 100 kΩ y 20 kΩ, ¿qué voltaje llega al pin con 12.6 V de batería?
6. ¿Qué podría pasar al soltar el paro si el ESP32 no se enteró de que fue pulsado, y qué decisión de diseño queda abierta?

Capítulo 5: Motores, drivers y el ESP32

Cómo se mueven las ruedas, el elevador y el empujador; qué es un driver, un motor a pasos y un bus RS485; qué pin del ESP32 hace qué en el diseño del carrito y con qué herramienta se programa.

El cerebro piensa lento y el músculo reacciona rápido

El robot de Aether lleva dos computadoras con trabajos opuestos. El **Jetson** piensa: ve la cámara, reconoce piezas, decide a dónde ir. Pero Linux, el sistema que corre en el Jetson, no está hecho para responder a un evento en microsegundos: puede "distraerse" un instante con cualquier otro proceso. Mover motores exige lo contrario: pulsos con la separación exacta, contadores leídos a tiempo y frenado inmediato.

Para eso existe el **ESP32**, un **microcontrolador**: un chip que ejecuta **un solo programa**, sin sistema operativo que lo interrumpa, con pines que puede encender, apagar o leer con precisión. Es el "cerebelo" del robot: recibe del Jetson órdenes de alto nivel ("avanza", "sube el elevador") y las convierte en pulsos y señales. El equipo ya tiene un ESP32 en su poder; el diseño de la hoja de armado usa un **ESP32 DevKitC de 38 pines** (tarjeta de desarrollo con chip ESP32, regulador de 3.3 V y puerto micro USB). El modelo exacto de la placa que se tiene: **PENDIENTE (P-07)**.

Tres tipos de actuadores en un solo robot

Actuador	Cuántos	Para qué	Cómo se manda
Motor-rueda M0601 (modelos FIT1042 y FIT1038)	2	Mover el carrito	Por un bus **RS485**: el ESP32 manda mensajes y el motor trae su propio driver y encoder
Motor a pasos NEMA 17 (17HS4401)	3	2 husillos del elevador + 1 piñón del empujador	Pulsos **STEP** y **DIR** a un driver **A4988**
Servo MG90S (180°)	1	La uña del empujador	Una señal **PWM** a un pin

El M0601 y los NEMA 17 son decisiones firmes del equipo (ya cotizados); si ya llegaron o están instalados es **PENDIENTE (P-01)**.

Motor, PWM y driver: la llave y la tubería

Un **motor de corriente continua** (DC) gira más rápido cuanto más voltaje recibe. Pero un pin del ESP32 solo da 3.3 V y unos pocos miliamperios: no puede alimentar un motor. La solución es un **driver**: un circuito de potencia que **obedece** a una señal chica y **entrega** la corriente grande desde la batería. Es como la llave de una manguera: la mano (la señal) es débil, pero controla un chorro enorme.

¿Cómo se controla la velocidad si solo hay "encendido" o "apagado"? Con **PWM** (*Pulse-Width Modulation*, modulación por ancho de pulso): se enciende y apaga el motor miles de veces por segundo y se cambia la **proporción de tiempo encendido**. 25 % encendido ≈ el motor gira a un cuarto de potencia. Es como pulsar el acelerador de un auto muy rápido: el motor "promedia".

El interior de un driver de motor DC es un **puente H**: cuatro interruptores en forma de H que, combinándose, dejan pasar la corriente en un sentido o en el contrario (adelante o atrás). Una guía vieja del equipo (9 de octubre) usaba para esto un **TB6612FNG**, un chip pequeño de ≈ 1 A por motor; la propia guía avisaba que "*si el robot es de tamaño persona, este driver no se usa*". Con 40 kg, no es el driver. En el diseño actual **las ruedas traen su propio driver dentro del motor**, así que el ESP32 no maneja corriente de ruedas; solo manda mensajes.

Las ruedas: órdenes por un bus RS485

El **RS485** es un estándar para comunicar equipos industriales con **dos cables trenzados** (**A** y **B**) a distancias largas y con mucho ruido eléctrico. Varios dispositivos cuelgan del mismo par de cables (un "bus"), cada uno con su **ID**: aquí los dos M0601 son el **ID 1 y el ID 2**.

El ESP32 habla en 3.3 V, pero el chip **MAX485** que traduce a RS485 trabaja a 5 V, así que el diseño añade un **convertidor de nivel** (un traductor 3.3 ↔ 5 V). Cableado exacto, según la hoja:

Señal	Pin del ESP32	Qué hace
DI del MAX485	GPIO 17	Lo que el ESP32 **manda** a los motores
DE + RE (unidos)	GPIO 4	En 1, el MAX485 **habla**; en 0, **escucha**. El ESP32 lo cambia en cada mensaje

Señal	Pin del ESP32	Qué hace
RO del MAX485	GPIO 16 (pasando por el convertidor de nivel)	Lo que **contestan** los motores; sale a 5 V y se baja a 3.3 V antes de entrar

Es un bus **half-duplex**: no se puede hablar y escuchar a la vez, por eso DE y RE van puenteados a un solo pin. Dos reglas de la hoja: **trenzar A con B** y **si los motores no contestan, invertir A y B** (síntoma típico de un RS485 recién armado).

Lo que no se sabe: el **protocolo** de mensajes del M0601 (qué bytes significan "gira a tal velocidad") **no está en el repositorio**. Hay que leer la hoja técnica del fabricante: **PENDIENTE (P-08)**. Sin eso no se puede escribir el programa de las ruedas.

Odometría: saber cuánto avanzó el robot

La hoja dice que "*el ESP32 cuenta las vueltas de los M0601 a 50 Hz para la odometría*". La **odometría** es estimar la posición del robot sumando cuánto giró cada rueda, como el cuentakilómetros de un auto. Una **vuelta** de rueda de Ø 102 mm avanza π × 0.102 ≈ **0.32 m**. Con las dos ruedas separadas **0.285 m** (±0.1425 m del centro, según la tabla de medidas), la cinemática diferencial (**conocimiento general**) es:

- Velocidad hacia adelante del robot: **v = (v_der + v_izq) / 2**
- Giro: **ω = (v_der − v_izq) / 0.285**

El viaje, según la hoja, se limita a **0.35 m/s y 0.4 m/s²**. La odometría **se acumula con error** (las ruedas patinan un poco), por eso el diseño la corrige con el **ArUco** que la cámara ve al llegar a cada estación (capítulo 14) y con una **IMU** (capítulo 6). Los **encoders** (sensores que cuentan cuánto giró el eje) vienen dentro de los M0601.

Motores a pasos: el motor que sabe contar

Un **motor a pasos** (NEMA 17 es solo su tamaño de brida: 42 × 42 mm) avanza un ángulo fijo por cada pulso. El 17HS4401 da **200 pasos por vuelta**. Para que gire más suave y silencioso, el driver **A4988** subdivide cada paso en **micropasos**; el diseño pone los pines MS1, MS2 y MS3 a 3.3 V, que activan el **micropaso 1/16**: **200 × 16 = 3200 pulsos por vuelta**. La hoja lo resume: "*Por eso 3200 pasos = 1 vuelta.*"

El A4988 recibe tres señales por motor:

- **STEP**: cada pulso es un micropaso.
- **DIR**: el nivel (alto o bajo) decide el sentido.

- **ENABLE**: en 0 el motor tiene fuerza (retiene); en 1 queda suelto.

Cuentas con las piezas del carrito

Con la hoja se calcula el tamaño de cada movimiento (**ejemplos de cuenta**, con micropaso 1/16):

- **Empujador:** piñón de módulo 1.5 y 20 dientes, Ø 30 mm → avanza π × 30 ≈ **94.2 mm por vuelta**. Una carrera de **250 mm** son 250 / 94.2 ≈ 2.65 vueltas ≈ **8 490 pulsos**. La hoja dice que la cremallera se mueve a **80 mm/s**: eso son 80 / 94.2 × 3200 ≈ **2 700 pulsos por segundo**, y la carrera completa dura ≈ 3 s.
- **Elevador:** un nivel son **70 mm**. Con un husillo de paso **P** mm (lo que avanza por vuelta), son 70 / P vueltas y 70 / P × 3200 pulsos. Si P = 8 mm: **28 000 pulsos**; si P = 2 mm: **112 000 pulsos**. La diferencia es enorme, por eso la hoja dice "*medir el avance al recibirlos*". **El paso real del husillo TR8 es PENDIENTE (P-09)** y cambia todo el programa del elevador.

Los **dos motores del elevador** comparten STEP, DIR y ENABLE: un mismo pulso llega a ambos drivers, así que giran **exactamente igual**. Funciona mientras ninguno pierda pasos; no hay sensor que compare los dos. La única corrección son los **finales de carrera**.

El mapa de pines del ESP32 (diseño, revisión 3.8)

Este es el "guion" que deberá respetar el programa del ESP32, leído del cableado de la placa cable por cable. **No está probado en hardware**: es diseño.

Pin (GPIO)	Función	Va a
25	STEP del elevador	A4988 #1 y #2
26	DIR del elevador	A4988 #1 y #2
27	STEP del empujador	A4988 #3
14	DIR del empujador	A4988 #3
13	ENABLE (de los 3 drivers)	A4988 #1, #2 y #3, con resistencia de 10 kΩ a 3.3 V
17 / 4 / 16	RS485: DI / DE+RE / RO	MAX485
21 / 22	I^2C: SDA / SCL	VL53L0X, PCF8574, BNO055 (capítulo 6)
18 / 19 / 23	Finales de carrera 1, 2 y 3	Elevador izquierdo, elevador derecho, home del empujador
32, 33, 34, 35, 36	Salidas S3 a S7 de la barra QTR-8A	Barra de línea (capítulo 6)

Pin (GPIO)	Función	Va a
39	Voltaje de la batería	Divisor 100 kΩ / 20 kΩ (capítulo 4)
5	PWM del servo	MG90S de la uña
2	Datos de la tira WS2812 (opcional)	Resistencia de 330 Ω a la tira de LEDs

Detalles de ingeniería que esa tabla esconde:

- **La resistencia de 10 kΩ en ENABLE.** Al encender el ESP32, sus pines tardan unos instantes en "tomar posición". Con ENABLE atado a 3.3 V, los motores **quedan sueltos** durante el arranque; así "*no hay movimientos raros al encender*" (palabras de la hoja).
- **Finales de carrera NC con pull-up.** Si el cable se rompe, el pin lee igual que "presionado" y el motor se detiene en lugar de chocar: la misma idea de falla segura del paro de emergencia.
- **Pines solo de entrada.** En el ESP32, los GPIO 34, 35, 36 y 39 **solo pueden leer**; por eso se asignan a la barra y a la batería (**conocimiento general** del ESP32).
- **Pines con trampa al arrancar.** Algunos pines (como el 2 y el 5) afectan cómo arranca el chip; la hoja anota que los LEDs van sin convertidor de nivel "*si no, el ESP32 no se deja programar*". Conviene dejar esas líneas desconectadas al subir un programa (alerta general, no medida).

El perro guardián y el contrato con el Jetson

Entre el Jetson y el ESP32 va un **cable USB de datos** ("*los de solo carga no sirven*", advierte la hoja). Por ahí viaja el **contrato**: qué mensajes se mandan y cómo. Ese contrato **no está escrito** para este hardware. Existe de ejemplo el de la guía vieja (del diseño con el TB6612): líneas de texto terminadas en salto de línea, `H` como **latido**, `M <izq> <der>` para velocidades de −100 a 100 y `S` para frenar; el ESP32 frenaba solo si pasaban **300 ms sin latido**. Es una buena plantilla, pero con otros motores, otro bus y otra lógica habrá que escribir el contrato nuevo. La hoja pide un watchdog de **1 s**. Queda por definir (**PENDIENTE (P-23)**): qué mensajes existen para ruedas, elevador, empujador, uña y lecturas; cada cuánto manda el Jetson el latido y a los cuántos milisegundos frena el ESP32; y qué hace el ESP32 cuando se pulsa el paro (capítulo 4, **P-06**).

Decisión: el ESP32 se programa con PlatformIO

El equipo decidió programar el ESP32 con **PlatformIO**. Es una extensión de **Visual Studio Code** (el editor de código) que compila y sube programas a microcontroladores, usa las mismas librerías del ecosistema **Arduino** y fija las versiones de cada librería dentro del proyecto. Las razones técnicas que respaldan la decisión (**conocimiento general**):

- Existen librerías ya hechas para los componentes del diseño: motores a pasos con aceleración, sensores **VL53L0X**, la IMU **BNO055** y el expansor **PCF8574**.
- El proyecto vive dentro del mismo repositorio de Git, con versiones de librerías reproducibles; con un editor básico de Arduino cada persona podría compilar con versiones distintas.
- Las alternativas se descartaron por costo de aprendizaje o de control: **ESP-IDF** (kit oficial en C, mayor control pero curva alta para una persona con tiempo limitado) y **MicroPython** (más lento y con temporización crítica de pulsos más difícil).

Un archivo de configuración de PlatformIO es de este estilo (**ejemplo genérico; no existe todavía en el repositorio**):

```
; platformio.ini  (ejemplo genérico)
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    ; aquí van las librerías con su versión fija
```

Comandos típicos de PlatformIO, ejecutados en `[Laptop · terminal integrada de VS Code, dentro de la carpeta del proyecto del firmware]` (**no probados para este proyecto**):

```
pio run                 # compila
pio run -t upload       # compila y sube al ESP32 conectado por USB
pio device monitor      # abre el monitor serie
```

Si el ESP32 no se deja programar, la causa más común es un cable USB de solo carga o un pin con trampa de arranque (sección anterior).

Probar sin riesgo

Una guía vieja del equipo lo dejó bien dicho y sigue valiendo: *"Las ruedas no se prueban en el suelo hasta que S y el botón de paro las hayan detenido en el aire."* Secuencia segura que se desprende de la hoja:

1. **Con el robot levantado**, ruedas sin tocar el piso.
2. Prueba de humo de la electrónica de potencia (capítulo 4).
3. Ajustar el **Vref** de cada A4988 **sin motor conectado**.
4. Mover el empujador **a mano** su recorrido de 25 cm y revisar que no roce cables.
5. Primero lógica (USB), después potencia, después movimiento. Un cambio a la vez.

Controladores de movimiento en la industria

En robots comerciales, el nivel bajo (motores, encoders, seguridad) lo ejecuta un **controlador de movimiento** dedicado y el nivel alto (percepción, planificación) corre en una computadora distinta; se comunican por **CAN bus**, **EtherCAT** o **RS485/Modbus**. Aether usa la misma idea con piezas de laboratorio: ESP32 como controlador de movimiento y RS485 para las ruedas. Y la **seguridad** (paro) siempre se cablea aparte del software.

Preguntas de repaso: motores y microcontrolador

1. ¿Por qué un ESP32 maneja mejor los pulsos de un motor a pasos que el Jetson?
2. ¿Qué diferencia hay entre STEP, DIR y ENABLE en un A4988?
3. Con 3200 micropasos por vuelta y un piñón que avanza 94.2 mm por vuelta, ¿cuántos pulsos necesita el empujador para recorrer 250 mm?
4. ¿Por qué DE y RE van unidos en el MAX485 y qué significa "half-duplex"?
5. ¿Para qué sirve la resistencia de 10 kΩ en el pin ENABLE al encender el robot?
6. Si el paso real del husillo resulta ser de 2 mm y no de 8 mm, ¿qué parte del programa del elevador cambia y por cuánto?

Capítulo 6: Sensores y cámaras

Cómo "siente" Aether: una cámara que lee códigos, sensores de distancia que frenan antes de chocar, una brújula giratoria, una barra que ve el piso y los interruptores que dicen "ya llegué".

Una cámara que mintió sobre su resolución

El 21 de octubre de 2026, durante la preparación del Jetson, apareció algo que le pasa a casi todo el que programa cámaras por primera vez. Al abrir la cámara con `cv2.VideoCapture(0)`, OpenCV (la librería de visión por computadora de Python) entregó un cuadro de **1536 × 2304** píxeles. Pero la cámara USB **no tiene** ese modo; sus modos reales son 640 × 480, 1920 × 1080, etc. Sin que nadie se lo pidiera, OpenCV abrió la cámara por **GStreamer** (un sistema de tuberías de video de Linux), que inventó un cuadro con un formato raro. Al forzar el backend **V4L2** (el controlador estándar de cámaras de Linux), llegó el cuadro correcto de **480 × 640**.

Esto está **VERIFICADO** en la bitácora del equipo y explica que, en la primera prueba del backend en el Jetson, el panel transmitiera a 2304 × 1536 (el cuádruple de píxeles necesarios) y se viera lento. La solución, ya en el repositorio (commit `c9f24cb`): variables de entorno opcionales `PERCEPTION_CAMERA_BACKEND`, `_FOURCC`, `_WIDTH` y `_HEIGHT` (y las equivalentes `LINE_VISION_CAMERA_*` de la celda) para forzar V4L2, MJPG y 640 × 480. Moraleja para cualquier proyecto con cámaras: **nunca asumir qué entrega el sistema; preguntar y medir.**

La cámara del robot

La **cámara** del diseño es una **Logitech 1080p USB** que Mario (responsable del software del robot y de la visión) ya tiene, montada **arriba, al frente**, mirando hacia la banda. Su trabajo, según la hoja de armado: **leer el QR de cada fixture** (el código viene en el travesaño trasero) y **los ArUco de las estaciones**, y alimentar a YOLO (capítulo 14). En el Jetson, esta cámara aparece así (**VERIFICADO**, salida reportada en la bitácora):

Dato	Valor
Nombre	"1080P Pro Stream" (puerto `usb-3610000.usb-2.3`)
Dispositivos	`/dev/video0` (captura) y `/dev/video1` (solo metadatos)
Identificador USB	`046d:0894` (en `lsusb` sale como Logitech CrystalCam; es la misma cámara)

Dato	Valor
Formatos	YUYV y MJPG; MJPG de 160 × 90 hasta 1920 × 1080 a 30 FPS

Comandos para repetir esa revisión, `[Jetson · SSH]`:

```
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats-ext
```

El primero lista las cámaras; el segundo, todos los modos que ofrece cada una.

FPS, exposición y los cuadros que se caen

Un **FPS** (*frames per second*, cuadros por segundo) mide cuántas imágenes entrega la cámara cada segundo. El agente de percepción del Jetson midió **29.0 FPS de cámara y 29.1 de entrega**, a 640 × 480, con una inferencia de YOLO de **68.0 ms** de promedio (**MEDIDO**, reportado por Mario el 23 de octubre).

Antes, en una prueba con **poca luz** y la cámara sin acomodar, la captura bajó a **10 FPS** con Python y a **15.3 FPS** con `v4l2-ctl`. La razón es un clásico: con poca luz, la cámara **alarga el tiempo de exposición** (mira más tiempo cada cuadro para juntar luz) y entrega menos cuadros por segundo. La opción `exposure_dynamic_framerate` permite que la cámara baje los FPS automáticamente si hace falta luz. Al apagarla **solo como prueba**, el promedio subió pero no se estabilizó: no es una medición representativa. **No se fija nada de exposición desde el código**; la medición buena, con luz real y la cámara en su posición final, queda **PENDIENTE (P-22)**.

Mensaje práctico: **la luz es un componente del sistema**, como la batería. Un robot que "anda perfecto en la mesa" y "se ve ciego" en el pasillo oscuro tiene un problema de iluminación, no de software.

ArUco y códigos: señales que el robot sabe leer

Un **ArUco** es un cuadrado blanco y negro con un patrón único, parecido a un código QR pero más simple, diseñado para que una cámara calcule **dónde está y hacia dónde mira**. Aether lo usa para **verificar su posición** al llegar a la banda y a la estación de descarga (el robot navega por **waypoints**, puntos de ruta, y el ArUco corrige el error acumulado). Los **códigos de barras y QR** se leen con la librería `pyzbar` y sirven para identificar cada fixture. El tamaño impreso de los marcadores y la distancia a la que deben leerse **no están en ningún documento**: **PENDIENTE (P-11)**.

Un hallazgo del propio proyecto, documentado en el informe integrador: el ArUco **puede dar un identificador incorrecto pero válido cuando hay reflejos de luz**, sin señal de error visible (capítulo 14).

El bus I²C: seis dispositivos, dos cables

Varios sensores del diseño comparten un **bus I²C** (*Inter-Integrated Circuit*): **dos cables** (**SDA** para datos y **SCL** para el reloj) por donde el ESP32 habla con muchos dispositivos, cada uno con una **dirección** única. Es como un grupo de mensajería donde cada persona tiene su nombre. En el diseño, los pines son el **GPIO 21 (SDA)** y el **GPIO 22 (SCL)** del ESP32, y las direcciones son:

Dispositivo	Dirección I²C	Qué hace
PCF8574 (expansor de pines)	`0x20`	Da 4 salidas extra para el truco de los VL53L0X
BNO055 (IMU)	`0x28`	Mide el giro del robot
VL53L0X frontal	`0x30`	Distancia al frente
VL53L0X lateral izquierdo	`0x31`	Distancia a la izquierda
VL53L0X lateral derecho	`0x32`	Distancia a la derecha
VL53L0X de la pila	`0x33`	¿Hay un fixture arriba de la torre?

La hoja pide una comprobación con un **escáner I²C** (un programita que pregunta a todas las direcciones quién contesta): "*El escáner I²C lo encuentra en 0x20*", etc. Si el escáner no ve un dispositivo, el problema es de cableado, no de programa.

VL53L0X: medir distancias con un rayo de luz

El **VL53L0X** es un sensor de **tiempo de vuelo** (*Time-of-Flight*, ToF): manda un pulso de luz infrarroja, espera a que rebote en un objeto y, con el **tiempo** que tardó, calcula la distancia (la luz viaja a 300 000 km/s, así que hace falta un reloj muy rápido). El diseño pone **cuatro**:

- **Tres de obstáculos** a unos **15 cm del piso**: uno al frente y uno a cada lado. Si ven algo a **menos de 40 cm**, **frenan el carrito sin esperar al Jetson**: es una capa de seguridad **reactiva** (decide el ESP32 al instante, sin mandar nada por USB).
- **Uno arriba de la torre**, mirando hacia abajo, que **confirma que hay un fixture** en la pila (presencia, con ±25 mm).

Un problema típico: **los cuatro sensores vienen con la misma dirección de fábrica**, así que en el mismo bus chocarían. La solución del diseño: cada sensor tiene un pin **XSHUT** (apagado). Al arrancar, el ESP32 los apaga a todos mediante el **PCF8574** (salidas P0 a P3), los **prende uno por uno** y a cada uno le cambia la dirección a `0x30`, `0x31`, `0x32` y `0x33`.

La hoja pide verificar "*Mide bien a 10, 30 y 60 cm*" con una regla. Una advertencia práctica: **los cables de señal pegados a los de potencia hacen que los VL53L0X y el RS485 fallen "al azar"**. El alcance máximo y la precisión son datos del fabricante (**conocimiento general**: del orden de un par de metros); no están medidos en el proyecto.

BNO055: una brújula de giro (sin brújula)

El **BNO055** es una **IMU** (*Inertial Measurement Unit*, unidad de medición inercial): un chip con **acelerómetro** (mide aceleraciones), **giroscopio** (mide cuánto gira) y, en su modo completo, **magnetómetro** (mide el campo magnético, como una brújula). En el diseño sirve para **corregir la odometría** de las ruedas: las ruedas pueden patinar y "creerse" que giraron 90° cuando giraron 85°; el giroscopio no se deja engañar.

Detalle de oficio: la hoja pide el **modo IMU, sin magnetómetro**, "*porque los motores y la batería lo confunden*": los imanes y las corrientes grandes alteran el campo magnético alrededor del robot. Comprobación del diseño: "*al girar el carrito 90° marca ~90°*". Y una nota de compra: el módulo adquirido trae sus 8 pines en una sola fila, distinto del modelo de Adafruit dibujado en la hoja; hay que leer la serigrafía del módulo real antes de soldar.

QTR-8A: una barra que ve el piso (de respaldo)

La **QTR-8A** es una barra con **8 sensores infrarrojos** que miran el piso y devuelven un valor **analógico** (un voltaje entre 0 y 3.3 V) según qué tan oscura sea la superficie: cinta negra sobre piso claro, por ejemplo. Va **debajo del frente, a 3–5 mm del piso**, alimentada a **3.3 V (nunca 5 V)**, y usa solo **5 de los 8** (las del centro: salidas 3 a 7, GPIO 32 a 36). Es un **plan B**: "*seguir una línea en el piso si la navegación por waypoints falla*".

Finales de carrera: "ya llegué" con un clic

Los **tres finales de carrera** son interruptores mecánicos que el carro pisa al llegar al límite. Dan el **cero** (*home*): el programa mueve el motor hasta que el interruptor se activa y a partir de ahí **cuenta pasos**. Van como **contacto normalmente cerrado (NC)** con **pull-up** (una resistencia que "tira" el pin hacia 3.3 V): "*un cable roto se lee como 'presionado' y el motor se detiene*". Es la misma idea de **falla segura** del capítulo 4.

Los infrarrojos de la celda

La celda tiene **dos infrarrojos industriales** (uno en la estación de visión y otro después del pistón), de los que habla el capítulo 7. Aquí solo conviene recordar la idea central del capítulo 2: **PNP o NPN**, y **24 V, no 5 V**.

Lo que no existe en el diseño

- **Lidar:** una guía vieja contemplaba un lidar 2D por USB ("cuando se compre") y reconocía que "*no existe todavía el programa de ruta*". **La hoja de armado revisión 3.8 no lo incluye.** Hoy la prevención de choques usa los tres VL53L0X y la navegación es por waypoints con ArUco. Si se comprará un lidar y qué sensores quedan definitivos: **PENDIENTE (P-10)**.
- **Micrófono y bocina del robot:** el micrófono de Lumina (la cara y voz del robot, capítulo 15) es el de la laptop. Si el robot tendrá micrófono propio **no se sabe**: **PENDIENTE (P-20)**.
- **Medición de corriente de la batería:** solo existe el divisor de voltaje del capítulo 4.

Calibración: el mundo físico no es ideal

Todos los sensores de este capítulo tienen un **error propio**: el VL53L0X lee unos milímetros de más o de menos según el color del objeto; el giroscopio "se desvía" lentamente; la cámara cambia con la luz; las ruedas patinan. Por eso casi cada comprobación de la hoja termina en una medición con regla o multímetro: "*Mide bien a 10, 30 y 60 cm*", "*marca ~90°*", "*5.0–5.1 V sin carga*". **Un buen programa deja perillas para ajustar** (desfases, ganancias, umbrales) en vez de asumir que el sensor es perfecto.

Capas de sensado en robots comerciales

Los robots móviles comerciales combinan **capas**: una **capa reactiva rápida** (sensores de distancia y bumpers que frenan al instante), una **capa de navegación** (mapas, **lidar**, odometría y cámara) y una **capa de seguridad certificada**. El diseño de Aether sigue esa lógica a escala de laboratorio: VL53L0X (reactiva), cámara + ArUco + odometría (navegación) y el paro físico (seguridad). Los productos comerciales añaden **lidar de seguridad certificado** con zonas de frenado configurables.

Preguntas de repaso: sensores y cámaras

1. ¿Por qué una cámara entrega menos FPS con poca luz y qué cambia al poner una lámpara?
2. ¿Qué es una dirección I^2C y por qué los cuatro VL53L0X necesitan el truco del XSHUT?
3. ¿Qué ventaja tiene que el ESP32 frene el robot al ver algo a menos de 40 cm, en lugar de que lo decida el Jetson?
4. ¿Por qué se usa el BNO055 en modo IMU (sin magnetómetro) en un robot con motores y batería?
5. ¿Qué valor lee un final de carrera normalmente cerrado si se rompe su cable?
6. ¿Qué dato falta hoy para que el ArUco sirva de verdad: tamaño impreso, distancia o iluminación?

Capítulo 7: La celda de manufactura

Banda, fixtures, cámara, relevadores, un controlador LOGO y un pistón: qué hace cada cosa, qué dice el código y qué quedó de los planes anteriores con PLC.

Dos historias de la misma celda

En la documentación de hardware del repositorio hay dos relatos de cómo funciona la celda. La **hoja de conexión** del 5 de octubre cuenta que una Raspberry Pi 4 lee un **PLC** (*Programmable Logic Controller*, el cerebro industrial de los tableros) por **Modbus TCP** y que "*no hay ningún hilo entre el PLC y los GPIO de la Pi*". El **código** (`backend/cell_line.py`) cuenta otra: la Pi enciende **dos relevadores** con sus propios pines (**GPIO 8** para el pistón y **GPIO 9** hacia la entrada **I4 de un LOGO**, un mini-PLC), y el LOGO mueve la banda.

El equipo resolvió la contradicción: **el hardware real de la celda es por GPIO, con relevadores hacia el LOGO, como dice el código.** La parte de PLC por Modbus de los documentos queda como **plan o documentación vieja** (se resume en una sección de este capítulo). La auditoría del 22 de octubre ya había registrado la discrepancia como cuatro versiones distintas; la decisión la cierra a favor del código. Lo que sigue **PENDIENTE (P-12)** son los detalles físicos que ningún documento confirma: qué pines y qué tipo de placa de relevadores se usan en realidad y qué modelo de LOGO.

El equipo detrás de la celda

El proyecto lo desarrollan seis personas con áreas separadas. En la celda intervienen:

Persona	Área	Qué falta confirmar con esa persona
Mario	Todo el software del robot y la visión	—
Isaac	Diseño mecánico, CAD e impresión 3D (la banda y los fixtures)	Dimensiones oficiales y material de los fixtures (P-21)
Pedro	El pistón de scrap (la pieza que expulsa lo rechazado)	Tipo de pistón, electroválvula y tiempos reales (P-14)
Uriel	Comunicación de la Raspberry con el PLC y con la visión	Detalle del cableado y de la comunicación (P-12)
David	Ensamble mecánico y electrónica	Relevadores, fuente y paro de emergencia (P-16)

Persona	Área	Qué falta confirmar con esa persona
Emmanuel	HMI/GUI (la pantalla de operación) aparte	Alcance de la pantalla de la celda (P-17)

La cadena física, de la banda al scrap

La idea, que viene de la **ADR 0010** (un documento corto del equipo que registra una decisión de arquitectura): una **banda transportadora** lleva piezas, cada una sobre un **fixture** (la base que la sostiene en posición). Frente a la **cámara** la pieza se detiene un instante; la **visión** decide si es **PASS** (pieza real registrada) o **FAIL** (no lo es); si es FAIL, un **pistón** la empuja fuera de la banda (*scrap*, desecho); si es PASS, la banda sigue y la recoge el robot.

Lo que dice el código sobre quién hace qué:

- **La Raspberry Pi 4** corre `celda_agent.py`: abre la cámara, corre el modelo de visión, decide y **enciende los dos relevadores** (pistón y banda).
- **El LOGO** (mini-PLC) mueve la banda. El código lo dice así: "*El LOGO, no este archivo, ignora el infrarrojo 5 s al ver I4*". Es decir: el LOGO tiene un infrarrojo conectado a sí mismo y, cuando la Pi manda el pulso a su entrada **I4**, avanza la banda e **ignora el sensor 5 s** para no volver a detectar la misma pieza. Marca y modelo exactos del LOGO: **PENDIENTE (P-12)**.
- **El pistón** es un cilindro neumático (un émbolo movido con aire comprimido) accionado por una **electroválvula** que abre el aire cuando su **relevador** recibe el pulso de la Pi.

Qué decide la visión: un PASS que no es lo que parece

Una honestidad que el propio código declara. En `backend/dataset_store.py`, la función `clasificar_pieza` dice: "*PASS significa que el nombre está en clases, no que la pieza esté libre de defectos.*" Es decir:

- El modelo reconoce **tres piezas registradas**: `computadora1`, `computadora2` y `computadora3` (**VERIFICADO**, archivo `classes.txt`). Son las **ECU** (computadoras de motor) que viajan en los fixtures.
- **PASS** = el modelo vio una de esas tres con **confianza ≥ 0.50** y sin ambigüedad (la segunda candidata queda al menos **0.10** por debajo).
- **FAIL** = el modelo vio algo que **no es una pieza registrada** (o dudó entre dos).
- **SIN_PIEZA** = el modelo no vio nada: no hace nada.

No hay inspección de defectos todavía. Una ECU rayada o con un conector roto pasaría como PASS. El criterio real de calidad es una decisión pendiente de diseño: **PENDIENTE (P-24)**. Lo honesto ante un jurado es decir "*la celda distingue piezas registradas de objetos que no lo son*", no "*la celda inspecciona calidad*".

Cifras del modelo (**MEDIDO**, `parts_report.json`, entrenamiento del 15 de octubre de 2026): **mAP50 = 0.995**. El **mAP50** es una nota de qué tan bien el modelo localiza y acierta objetos (1.0 es perfecto). Con un dataset tan chico (el repositorio conserva 19 fotos en la bandeja de entrada), esa nota es **optimista**: el modelo no ha visto la variedad de ángulos, luces y fondos del mundo real. La bitácora midió además que el modelo exportado a **ONNX** juzgó **igual que el original en 19 de 19 fotos** (IoU medio 0.994), pero que **no se probó con objetos que no sean pieza**, así que el FAIL real **no está medido** con este modelo.

Qué pasa con una pieza, segundo a segundo

Los números salen de las variables de entorno **por defecto** de `cell_line.py` (**VERIFICADO** en código; **NO MEDIDO** en una Pi real):

1. **La visión juzga cada 0.5 s** (`LINE_ANALISIS_CADA_S`) un solo cuadro con el modelo ONNX.
2. **Se exigen 5 juicios iguales seguidos** (`LINE_ESTABLES`) antes de actuar. Así un parpadeo de luz no dispara el pistón. Tarda unos 2.5 s más lo que dure cada análisis. La documentación avisa: "*No bajar de 3 sin probar: un FAIL falso dispara el pistón.*"
3. **Una pieza dispara una sola vez**: tras actuar, el sistema se "desarma" hasta que ve **SIN_PIEZA**; así la misma pieza no se juzga dos veces.
4. **Si es FAIL:** espera **3 s** (`LINE_PISTON_ESPERA_S`: lo que tarda la pieza en llegar frente al pistón), **enciende el pistón 1 s** (`LINE_PISTON_ON_S`), lo apaga y espera **2 s** a que **regrese** (`LINE_PISTON_REGRESO_S`).
5. **Después, pulso de banda de 0.5 s** (`LINE_BANDA_ON_S`) por el GPIO 9: avanza la banda. En **PASS** este pulso sale **al terminar el análisis**, sin esperar al pistón.
6. Se anota el evento: `scrap_expulsado` (FAIL con todo en orden) o `pieza_aceptada` (PASS con todo en orden).

Un FAIL completo suma, con los valores por defecto: **≈ 2.5 s de estabilización + 3 s + 1 s + 2 s + 0.5 s ≈ 9 s** por pieza (estimación a partir de las constantes; **NO MEDIDA**). Todos esos tiempos se calibran **en la banda real** (**PENDIENTE (P-14, P-15)**); los números del código son puntos de partida, no la verdad.

Los registros de cada pieza

Cada pieza deja un registro con `resultado` (**PASS** o **FAIL**), `accion` y la hora. Las acciones que escribe el código:

Acción	Cuándo ocurre
`pieza_aceptada`	PASS y el pulso de banda salió
`scrap_expulsado`	FAIL, el pistón disparó, regresó y la banda avanzó
`fallo_actuacion`	FAIL pero algo falló al actuar (por ejemplo, un pulso no pudo enviarse)
`pulso_no_enviado`	PASS pero el pulso de banda no pudo enviarse

Dos filosofías que enseña la tabla: **no se finge** (si no hay forma de mover el relevador, el registro lo dice) y **un FAIL nunca se reporta como aceptado** (si la visión dijo FAIL y el pistón no actuó, es `fallo_actuacion`).

Sin `gpiod`, nada se mueve

`cell_line.py` solo mueve un pin si puede importar la librería `gpiod` (la que habla con los pines del chip en Linux moderno). Si no la encuentra, `poner()` devuelve `False` sin tocar nada y el código **no finge** que algo se movió. **Hoy esa librería no está instalada** ni en el Jetson (**VERIFICADO**: `python -c "import gpiod"` falla) ni se ha probado en la Pi. Es una protección involuntaria buena: mientras no se instale, ningún botón de la pantalla puede mover un pin.

La bitácora lo verificó leyendo el código y reproduciéndolo con las clases reales en una laptop: **ARRANQUE y PARO solo cambian el interruptor interno** (`running`); PARO además **aborta el ciclo en curso y apaga las salidas**; y el único camino a un pin es la decisión de la cámara tras **cinco juicios iguales**, nunca un botón. La regla escrita en `docs/guia_arranque.md`: "*No instalar `python3-libgpiod` hasta confirmar el cableado.*" Como el cableado por GPIO ya está decidido, el siguiente paso es medir con LED y multímetro (capítulo 8) antes de instalarla.

Una trampa de nombres: en el JSON de estado, `sources.plc` **no significa que haya un PLC**: es `salidas.abiertas`, es decir, "¿se abrió el chip GPIO?". Sin `gpiod` siempre es `false`, y por eso la pantalla muestra la alarma **"Enlace PLC perdido"** (la bitácora la marcó como *esperada: no hay PLC*). Es un nombre heredado de la versión PLC del diseño.

Estados, alarmas y sesión: la pantalla de la celda

La página `celda.html` (que sirve **la propia Pi**, no la laptop) muestra el estado. El **estado** (`state`) puede ser:

- **OPERANDO**: la celda está activa, la cámara viva y hay un ciclo permitido.
- **DETENIDA**: o el operador pulsó **Paro** ("Paro solicitado por el operador") o la cámara no ha juzgado ningún cuadro en los últimos **2 s** ("Cámara apagada").

(Un tercer estado, `SIN_CONEXION`, existe en la hoja de conexión del PLC, pero `cell_line.py` no lo produce.)

`cell_hmi.py` añade **alarmas** al estilo de la norma **ISA-18.2** (la que ordena cómo se muestran y reconocen las alarmas en planta): cada una con **prioridad, causa y acción**, y **viven en el servidor** (sobreviven si se recarga la página). Son tres: "controlador no responde", "visión en FAIL y el pistón no disparó" y "detenida". La **sesión** usa una cuenta única de laboratorio (la norma **IEC 62443** como referencia): quien manda el ciclo debe haber iniciado sesión. **La contraseña no está en el código** (antes era una cuenta fija `admin` / `admin`, ya eliminada); se toma de la variable de entorno `CELDA_HMI_CLAVE`. Sin esa variable *nadie* puede iniciar sesión y Arranque/Paro no responden. Hallazgo real de `iniciar_aether.ps1 -SoloComprobar`: el archivo `backend\.env` de la laptop **todavía no la tenía definida**. El módulo mismo aclara su alcance: *"No es una certificación del sistema. El paro de emergencia no pasa por aquí."*

El alcance final de la pantalla (quién la opera, si lleva monitor propio) es de **Emmanuel**: **PENDIENTE (P-17)**.

La celda se opera sola

El diseño exige que **la laptop sea el servidor central, pero que la celda tenga su propio arranque y paro y pueda operarse sola.** El código lo respalda:

- `celda_agent.py` arranca **solo la celda** (cámara, visión, pulsos, alarmas y `celda.html`); **no importa** Lumina, la base de datos, `torch` ni `ultralytics`. Una prueba automática lo comprueba.
- Sirve **su propio video** (`/line/vision/stream`) y su **propia página**; el video **no pasa por la laptop**.

- **Decide y actúa aunque la laptop o la red se caigan.** A la laptop solo le manda **eventos** (`POST /line/cell/events`) con un identificador único por pieza, en una **cola con reintento**; allá solo se **anotan**, no accionan nada.
- Los botones **Arranque y Paro** de la pantalla viven en la Pi (cuando hay sesión).

Lo que **no existe** en los documentos: un **arranque o paro físico** (un botón de tablero) independiente de la pantalla, ni el **paro de emergencia físico** de la celda: **PENDIENTE (P-16)**.

La Pi 4 corre la visión con un truco para no ahogarse

Una Pi 4 es mucho más modesta que una laptop. Para no saturarla, `celda_agent.py` hace tres cosas, medidas en una **laptop** (no en la Pi):

- Un **solo modelo** en lugar de dos.
- **Un análisis cada 0.5 s** en vez de cada cuadro.
- El modelo corre en **ONNX** con `cv2.dnn` de OpenCV, sin `torch` (que pesa mucho).

Resultado (**MEDIDO en la laptop**, 8 s con cámara, sin nadie viendo el video): el pipeline anterior de `main.py` gastaba **381 % de un núcleo** con unos 6 análisis por segundo; el agente nuevo, **18 %** con 2 análisis por segundo. **Falta medirlo en la Pi real**: **PENDIENTE (P-18)**. Un hallazgo curioso: con DirectShow (Windows) `cap.grab()` no espera al siguiente cuadro y daba 1.5 millones de vueltas por segundo ocupando un núcleo entero; con `read()` sí espera.

El plan con PLC y Modbus (documentación vieja)

La hoja de conexión del 5 de octubre y la guía describen un mapa de **señales Modbus** (`rechazar`, `banda_marcha`, `rpi_vivo`, `plc_vivo`, `accion`, `pieza_id`) con direcciones de ejemplo y una columna "Real" **vacía**. **Modbus TCP** es un protocolo industrial para leer y escribir registros de un PLC por Ethernet (puerto 502). La carpeta `edge/manipulation/line_actuation/plc_backend/` está vacía a propósito y el código **no consume** la variable `CELL_AGENT_URL` de ese plan. Es decir, el mapa de Modbus fue un **plan** para una versión con PLC, no lo que corre. Queda de referencia por si algún día la celda cambia a un PLC.

Documentos viejos contra código

Tema	Documento viejo	Código real (gana)
Quién enciende el pistón	PLC por Modbus; la Pi no lo energiza	La Pi, por GPIO 8 y un relevador
Quién mueve la banda	PLC	El LOGO, avisado por GPIO 9 → entrada I4
Criterio PASS/FAIL	"Hoy solo detecta presencia" (ADR 0010, hoja de conexión)	Ya distingue piezas registradas de otros objetos (`clasificar_pieza`); defectos: no
`tipo_objeto` (1, 2 o 3)	"No existe" (ADR 0010)	Existe: es la posición de la clase en `classes.txt` + 1
Dónde se ve `celda.html`	La laptop	La propia Pi

Probar la celda sin riesgo

1. En una laptop, **sin** `gpiod`: la celda corre completa, la pieza se juzga y se anota como `pulso_no_enviado` o `fallo_actuacion`. **Nada se mueve.**
2. Correr las pruebas, `[Laptop · PowerShell]`, desde la raíz del proyecto: `backend\.venv\Scripts\python.exe -m pytest tests\backend\test_cell_line.py tests\backend\test_celda_agent.py -q`. (Esos archivos de pruebas existen; las pruebas de `cell_line` usan una espera falsa y relevadores falsos.)
3. Para la Pi, **primero** LED y multímetro (capítulo 8), **después** instalar `python3-libgpiod`.

Celdas de inspección en planta

Las celdas industriales de inspección usan **PLC certificados** para la lógica de seguridad y movimiento, y una **PC o cámara inteligente** para la visión; se comunican por **Modbus, PROFINET o EtherNet/IP**, y el **paro de emergencia es un relevador de seguridad** cableado, nunca una pantalla. La celda de Aether sigue ese principio: la visión corre en la Pi y la potencia se maneja con relevadores y un LOGO.

Preguntas de repaso: la celda

1. ¿Por qué `PASS` en esta celda no significa "sin defectos"? ¿Qué se le diría a un jurado?
2. ¿Qué evita que un parpadeo de luz dispare el pistón?

3. Con los valores por defecto, ¿cuánto dura un FAIL completo y qué parte se afina con la banda real?
4. ¿Por qué `sources.plc` es `false` aunque no haya ningún PLC, y qué alarma provoca?
5. ¿Qué diferencia hay entre el botón "Paro" de la pantalla y un paro de emergencia?
6. ¿Qué papel cumple cada una de las seis personas del equipo en la celda?

Capítulo 8: La Raspberry Pi 4 y la seguridad eléctrica del GPIO

Para qué sirve la Pi de la celda, cómo se instala sin torch, por qué un pin puede disparar el pistón al encender y qué se mide antes de dejar que el código toque algo real.

Una computadora del tamaño de una tarjeta, con pines que tocan el mundo

La **Raspberry Pi** es una computadora completa en una placa del tamaño de una tarjeta: procesador tipo móvil (ARM), memoria, puertos USB y de red, ranura para tarjeta **microSD** (donde vive el sistema) y, en un costado, **40 pines** llamados **GPIO** (*General Purpose Input/Output*, entrada y salida de propósito general) con los que puede leer un botón o encender un relevador. Corre **Linux**: el sistema oficial se llama *Raspberry Pi OS* y su versión actual se basa en **Debian 12 "Bookworm"**.

En Aether aparecen **dos modelos** y conviene separarlos:

Raspberry	Papel	Estado
Raspberry Pi 4	Computadora de la **celda de manufactura**, junto a la banda: cámara de línea, visión, relevadores y la página `celda.html`	**Vigente**, pero **sin probar** en el hardware real (**PENDIENTE (P-18)**)
Raspberry Pi 5 con AI HAT+	Se pensó para el robot (plan del 30 de septiembre)	**Descartada**; la guía del 9 de octubre que la describe quedó vieja

El cerebro del robot es el **Jetson** (capítulo 9), no una Pi. Los conceptos eléctricos básicos (voltaje, corriente, niveles lógicos) están en el capítulo 2; este capítulo añade lo que el código y la hoja de conexión de la celda dan.

Instalar la celda en la Pi: sin torch, sin ultralytics

El agente de la celda (`backend/celda_agent.py`) está pensado para que la Pi **no** cargue Lumina (el asistente del proyecto, capítulo 15), ni la base de datos, ni `torch` ni `ultralytics`: son pesadísimos. El modelo de visión corre en **ONNX** (un formato neutral de modelos) con `cv2.dnn`, un módulo de OpenCV. El archivo `backend/requirements-pi.txt` lo dice con todas sus letras:

> Se instala en un venv con `--system-site-packages`: OpenCV y NumPy salen de `apt` (`python3-opencv`, `python3-numpy`), **no de pip**. Prohibido en la Pi: `opencv-python`, `opencv-contrib-python`, `torch` y `ultralytics`. Requiere Python ≥ 3.10 (Raspberry Pi OS Bookworm trae 3.11).

¿Por qué `--system-site-packages`? Un **venv** (entorno virtual de Python) normalmente es una caja aislada; esa opción le permite **ver** los paquetes instalados por `apt`. Así la Pi usa el OpenCV compilado para ella (rápido) y pip solo instala `fastapi` y `uvicorn`.

Pasos (copiados de `COMANDOS_LINUX.md`, sección 7; ninguno corrido todavía en la Pi)

Una vez, en la laptop (el modelo se exporta allí porque la Pi no tiene torch), `[Laptop · PowerShell]`, desde la raíz del proyecto:

```
edge\.venv\Scripts\python.exe -m pip install onnx
$env:YOLO_AUTOINSTALL = "False"
edge\.venv\Scripts\python.exe -c "from ultralytics import YOLO;
YOLO('edge/perception/weights/parts.pt').export(format='onnx', imgsz=320,
opset=12)"
```

Luego se copia el archivo a la Pi (`<usuario>` y `<ip-pi>` son los de esa Pi y **no están en el repositorio**), `[Laptop · PowerShell]`:

```
ssh <usuario>@<ip-pi> "mkdir -p ~/aether/edge/perception/weights"
scp edge\perception\weights\parts.onnx <usuario>@<ip-
pi>:~/aether/edge/perception/weights/
```

Una vez, en la Pi, `[Pi 4 · SSH]`:

```
sudo apt install -y git python3-venv python3-opencv python3-numpy
git clone --depth 1 --filter=blob:none --sparse <url-del-repo> ~/aether && cd
~/aether
git sparse-checkout set backend dashboard edge/perception/parts
python3 -m venv --system-site-packages ~/celda-venv
~/celda-venv/bin/pip install -r backend/requirements-pi.txt
```

El `git sparse-checkout` baja **solo las carpetas necesarias** (`backend`, `dashboard` y las clases del modelo), no todo el repositorio.

Arrancar (dentro de `tmux`, un programa que mantiene la sesión viva aunque se cierre la ventana de SSH), `[Pi 4 · SSH]`:

```
cd ~/aether/backend && ~/celda-venv/bin/python -m uvicorn celda_agent:app --host
0.0.0.0 --port 8000
curl -s localhost:8000/health   # esperado: {"status":"ok","error":null}
```

Después, desde la laptop, en el navegador: `http://<ip-pi>:8000/celda.html`. Si `health` falla, el campo `"error"` dice por qué.

Una incógnita técnica real

El modelo ONNX se probó con **OpenCV 5.0** en una laptop. **Bookworm trae OpenCV 4.6.** Si `cv2.dnn` de esa versión no sabe leer el ONNX (por un operador que cambió entre versiones), la celda no arranca. La bitácora lo marcó como **NO VERIFICADO**, y la primera prueba en la Pi debe incluir `top` (para ver CPU) y `curl /line/cell/status` con una pieza delante. Sistema, RAM y cámara exactos de esa Pi 4: **PENDIENTE (P-18)**.

Los 40 pines: dos formas de contarlos

Una confusión clásica del GPIO: **cada pin tiene dos números**.

- El **número físico**: la posición en el conector, del 1 al 40.
- El **número BCM** (por el chip de Broadcom): el nombre "lógico" del GPIO, el que usa el software.

`cell_line.py` usa números **BCM**: **GPIO 8** para el pistón y **GPIO 9** para la banda. En el conector físico corresponden (**regla general; verificar en el diagrama oficial `pinout.xyz` antes de cablear**) a los pines **24 y 21**. Los números se pueden cambiar con las variables `LINE_GPIO_PISTON` y `LINE_GPIO_BANDA`. Un riesgo derivado, señalado por la bitácora: esos números son **BCM de Raspberry Pi**; en el **Jetson** serían **otras líneas** del chip, así que **jamás** debe instalarse `gpiod` en el Jetson (los scripts de arranque del agente se niegan a seguir si lo detectan).

El estado del pin al encender: la trampa que no se ve en el código

Esta es la lección más valiosa del capítulo y es **regla general** (no medida en la Pi del proyecto): al **encender** una Raspberry, **antes** de que corra el programa, los pines no están "apagados": cada uno tiene una resistencia interna que lo **jala** hacia 3.3 V (*pull-up*) o hacia

0 V (*pull-down*). Según la documentación del chip, los **GPIO 0 a 8 arrancan con pull-up** (nivel alto) y los GPIO 9 a 27 con pull-down (nivel bajo). Aplicado a la celda:

- **GPIO 8 (pistón)** arrancaría en **alto**.
- **GPIO 9 (banda)** arrancaría en **bajo**.

¿Qué hace una placa de relevadores con un pin en alto? Depende de **cómo esté hecha**: muchas placas baratas son **"activas en bajo"** (el relevador se enciende cuando el pin está en **bajo**), otras **"activas en alto"**. Con una placa activa en alto, **el pistón se dispararía solo al encender la Pi**. Qué tipo de placa se usa y cómo está alimentada su bobina **no está escrito en ningún lado**: **PENDIENTE (P-12)**. Por eso la regla de seguridad exige medir **el estado de los pines al arrancar con un LED o un multímetro, sin carga conectada**, antes de conectar el pistón real.

Qué hay entre un pin y el pistón

Un pin de 3.3 V no puede mover una electroválvula de 24 V. La cadena segura es:

1. **Pin GPIO (3.3 V, pocos miliamperios)** →
2. **Módulo de relevadores** (idealmente con entrada aislada por **optoacoplador**, que separa eléctricamente el mundo de la Pi del de 24 V) →
3. **Contacto del relevador** (aguanta los 24 V y la corriente) →
4. **Electroválvula del pistón**, con su **diodo de protección** en paralelo con la bobina si es de corriente continua. Ese diodo (*flyback*) absorbe el pico de voltaje que genera una bobina al apagarse; sin él, ese pico puede dañar el contacto del relevador.

La Pi se alimenta con **su propia fuente de 5 V** (la oficial es de 5 V y 3 A por USB-C), separada de los 24 V. Una Pi con poca energía muestra un aviso de **bajo voltaje** y puede comportarse raro.

Lista antes de energizar algo real

Basada en el plan de seguridad del proyecto y en la regla del repositorio:

1. **Sin cargas.** Pi encendida, pines libres. Con multímetro, medir el voltaje de los GPIO 8 y 9 **al encender** y anotar los valores.
2. **Un LED con resistencia** (capítulo 2) en lugar del relevador. Verificar que parpadea cuando el código lo pide y que **no** parpadea al arrancar.
3. **Probar** `cell_line.py` con `gpiod` instalado pero **sin carga real**: ¿qué hace al detectar `FAIL`?

4. **Relevador sin pistón.** Escuchar el clic y medir su contacto.
5. **Solo entonces** el pistón, con aire a presión baja y **sin pieza**.
6. **Paro de emergencia físico** cableado **antes** de cualquier actuador real (**PENDIENTE (P-16)**).
7. **Instalar** `python3-libgpiod` (`sudo apt install python3-libgpiod` y el usuario en el grupo `gpio`) **solo después** de los pasos anteriores.

Arrancar la celda sola, sin abrir una ventana

La celda debe poder operarse sola; eso implica que, al darle corriente a la Pi, el agente tendría que **arrancar solo**. Hoy se arranca a mano dentro de `tmux`. La opción de **systemd** (el sistema de Linux para servicios que arrancan solos) se descartó para el Jetson en la ADR 0012 *("otra pieza que mantener")* y se anotó como posible mejora cuando el agente resulte estable. Para la celda, que se espera que funcione sin supervisión, vale la pena reconsiderarlo: **PENDIENTE (P-25)**. Mientras tanto, el agente reabre la cámara solo si se cae y no necesita que alguien abra el video.

La Raspberry Pi 5 de la guía vieja

Para que no confunda cuando aparezca en los documentos: la guía del 9 de octubre describe una **Pi 5** (8 GB) con fuente de 27 W, una pantalla HDMI, un ESP32 conectado por serial UART (activando `dtparam=uart0=on` en `/boot/firmware/config.txt`) y un navegador mostrando la cara de Lumina. Todo eso **corresponde al plan descartado**. No es necesario para el robot, que usa el Jetson y el ESP32 por USB. El **problema del navegador del Jetson** (capítulo 9) es justamente la contraparte de esa decisión.

La Pi en el mundo industrial

En planta, una **Raspberry Pi** se usa para prototipos, visión y supervisión; los productos serios emplean **versiones industriales** (con caja, riel DIN y alimentación de 24 V, como las que venden fabricantes de computadoras de borde) y **nunca** como única barrera de seguridad: eso se deja a **relevadores de seguridad** y **PLC certificados**. Decir esto ante un jurado, con estas palabras, demuestra criterio.

Preguntas de repaso: Raspberry y GPIO

1. ¿Por qué la Pi no lleva `torch` ni `ultralytics`, y qué usa en su lugar?
2. ¿Qué diferencia hay entre el número físico y el número BCM de un pin?

3. ¿Qué podría pasar al encender la Pi si la placa de relevadores es "activa en alto" y GPIO 8 arranca en alto?
4. ¿Para qué sirve el diodo en paralelo con la bobina de una electroválvula?
5. Escribir, en orden, los cuatro primeros pasos de la lista "antes de energizar algo real".
6. ¿Qué parte de `COMANDOS_LINUX.md`, sección 7, ya se corrió en la Pi y cuál no?

Capítulo 9: El Jetson Orin Nano Super

El cerebro del robot, explicado desde cero: qué es, cómo se le instaló PyTorch con GPU, qué problemas reales dio entre el 21 y el 23 de octubre y cómo se diagnosticó cada uno.

Una computadora con una tarjeta gráfica pegada

El **Jetson Orin Nano Super** (de la empresa NVIDIA) es una computadora del tamaño de una cartera que lleva, en el mismo chip, un procesador y una **GPU** (*Graphics Processing Unit*, unidad de procesamiento gráfico). Una GPU es un chip con **miles de núcleos pequeños** que hacen la misma operación a la vez; sirve para pintar videojuegos y, por la misma razón, para multiplicar las enormes tablas de números de las redes neuronales. Es la pieza que permite que **YOLO** (el modelo que detecta objetos, capítulo 14) corra con el robot en movimiento, sin depender de internet.

Datos del Jetson del equipo (todos **reportados por Mario**, responsable del software del robot, con salidas reales; fuente: `docs/jetson/SETUP_JETSON.md`):

Dato	Valor
Equipo	Jetson Orin Nano Super 8 GB, nombre de máquina `mario-desktop`, usuario `mario`
Sistema	Ubuntu 22.04, kernel `5.15.148-tegra`, arquitectura `aarch64` (ARM de 64 bits)
Jetson Linux	L4T R36 revisión 4.7 (lo que el equipo llama "JetPack 6")
Python	3.10.12
CUDA / cuDNN / TensorRT	12.6 / 9.3 / 10.3
OpenCV del sistema	4.8.0 (con GStreamer 1.20.3)
Memoria	7.4 GiB de RAM, 3.7 GiB de swap, disco de 47 GB (≈ 23 GB libres)
Modo de energía	**25 W** (ID 1). Existen 7 W, 15 W, 25 W y `MAXN_SUPER` (el máximo, **no usado**)

Siglas, para que no queden como magia:

- **CUDA**: el lenguaje y las herramientas con que los programas "hablan" con la GPU de NVIDIA.
- **cuDNN**: una biblioteca de operaciones de redes neuronales optimizadas sobre CUDA.

- **TensorRT**: un "compilador" que reorganiza un modelo para que corra más rápido en el Jetson. **No se usa todavía**: la decisión de adoptarlo se dejó para después de medir (ADR 0012).
- **JetPack / L4T** (*Linux for Tegra*): el paquete de sistema operativo y controladores de NVIDIA para Jetson.

Por qué se anota el modo de energía en toda medición: el mismo Jetson rinde distinto a 7 W que a 25 W. Una cifra de FPS sin su modo es una cifra que nadie puede comparar. La bitácora lo exige: "*toda medición de FPS debe anotar el modo de* `nvpmodel`".

Qué hace el Jetson en Aether y qué no

Según la ADR 0012 (**propuesta implementada en la rama `jetson-agente`, aún no aceptada formalmente por Mario**; **PENDIENTE (P-32)**), el Jetson ejecuta un **agente**: un programa (`backend/aether_agent.py`, puerto 8001) que **abre la cámara del Jetson**, corre YOLO + lectura de códigos + ArUco, **sirve el video anotado** y manda a la laptop **solo observaciones en JSON**, cada una con un identificador único. El **video nunca viaja por WiFi**: lo que viaja son datos chicos. La **laptop** sigue siendo el servidor central (base de datos, Lumina). Esa filosofía, *mover datos chicos y no datos grandes*, es la del capítulo 1. El Jetson **no** ejecuta la voz de Lumina (el micrófono es el de la laptop) ni mueve motores directamente: eso lo hace el **ESP32** (capítulo 5), al que le manda órdenes por **USB**.

La bitácora como cuento policíaco

Entre el 21 y el 23 de octubre el equipo preparó el Jetson y, en cada paso, apareció un problema real. Es la mejor escuela de diagnóstico del proyecto.

Caso 1: el SSH que se cortaba solo (21 de octubre)

Síntoma: las sesiones de **SSH** (la conexión remota de terminal) se cortaban con `Connection reset` mientras el Jetson tenía **WiFi y cable Ethernet a la vez** en la misma red. **Hipótesis (sin comprobar):** con dos interfaces en la misma subred, el equipo tiene **dos IP** (`.18` y `.19`) y la ruta o el nombre `mario-desktop.local` pueden "saltar" de una a otra a mitad de sesión. **Qué se hizo:** se apagó el WiFi (`nmcli radio wifi off`) y se dejó solo el cable. Para sobrevivir a cortes de todos modos se usa `tmux`, un programa que mantiene vivas las ventanas de terminal aunque se caiga la conexión. **Estado:** la causa **no está diagnosticada** y la estabilidad con solo el cable **no está verificada** en uso largo (**PENDIENTE (P-26)**).

Caso 2: una librería que PyTorch pedía y NVIDIA no explicaba (21 de octubre)

Síntoma: al planear la instalación de PyTorch de NVIDIA, la documentación mencionaba `libcusparseLt.so.0` y un script que **no cubre CUDA 12.6**. **Qué se hizo:** en vez de adivinar, se buscó el archivo oficial para Jetson (`linux-aarch64`, versión **0.6.2.3**; la 0.7.0 exige CUDA 12.8). Se descargó, se **comprobó su huella** (SHA-256 igual a la calculada en la laptop), se verificó que era un binario ARM de 64 bits y se copió a `/usr/local/cuda/lib64`. Prueba: `ctypes.CDLL('libcusparseLt.so.0')` imprime "carga OK". **Límite honesto:** solo se probó que **se carga**, no que PyTorch funcione con ella (eso vino en el caso 4). **Nota de seguridad:** `cp -a` conserva el dueño del archivo (`mario`) y una librería del sistema que un usuario común puede reescribir es un riesgo; se propuso `sudo chown root:root` (estado: **NO verificado** que se aplicara; **P-26**).

Caso 3: una dependencia que arrastraba el torch equivocado (21 de octubre)

Síntoma: al simular la instalación de `requirements-jetson.txt`, `pip` traía `torch` 2.14.1 **de PyPI**, que **no** tiene soporte de GPU de NVIDIA, a través de un paquete llamado `ultralytics-thop`. **Por qué importa:** habría **pisado** el torch bueno. **Corrección:** se sacó `ultralytics-thop` del archivo de requisitos y se instaló aparte con `pip install --no-deps` (sin dependencias). Resultado verificado con `pip --dry-run`: 43 paquetes, **ninguno** `torch`, `torchvision` ni `opencv`, y `numpy` 1.26.4 (menor que 2). **Regla que sale de ahí: al instalar algo con `pip` sobre un entorno delicado, se simula primero (`--dry-run`) y se mira qué quiere cambiar.**

Caso 4: PyTorch con GPU funciona (21 de octubre)

Se instalaron las "ruedas" (*wheels*, paquetes precompilados) de NVIDIA: `torch 2.5.0a0+872d972e41.nv24.08` y `torchvision 0.20.0a0+afc54f7`. Resultados **reportados**:

Prueba	Resultado
`torch.cuda.is_available()`	`True`
Dispositivo	Orin
`torch.version.cuda`	12.6
Multiplicación de matrices 512×512 en GPU	resultado finito (correcto)
¿`pip` cambió el torch por el de PyPI?	No: conserva el sufijo `nv`

Y una detección con YOLOv8n sobre una imagen de archivo (`device=0`): detectó `bus`, 4 `person` y `stop sign` en **46 ms** (modo 25 W). **Límite:** una sola medición con imagen estática; no son FPS y no sirve para los umbrales de aceptación.

Dos detalles de oficio que ahorran tardes:

- `YOLO_AUTOINSTALL=False` es obligatorio: sin él, `ultralytics` intenta instalar `opencv-python` al no ver los metadatos de pip del OpenCV del sistema, y rompe el entorno.
- Si aparece `Uninstalling torch` o `Collecting torch==…` durante una instalación, se **cancela con** `Ctrl+C` antes de que diga `Successfully installed`.

La bitácora anota con honestidad dos dudas: la rueda se describió como compilada para CUDA 12.4 pero el sistema dice 12.6 (**no se verificó con qué CUDA se compiló**), y un hilo de foro de NVIDIA reporta, en L4T 36.4.7, el error "unable to allocate CUDA0 buffer" tras actualizar paquetes de Ubuntu; por eso **no se hizo** `apt upgrade` y no se probó YOLO bajo carga larga: **NO VERIFICADO**.

Caso 5: una cámara con una resolución que no existe (21 de octubre)

Ya descrito en el capítulo 6: OpenCV por GStreamer entregaba 1536 × 2304. Solución: forzar V4L2 con variables de entorno. Estado: falta pegar el log que confirma 640 × 480 con esas variables.

Caso 6: el backend corre en el Jetson… pero no debía (21 de octubre)

Como **prueba**, se corrió el backend completo en el puerto 8001. Funcionó: `/health`, `/line/cell/status` e `/inventory` respondieron. Pero no era la arquitectura prevista (el backend debe vivir en la laptop) y arrancó partes de Lumina que no tienen sentido en el Jetson. Resultado útil: se leyó el código y se comprobó que **ningún botón de la celda puede mover un pin** mientras `gpiod` no esté instalado (capítulo 7).

Caso 7: ningún navegador abría (21 de octubre)

Síntoma: los navegadores instalados como **snap** (paquetes autocontenidos de Ubuntu; por ejemplo Chromium) fallaban con `snap-confine ... required permitted capability cap_dac_override not found`. **Hipótesis de fuentes externas:** una versión nueva de `snapd` en un kernel sin ciertas opciones. Hay **dos** `snapd` (el `.deb` 2.67.1 y el snap 2.77.1). La causa raíz **no está diagnosticada**. **Solución parcial:** se instaló Firefox desde el **PPA de Mozilla** (repositorio oficial) como `.deb`, versión `158.0.1~mt1`, con `--allow-downgrades`. **Abre** el panel de la laptop, pero **va lento por VNC** (un escritorio remoto; ≈ 87 % de un

núcleo en `firefox`): se interpreta como renderizado por software a través de VNC; **no está medido con un monitor HDMI**. **Efecto en Lumina:** `robot.html` usa `SpeechRecognition` del navegador, que no está activo en Firefox; **por eso la voz de Lumina quedó en la laptop** (ADR 0012, capítulo 15).

Caso 8: la cámara que el navegador no mostraba (23 de octubre)

El agente del Jetson funcionó: **29.0 FPS** de cámara, **29.1** de entrega a 640 × 480, inferencia **68.0 ms** de promedio, `laptop.reachable` verdadero y `/stream` con respuesta 200 en `127.0.0.1:8001`. Pero la página **no mostró el video**: Firefox bloqueó la imagen por la **política de seguridad (CSP)**. La causa y la corrección están en el capítulo 11; aquí queda como ejemplo de que **el agente puede estar perfecto y la pantalla vacía**, y de que hay que diagnosticar por capas.

Caso 9: el audio de Lumina bloqueado (23 de octubre)

Otro bloqueo del mismo tipo, esta vez con el **audio** (`media-src`). Capítulo 11.

La regla de seguridad del Jetson: nunca `gpiod`

El usuario del Jetson pertenece al grupo `gpio`. `cell_line.py` hace `import gpiod` al cargarse y, **si existe**, configuraría como **salidas** las líneas 8 y 9 de `/dev/gpiochip0`: números **BCM de Raspberry Pi**, que en el **chip del Jetson** corresponden a **otras líneas**. Por eso:

- **No** se instala `gpiod` de Python en el Jetson (**VERIFICADO**: `python -c "import gpiod"` falla; sí están las herramientas del sistema `gpiod` 1.6.3 y `libgpiod2`).
- El script de arranque del agente **se niega a seguir** si lo detecta.
- Antes de arrancar cualquier backend se comprueba, `[Jetson · SSH]`, dentro del entorno virtual:

```
python -c "import gpiod"      # DEBE fallar con ModuleNotFoundError
```

Arrancar el agente del Jetson

Rutas y nombres vienen de `COMANDOS_LINUX.md`, sección 8 (**construido y probado solo en la laptop; los scripts `.sh` no se han corrido en el Jetson**, salvo la prueba reportada del 23 de octubre).

```
# Una vez: configuración (editar AETHER_LAPTOP_URL con la IP de la laptop)
[Jetson · SSH]
cp ~/aether/scripts/jetson/aether-agent.env.example ~/.aether-agent.env && nano
~/.aether-agent.env

# Arrancar el agente dentro de tmux (sesión "agente")
[Jetson · SSH]
bash ~/aether/scripts/jetson/iniciar-agente.sh

# Medir: FPS de cámara y de entrega, ms de inferencia, CPU y RAM
[Jetson · SSH]
python3 ~/aether/scripts/probar_agente.py http://localhost:8001 --segundos 15

# Detener
[Jetson · SSH]
bash ~/aether/scripts/jetson/detener-agente.sh
```

`iniciar-agente.sh` hace varias comprobaciones antes de arrancar (que exista `tmux`, el entorno virtual, `/dev/video0`, el archivo de configuración, y que `gpiod` **no** esté), dice **qué falta y cómo arreglarlo**, y se puede repetir sin miedo: si el agente ya corre, lo dice y no hace nada. El log queda en `~/aether-agente.log`. Para ver la página de la laptop **dentro del Jetson** (por VNC): `bash ~/aether/scripts/ver-aether.sh robot.html`.

Las dos direcciones de la red del laboratorio

Según la bitácora, la red de esos días fue `192.168.100.x`: el Jetson por cable en `192.168.100.19` y la laptop por WiFi en `192.168.100.14`. El Jetson alcanzó el backend de la laptop (`curl http://192.168.100.14:8000/health` devolvió `{"status":"ok"}`); antes dio *timeout* simplemente porque el backend no estaba corriendo. Esas direcciones **cambian con la red**, por eso `agent_config.py` admite **nombres** (`mario-desktop.local`) y los compara con las IP a las que apuntan (con una caché de 60 s). Que el nombre se resuelva depende de la red y **no está verificado**.

Qué falta medir

La cifra que más falta es la de **FPS con buena luz, la cámara en su lugar y el modo de energía anotado** (**PENDIENTE (P-22)**): el objetivo de **≥ 30 FPS** es de diseño y la prueba de la laptop (30.0 FPS con webcam y YOLO en CPU) no dice nada del Jetson. Tampoco se ha probado el modo `MAXN_SUPER`, ni TensorRT (se decidirá con datos), ni cómo se

alimentará el Jetson dentro del robot (capítulo 4, **P-19**). Los diagnósticos abiertos (cortes de SSH, `snap-confine`, propiedad de `libcusparseLt`) están en la lista de pendientes del final del libro (**P-26**), junto con la duda de si el robot tendrá micrófono (**P-20**).

Jetson en productos reales

La línea **Jetson** de NVIDIA es la plataforma típica de robots autónomos pequeños, cámaras inteligentes y drones: percepción en el borde (*edge*), sin nube, con consumo de unos pocos vatios. En producción se usan **imágenes de sistema ya empaquetadas**, contenedores (Docker) para reproducir el entorno y **TensorRT** para exprimir el rendimiento. Aether sigue el camino del laboratorio (venv + archivo de requisitos), más fácil de entender, con una tabla de versiones reportadas como la de arriba, y deja Docker y TensorRT como pasos de madurez.

Preguntas de repaso: el Jetson

1. ¿Qué dos cosas distintas protegen `YOLO_AUTOINSTALL=False` y el `--no-deps` al instalar `ultralytics`?
2. ¿Por qué el torch de PyPI y el torch de NVIDIA no son lo mismo aunque se llamen igual?
3. Si `torch.cuda.is_available()` devolviera `False`, ¿qué tres cosas se revisarían primero según la bitácora?
4. ¿Por qué una cifra de FPS necesita decir en qué modo de energía se midió?
5. ¿Por qué el agente puede funcionar bien y aun así no verse el video en la página?
6. ¿Por qué nunca debe instalarse `gpiod` en el Jetson?

Capítulo 10: Redes desde cero

Direcciones IP, puertos, nombres, firewall y SSH: lo que hay que entender para que tres computadoras se hablen y para saber dónde buscar cuando no lo hacen.

Un timeout que parecía un misterio

El 21 de octubre de 2026, desde el Jetson, el primer intento de saludar al servidor de la laptop (`curl http://192.168.100.14:8000/health`) terminó en **timeout**: esperó y nadie contestó. Había tres sospechosos clásicos (el firewall de Windows, una subred distinta, una dirección equivocada), pero la causa resultó simple: **el servidor no estaba corriendo en la laptop**. Al arrancarlo, el mismo comando devolvió `{"status":"ok"}`. La lección es general: **una computadora no tiene "un" lugar fijo; tiene la dirección que la red le presta**, y antes de culpar a la red conviene comprobar que el otro extremo realmente escucha.

La bitácora de hardware del equipo guarda las direcciones de los días de trabajo, y sirven como ejemplo real:

Situación	Laptop	Jetson	Observación (bitácora)
WiFi de la universidad (misma subred privada)	dirección privada	dirección privada	Funcionó con una regla de firewall
Cable de red en la universidad	—	cae en **otra subred** privada	Laptop y Jetson **no se ven**; se usa el WiFi o el USB-C
Cable USB-C entre laptop y Jetson (RNDIS)	`192.168.55.100`	`192.168.55.1`	Respaldo confiable; ocupa el cable
Casa	`192.168.100.14` (WiFi)	`192.168.100.19` (cable)	Funcionó sin regla nueva

(Las direcciones exactas de la red de la universidad **no se publican en este libro**: el equipo decidió sacarlas del repositorio y guardarlas en un archivo local del Jetson. Las de la casa y del cable USB-C sí están documentadas por decisión del equipo; son direcciones privadas, no accesibles desde internet.)

La IP: la dirección de la casa

Una **IP** (*Internet Protocol*, "protocolo de internet") es la **dirección de una computadora en una red**, escrita como cuatro números entre 0 y 255 separados por puntos (`192.168.100.14`). Existen rangos reservados para redes **privadas** (las de una casa, una oficina o un laboratorio) que no son alcanzables desde internet: `10.x.x.x`, `172.16–31.x.x` y `192.168.x.x`. Todas las direcciones del cuadro anterior son privadas.

Una **subred** es un grupo de direcciones que "se ven" directamente. Se describe con una **máscara** o con la notación **CIDR**: `192.168.100.0/24` significa "las direcciones que empiezan con `192.168.100.` (256 posibles)". Dos computadoras en subredes distintas **no se ven sin un enrutador que las conecte**. Es exactamente lo que le pasó al Jetson por cable en la universidad: cayó en otra subred y quedó "al otro lado de la pared".

Aether usa esta idea de forma explícita en su seguridad: `backend/lan_guard.py` acepta peticiones de los agentes y de la voz **solo desde la red local**, definida por la variable `AETHER_LAN_CIDRS` (por defecto, los rangos privados y la propia máquina).

El puerto: la puerta dentro de la casa

Una computadora ejecuta muchos programas a la vez y todos usan la misma dirección. Para saber a quién se dirige un mensaje existe el **puerto**: un número del 0 al 65535, como el número de departamento dentro de un edificio. Un programa que "escucha" en un puerto es un **servidor**; quien se conecta a él es el **cliente**.

Los puertos de Aether (**VERIFICADO** en `docs/guia_arranque.md` y en el código):

Puerto	Equipo	Qué escucha
8000	Laptop	El servidor central de Aether (panel, API, Lumina)
8000	Raspberry Pi 4	El agente de la celda (`celda.html`)
8001	Jetson	El agente de percepción
11434	Laptop	Ollama, el modelo de lenguaje local de Lumina
5055	Laptop	El reproductor del video de presentación (`npm run show`)
8888	Laptop (solo `127.0.0.1`)	La devolución de llamada (*callback*) de Spotify

Una dirección completa se escribe `http://192.168.100.14:8000`: protocolo, IP y puerto.

localhost: hablar consigo mismo

La dirección `127.0.0.1` (o el nombre `localhost`) significa "esta misma computadora". Un programa que escucha en `127.0.0.1` **solo acepta conexiones de su propia máquina**; uno que escucha en `0.0.0.0` acepta conexiones de **cualquier** interfaz de red. La diferencia es de seguridad: el agente del Jetson y el de la Pi escuchan en `0.0.0.0` (para que la laptop los alcance) y por eso llevan un guardia de red local; el callback de Spotify solo en `127.0.0.1` porque nadie más debe usarlo.

Este truco tiene un uso fino en Aether: cuando la página se abre en el mismo Jetson, el servidor le entrega la dirección del video como `127.0.0.1:8001`, para que el video **no salga** de ese equipo (capítulo 1).

Nombres en lugar de números: DNS y mDNS

Memorizar `192.168.100.19` es incómodo; por eso existen los **nombres**. El **DNS** (*Domain Name System*) es el directorio telefónico de internet: traduce `aethermx.online` a una IP. En una red local pequeña se usa **mDNS** (*multicast DNS*): cada equipo anuncia su propio nombre terminado en `.local` (como `mario-desktop.local`) a toda la red por **multidifusión**, sin servidor central.

Aether admite nombres de host en las variables de dirección (`AETHER_PERCEPTION_AGENT_URL=http://mario-desktop.local:8001`), pero con una advertencia que el propio proyecto vivió: **muchas redes de universidad bloquean mDNS** y, de hecho, ya hubo cortes con `.local`. La documentación lo dice sin rodeos: "*que el nombre se resuelva depende de la red y no está verificado*". La regla práctica: **se prueba primero**, `[Laptop · PowerShell]`:

```
[System.Net.Dns]::GetHostAddresses('mario-desktop.local')
```

Si responde "No such host is known", se usa la IP. Un detalle de implementación: el servidor recuerda 60 segundos lo que resuelve un nombre (también si falló) para no hacer esperar a cada petición.

TCP: una llamada que no se corta sin avisar

El protocolo que casi todo Aether usa para hablar es **TCP** (*Transmission Control Protocol*): establece una "llamada" entre dos programas, numera los mensajes, reenvía los que se pierden y avisa si la llamada se cortó. Sobre TCP viajan **HTTP** (la web; capítulo 11) y **SSH** (la terminal remota). Su hermano **UDP** manda mensajes sin llamada ni confirmación; es más rápido y menos fiable. Aether no usa UDP de forma explícita.

El firewall: el portero de Windows

El **firewall** (cortafuegos) es el portero de cada computadora: decide qué conexiones entrantes deja pasar. El de Windows, por defecto, **no deja entrar** conexiones desde otras máquinas al puerto 8000. Por eso, la primera vez que el Jetson quiso hablar con el servidor de la laptop, falló por **timeout** (esperó y nadie contestó).

La solución del proyecto es **una regla mínima y segura**: abrir el puerto 8000 **solo** para la subred local y **solo** cuando la red está marcada como **Privada** (Windows distingue redes Privadas de Públicas, y no conviene abrir nada en una pública). El comando, `[Laptop · PowerShell, como administrador]`:

```
New-NetFirewallRule -DisplayName "Aether 8000 (red local)" -Direction Inbound -Protocol TCP -LocalPort 8000 -Action Allow -Profile Private -RemoteAddress LocalSubnet
```

Si la red aparece como **Pública**, se cambia: `Set-NetConnectionProfile -InterfaceAlias "<alias>" -NetworkCategory Private` (el alias sale de `Get-NetConnectionProfile`). El script `iniciar_aether.ps1 -SoloComprobar` revisa si falta la regla. **La regla de firewall en Privado no se ha comprobado de punta a punta** con el Jetson después del cambio de red: **NO VERIFICADO**.

SSH: manejar el Jetson desde la laptop

SSH (*Secure Shell*) abre una **terminal remota cifrada**: se escribe en la laptop y los comandos corren en el Jetson. Es la herramienta diaria con el Jetson y la Pi. Ejemplo (`<usuario>` y la IP o nombre son los de ese equipo), `[Laptop · PowerShell]`:

```
ssh mario@192.168.100.19
```

Tres lecciones reales de la bitácora:

1. **Dos interfaces en la misma red rompen SSH.** Con WiFi y cable activos a la vez, el Jetson tenía dos IP (`.18` y `.19`) y las sesiones se cortaban con `Connection reset`; se apagó el WiFi (`nmcli radio wifi off`). La causa exacta no está diagnosticada (**P-26**).
2. **Un corte de SSH mata lo que corría dentro.** Por eso lo largo se ejecuta dentro de `tmux` (capítulo 12).
3. **SSH sin contraseña** (con llaves y un archivo `~/.ssh/config`) quedó pendiente (**P-26**). El script `iniciar_aether.ps1 -Jetson` ya lo da por hecho, con un nombre de host `jetson-uni`.

Y una limitación de **VNC** (un escritorio remoto): la sesión de VNC del Jetson no copia ni pega bien ni detecta las mayúsculas, por eso los comandos se mandan por SSH y VNC solo sirve para *mirar* el escritorio.

Cuatro preguntas para ubicar cualquier falla de red

El equipo documentó un orden de diagnóstico, de lo más cercano a lo más lejano. Funciona para casi cualquier "no conecta":

1. **¿Corre el servidor?** `[Laptop · PowerShell]`: `.\iniciar_aether.ps1 -SoloComprobar` (revisa entorno, clave de la celda, puerto, perfil de red y firewall sin arrancar nada).
2. **¿Se ve desde otra máquina?** `[Jetson · SSH]`: `curl -s -m 3 http://<IP laptop>:8000/health`. Debe responder `{"status":"ok"}`. Si da *timeout*: firewall, subred o el servidor no corre.
3. **¿Corre el agente local?** `[Jetson · SSH]`: `curl -s -m 3 http://localhost:8001/health`.
4. **¿Qué mide el agente?** `[Jetson · SSH]`: `python3 ~/aether/scripts/probar_agente.py http://localhost:8001 --segundos 15`.

Síntomas típicos y qué significan:

Síntoma	Causa más probable
`timeout` al `curl` hacia la laptop	Firewall o subred distinta
`Connection refused`	La máquina responde pero **nadie escucha** en ese puerto: el servidor no está corriendo
`No such host is known`	El nombre `.local` no se resuelve: usar la IP

Síntoma	Causa más probable
Funciona unos minutos y se corta	Suspensión de la laptop o dos IP en el Jetson

Una causa que parece de red y no lo es: **Windows suspende la red o la laptop**. La guía pide configurar *Energía → Nunca suspender cuando esté conectado*; si la laptop se duerme, el Jetson pierde al servidor.

Una red en una empresa con cientos de equipos

En una planta real, las redes se segmentan en **VLAN** (redes virtuales separadas por un conmutador): una para oficinas, otra para las máquinas, otra para las cámaras. Las IP se asignan de forma fija o por reservas del servidor **DHCP**, y la seguridad usa **firewalls de planta** y listas de dispositivos permitidos. El principio que Aether ya aplica en pequeño es el mismo: **mínimos permisos, red local separada, nada de abrir puertos "para probar".**

Preguntas de repaso: la red del laboratorio

1. ¿Por qué el Jetson por cable y la laptop por WiFi no se veían en la universidad pero sí en la casa?
2. ¿Qué diferencia hay entre escuchar en `127.0.0.1` y en `0.0.0.0`?
3. ¿Por qué `curl` da `timeout` en un caso y `Connection refused` en otro, y qué dice cada uno?
4. ¿Qué hace la regla de firewall del proyecto y por qué es "Privado" y no "Público"?
5. ¿Para qué se usa `tmux` junto con SSH?
6. ¿Por qué un nombre `.local` puede fallar en una red de universidad?

Capítulo 11: La web por dentro

Cómo conversan el navegador y el servidor, por qué un video se puede mandar como una foto interminable y qué es la política de seguridad que bloqueó dos veces lo que el equipo quería mostrar.

Dos veces el navegador dijo que no

El 23 de octubre de 2026, en la primera prueba real del agente del Jetson, todo funcionaba: la cámara entregaba 29 cuadros por segundo, el servicio respondía y `/stream` daba código 200. Pero la página **no mostraba el video**. Horas después ocurrió algo parecido con el **sonido** de Lumina: el servidor devolvía el audio, y el navegador se negaba a reproducirlo.

En ambos casos el responsable fue el mismo guardia, una **política de seguridad de contenido** (*Content Security Policy*, **CSP**), y el diagnóstico siguió el mismo método: separar **qué responde el servidor** de **qué permite el navegador**. Este capítulo enseña las piezas para entender esos casos y muchos otros.

Pedir y contestar: así habla la web

La web funciona con un diálogo muy simple. El **navegador** (cliente) manda una **petición** (*request*) a un servidor: "dame `/inventory`". El servidor contesta con una **respuesta** (*response*) que lleva un **código de estado** y un cuerpo. El protocolo se llama **HTTP** (*HyperText Transfer Protocol*). Las peticiones tienen un **verbo**:

Verbo	Significa	Ejemplo en Aether
`GET`	"Dame algo"	`GET /inventory` devuelve el inventario
`POST`	"Toma estos datos y haz algo"	`POST /robot/ask` manda una pregunta a Lumina

Verbo	Significa	Ejemplo en Aether
`PATCH`	"Cambia una parte de algo que ya existe"	`PATCH /inventory/{identifier}/retire` retira una entrada
`DELETE`	"Borra"	`DELETE /line/cell/session` cierra la sesión de la celda

Los **códigos de estado** que aparecen en el proyecto, y qué dicen:

Código	Significado	Dónde aparece
200	Todo bien	`GET /health`
403	Prohibido	Contraseña incorrecta para retirar inventario; ruta cerrada al público
404	No existe	Un `identifier` que no está en el inventario; rutas ocultas como `/.env`
422	Datos inválidos	Una pregunta vacía a Lumina; un mensaje de agente que no cumple el contrato
429	Demasiadas peticiones	Más de 120 peticiones por minuto desde un cliente fuera de la red local
503	Servicio no disponible	Ollama no responde; ElevenLabs no está configurado

El servidor de Aether está hecho con **FastAPI** (capítulo 13), que además genera gratis una página de documentación interactiva en `/docs`.

Cookies: cómo el servidor reconoce a alguien

HTTP no recuerda a nadie entre una petición y la siguiente. Para "recordar" que alguien ya inició sesión se usa una **cookie**: un texto pequeño que el servidor entrega y el navegador devuelve en cada petición siguiente. En Aether la usa la celda: `POST /line/cell/session` entrega la cookie `celda_session`, y los botones de Arranque y Paro **exigen** esa cookie. Se crea con dos protecciones: `httponly` (el JavaScript de la página no puede leerla, así un

script malicioso no la roba) y `samesite="lax"` (el navegador no la manda en peticiones iniciadas por otro sitio). La contraseña no está en el código; viene de `CELDA_HMI_CLAVE` (capítulo 7).

Una llamada que se queda abierta: los WebSocket

Con HTTP normal, el servidor solo habla cuando le preguntan. Para cosas que ocurren en vivo (alguien dijo una frase; Lumina empezó a hablar) existe el **WebSocket**: una conexión que se abre una vez y por la que **ambos lados pueden mandarse mensajes cuando quieran**. Aether tiene dos:

- `/voice/ws`: el servicio de voz de la laptop manda **texto** reconocido y las páginas de Lumina mandan "estoy hablando" (anti-eco). **Nunca viaja audio.** Hay dos papeles: *service* (solo uno a la vez) y *listener* (las páginas; el servidor admite hasta 8 conexiones en total).
- `/show/ws`: un relevo de eventos entre Lumina y el video de presentación, que corren en equipos distintos. Reenvía cada mensaje a todos los demás conectados, sin interpretarlo.

Ambos validan el origen: el WebSocket de un navegador debe venir del mismo servidor al que se conecta (`origin_matches_host`), y el canal de voz solo acepta clientes de la red local.

Video por la web: el truco del JPEG interminable

Mostrar una cámara en una página web se puede hacer sin tecnologías de video complicadas. Aether usa un truco clásico llamado **MJPEG por HTTP**: el servidor responde con el tipo de contenido `multipart/x-mixed-replace; boundary=frame` y **va mandando una foto JPEG tras otra** por la misma respuesta, que nunca termina. El navegador reemplaza la imagen cada vez que llega una nueva. En la página, basta con una etiqueta `<img src="/stream">`. Se ve en `backend/main.py`, `backend/celda_agent.py` y el agente del Jetson.

Ventajas: es simple y funciona en cualquier navegador sin código extra. Inconvenientes: gasta más ancho de banda que un video comprimido real (por eso el video **no viaja por la red hacia la laptop**; capítulo 1) y un `<img>` que falla da solo un evento `error` genérico, sin decir por qué, lo que complicó el diagnóstico de esta semana.

El guardia del navegador: la política CSP

Una **CSP** es una lista que la página (o el servidor) le entrega al navegador diciendo **desde dónde puede cargar qué**: imágenes de aquí, scripts de allá, audio de otro lado. Si algo intenta cargarse desde un lugar no autorizado, el navegador **lo bloquea** y lo anota en la consola. Protege contra scripts maliciosos inyectados en la página.

La CSP de Aether (en `backend/public_guard.py`) es estricta. Entre sus directivas:

Directiva	Valor	Para qué
`default-src`	`'self'`	Por defecto, solo del mismo servidor
`script-src`	`'self'` y `https://cdn.jsdelivr.net`	Scripts propios y de esa red de distribución
`img-src`	`'self' data: blob:` (más los agentes configurados)	Imágenes, y el video de los agentes
`media-src`	`'self' blob: data:`	Audio y video
`frame-ancestors`, `form-action`	`'self'`, `'none'`	Nadie puede incrustar la página ni mandar formularios a otro lado

Hay dos lugares donde una CSP puede vivir, y el navegador las aplica a la vez. Una es el **encabezado** de la respuesta HTTP; la otra es una etiqueta `<meta http-equiv="Content-Security-Policy">` dentro del HTML. Cuando las dos existen, **gana la más estricta** de cada directiva. Ese detalle explica los dos casos de la semana.

Caso 1: el video del agente (23 de octubre)

El servidor sí había incluido el origen del agente (`http://mario-desktop.local:8001` y `http://127.0.0.1:8001`) en el **encabezado**. Pero `dashboard/index.html` tiene su **propia** etiqueta `<meta>` con `img-src 'self' data: blob:`, y el navegador aplicó las dos: ganó la más estricta y bloqueó la imagen. `celda.html` y `robot.html` no tienen ese `<meta>`, por eso solo fallaba `index.html`.

Corrección (en `backend/agent_config.py`, función `extend_meta_csp`): el servidor entrega `index.html` con la **misma lista exacta de orígenes** dentro de su `<meta>`. Sin agente configurado el archivo sale idéntico. Además se quitó `localhost` de la lista, porque el servidor nunca lo entrega. Las pruebas nuevas **fallan con el código anterior y pasan con el nuevo**. (**NO verificado** en un navegador real: falta repetir la prueba en el Jetson con `Ctrl+F5`.)

Caso 2: el audio de Lumina (23 de octubre)

Firefox bloqueó `blob:` (los MP3 de ElevenLabs que reproduce `robot.js`) y `data:audio/mp3` (un MP3 mudo del reproductor de video empaquetado). Causa: `media-src 'self'` estaba en el encabezado (`backend/public_guard.py`) y en el `<meta>` de `index.html`. El `fetch("/robot/tts")` **no falló**: el servidor devolvió audio y la CSP impidió reproducirlo; el error posterior de "ElevenLabs no disponible" fue una consecuencia, no la causa. **Corrección:** `media-src 'self' blob: data:` en los dos sitios, sin tocar `script-src`, `connect-src` ni `img-src`. Una prueba verifica que encabezado y `<meta>` coinciden y que ni `blob:` ni `data:` aparecen en `script-src` ni en `connect-src`. (**NO verificado** en un navegador real, ni que la clave de ElevenLabs sea válida y tenga créditos.)

Cómo se mejoró el diagnóstico

Como un `<img>` que no carga solo dispara `error`, el equipo escribió `dashboard/stream-errors.js`: escucha el evento `securitypolicyviolation`, que el navegador lanza **antes** del error cuando el bloqueo es por CSP, y ofrece mensajes distintos: "*Bloqueado por la política de seguridad (CSP) de esta página, NO por la cámara*" contra "*No se pudo cargar el video… el agente no responde*". Es un ejemplo de buena ingeniería: **hacer que el error diga la causa real.**

Un clic que Firefox exige antes de sonar

Los navegadores crean el contexto de audio "suspendido" hasta el primer gesto del usuario (un clic o una tecla): es la política de *autoplay*, no un defecto. El comentario de `dashboard/robot.js` lo dice así. Por eso el reproductor de video y la pantalla de Lumina piden un primer clic o la tecla de un control remoto para "armar" el audio.

Un servidor que se cierra solo al llegar de internet

El dominio público (`aethermx.online`) y la red local no deben tener los mismos permisos. `backend/public_guard.py` es un **guardia** que, cuando la petición viene del dominio público o de una IP no local, **solo permite** `GET /health`, `GET /inventory` y `GET /inventory/history`. Todo lo demás (cámaras, celda, robot, dataset, Spotify, voz, configuración y cualquier método que modifique) responde **403**. Además aplica un límite de **120 peticiones por minuto** por IP, oculta rutas peligrosas (`/.env`, `/.git`, `*.db`,

`*.bak`) con un 404, y añade encabezados de seguridad (`X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, la CSP). El modo se controla con `AETHER_PUBLIC_MODE` (`auto`, `public` o `local`). El capítulo 21 completa la historia.

Dónde está cada cosa en los archivos

El panel web (`dashboard/`) se sirve **desde el mismo proceso** del servidor, en el mismo origen: no hay CORS entre página y API (decisión 7, opción B). Cada página y su guion:

Página	Para qué	Guion principal
`index.html`	Portada del proyecto, módulos, arquitectura	`portada.js`, `motion.js`, `loader.js`, `node-field.js`
`inventory.html`	Inventario declarado e historial	`app.js`
`celda.html`	Pantalla de la celda de manufactura	`celda.js`, `line-vision.js`
`robot.html`	Lumina: cara, voz, cámara y llamadas	`robot.js` (unas 2 600 líneas), `camera-stream.js`, `call-screen.js`
`dataset.html`	Fotos y etiquetas de piezas para entrenar el modelo	`dataset.js`
`armado.html`, `holo.html`	Hoja de armado 3D y vista holográfica del carrito	Código propio dentro de la página

Investigar desde el navegador

Las **herramientas de desarrollador** (tecla `F12`) son el mejor microscopio de una página: la pestaña **Red** muestra cada petición con su código; la **Consola**, los errores y los bloqueos de CSP. Un recorrido útil para un video que no aparece: abrir `F12`, recargar con `Ctrl+F5` (ignora la caché), filtrar la pestaña Red por la ruta `/stream`, mirar el código y, si es 200 pero no hay imagen, buscar en la Consola un mensaje de *Content Security Policy*.

Paneles de planta: la misma web a gran escala

Los paneles de supervisión industrial (**SCADA** y **HMI**) modernos son, en esencia, aplicaciones web conectadas por WebSocket a controladores, con **alarmas** priorizadas (norma ISA-18.2) y **roles** de usuario. La celda de Aether sigue esa receta en pequeño; lo

que no tiene todavía es autenticación por usuarios y roles reales (hay una sola cuenta de laboratorio).

Preguntas de repaso: web y navegador

1. ¿Qué diferencia hay entre un `GET` y un `POST`, y por qué `GET /inventory` no puede modificar nada?
2. ¿Qué dos protecciones lleva la cookie `celda_session`?
3. ¿Por qué el video de la cámara no viaja como un archivo de video normal sino como una "foto interminable"?
4. Explicar por qué el video del agente se bloqueó aunque el encabezado de la CSP lo permitía.
5. ¿Qué significa que `fetch("/robot/tts")` no falló y aun así el audio no sonó?
6. ¿Qué códigos devuelve el guardia público ante una cámara, una ruta oculta y un exceso de peticiones?

Capítulo 12: La terminal, Linux y los procesos

Qué ventana es cuál, cómo se mantiene vivo un programa cuando se corta la conexión, qué son las variables de entorno y por qué hay cosas que el equipo se prohíbe hacer en el Jetson.

Tres ventanas, tres idiomas

Aether se opera desde tres tipos de ventana de texto, y confundirlas es la fuente de errores más común:

Ventana	Sistema	Dónde se usa	Ejemplo
PowerShell	Windows	La laptop	`.\iniciar_aether.ps1`
Git Bash	Windows (imita a Linux)	La laptop, para comandos de Linux o de Git	`ls`, `git status`
Bash (terminal de Linux)	Ubuntu o Raspberry Pi OS	El Jetson y la Pi, normalmente por **SSH**	`bash ~/aether/scripts/jetson/iniciar-agente.sh`

Una **terminal** (o **consola**) es una ventana donde se escriben comandos y se leen respuestas, sin ratón. Una **shell** es el intérprete que entiende esos comandos (PowerShell y Bash son dos shells distintas). Por eso, en este libro, cada comando lleva su etiqueta de máquina y ventana: un comando de Bash pegado en PowerShell no hace lo mismo. Las diferencias de sintaxis que ya causaron tropiezos en el proyecto: PowerShell usa `$env:NOMBRE = "valor"` para una variable, Bash usa `export NOMBRE=valor`; las rutas de Windows llevan `\` y las de Linux `/`; y los guiones de PowerShell (`.ps1`) exigen `.\` para ejecutarlos desde la carpeta actual.

Moverse y mirar: cinco comandos que bastan

En Bash casi todo lo cotidiano se resuelve con cinco comandos:

```
pwd                       # ¿en qué carpeta estoy?
ls -la                    # qué hay aquí (con archivos ocultos)
cd ~/aether/backend       # entrar a una carpeta (~ es la carpeta personal)
cat archivo.txt           # mostrar un archivo
tail -n 20 ~/aether-agente.log   # las últimas 20 líneas de un registro
```

Estos tres suman: `grep texto archivo` (buscar texto), `curl URL` (pedir una página, como un navegador sin pantalla) y `top` (qué procesos gastan CPU).

Qué es un proceso y por qué "prendido" no es una luz

Un **programa** es un archivo; un **proceso** es ese programa **corriendo** en este instante, con su memoria y su identificador numérico (**PID**). "El servidor está prendido" quiere decir "existe un proceso del servidor". Esto explica síntomas de Aether:

- **"Connection refused" al `curl`**: no hay ningún proceso escuchando en ese puerto.
- **Spotify avisó que el puerto 8888 estaba ocupado** (bitácora del Jetson, sesión 5): había otro `uvicorn` vivo de una prueba anterior. Dos procesos no pueden escuchar en el mismo puerto.
- **El primer intento de arrancar el servidor falló con "No module named 'sqlalchemy'"**: se usó el Python del sistema en lugar del Python del **entorno virtual** del proyecto (`backend/.venv`), que es el que tiene instaladas las librerías (capítulo 13).

Para ver y terminar procesos, `[Jetson · SSH]`: `ps aux | grep uvicorn` los lista; `kill <PID>` pide que uno termine.

Sobrevivir a una ventana cerrada: tmux

Un programa que se inicia en una ventana de SSH **muere cuando la ventana se cierra** (o la conexión se corta). Para el Jetson y la Pi eso es inaceptable: el agente debe seguir vivo. La solución es **`tmux`**, un "multiplexor de terminales": crea una **sesión** de terminal que vive en el equipo, independiente de la conexión. Los tres comandos del proyecto, `[Jetson · SSH]`:

```
tmux new -s trabajo            # crear una sesión llamada "trabajo"
# (soltarla sin detener nada: Ctrl+B y luego D)
tmux attach -t trabajo         # volver a ella
tmux ls                        # listar sesiones
```

El script `iniciar-agente.sh` ya lo hace por su cuenta: crea una sesión llamada **`agente`**, arranca el agente adentro y, si ya existía, dice "El agente ya corre" y no hace nada (se puede repetir sin miedo).

El sistema operativo y su carpeta de servicios

En Linux, los programas que deben estar siempre corriendo suelen registrarse como **servicios** de **systemd** (el gestor que arranca y vigila servicios, y los reinicia si se caen). El equipo **decidió no usar systemd todavía**: la ADR 0012 lo descarta "por ahora" porque *añade otra pieza que mantener* y `tmux` ya es el procedimiento; se anota como posible mejora cuando el agente resulte estable, y para la celda (que debe operarse sola) se reconsidera (**PENDIENTE (P-25)**, capítulo 8).

Variables de entorno y el archivo .env

Una **variable de entorno** es un nombre con un valor que el sistema entrega a cada programa que arranca: sirve para **configurar sin tocar el código**. Aether se configura casi todo así (capítulo 13). Dos formas de definirlas:

- **Archivo** `.env` en `backend/` (en la laptop): se carga solo al arrancar y **nunca se sube a Git** (está en `.gitignore`). Ahí van, por ejemplo, las claves y las direcciones de los agentes.
- **Archivo** `~/.aether-agent.env` en el Jetson: se copia de una plantilla (`scripts/jetson/aether-agent.env.example`) y se edita. Su plantilla lo advierte: "*NO pongas secretos aquí*".

Hay una regla de higiene que aparece en la guía del Jetson: **un valor mal escrito debe detener el programa al arrancar con un mensaje que nombre la variable** (`backend/settings.py`), en lugar de fallar misteriosamente más tarde.

Scripts: comandos guardados en un archivo

Un **script** es un archivo con una lista de comandos. Aether tiene tres para arrancar, cada uno para su máquina:

Script	Máquina	Qué hace
`iniciar_aether.ps1`	Laptop	Comprueba entorno, clave de la celda (sin mostrarla), puerto libre, perfil de red, firewall y voz; imprime las IP; arranca el servidor
`scripts/jetson/iniciar-agente.sh`	Jetson	Comprueba `tmux`, el entorno virtual, la cámara, que `gpiod` **no** esté y la configuración; arranca el agente en una sesión `tmux`

Script	Máquina	Qué hace
`scripts/ver-aether.sh`	Jetson (en el escritorio por VNC)	Abre Firefox en la primera laptop que responda de una lista de IP

Sus principios son notables: **no instalan nada, no usan `sudo`, no abren puertos ni cambian el firewall**; si algo falta, dicen **qué** y **con qué comando** arreglarlo. `iniciar_aether.ps1` está escrito en **ASCII a propósito**: PowerShell 5.1 lee mal los acentos de un archivo sin marca de orden de bytes (BOM).

Lo que se pierde al reiniciar el Jetson

Al apagar y encender el Jetson desaparecen las sesiones de `tmux`, así que **hay que volver a arrancar el agente** con `iniciar-agente.sh`. Lo que **sí persiste**: los archivos (`~/.aether-agent.env`, el repositorio en `~/aether`, los entornos virtuales), la regla de que `suspend`, `sleep` e `hibernate` quedaron enmascarados (el equipo lo configuró; el comando exacto **no se registró**) y las librerías instaladas (por ejemplo `libcusparseLt`). Qué más sobrevive al reinicio (por ejemplo el modo de energía de `nvpmodel` o el WiFi apagado con `nmcli radio wifi off`) **no está comprobado**: la bitácora no registra una prueba de reinicio, así que conviene volver a verificarlo con `sudo nvpmodel -q` después de cada encendido.

Prohibiciones: lo que el equipo se impide en el Jetson

La sección "Precauciones" de `COMANDOS_LINUX.md` es una lista de cosas que **no** se hacen y por qué. Vale la pena leerla como ejemplo de disciplina:

- **No instalar `gpiod` de Python** (capítulo 9): los números de pin son de Raspberry Pi.
- **No correr `sudo apt upgrade`:** un hilo del foro de NVIDIA reporta errores de memoria CUDA tras actualizar en esa versión; no está comprobado en este equipo, pero el costo de equivocarse es alto. Antes de cualquier `sudo`, se simula con `-s`.
- **No tocar `snapd` ni usar `--no-sandbox`** en los navegadores.
- **No instalar `opencv-python` por pip** y mantener `numpy<2`.
- **Guardar salidas importantes** con `| tee ~/archivo.txt`.
- **Repositorio público:** nunca subir `.env`, tokens, `aether.db`, pesos de modelos (`.pt`, `.engine`), fotos del dataset, contactos ni llaves; revisar `git diff --stat` antes de cada `push`.

Por qué las plantas no arrancan nada a mano

En una planta, nadie escribe comandos para encender un servicio: el sistema operativo arranca los servicios solo y un **supervisor** los reinicia si fallan; hay **monitoreo** que avisa si una máquina deja de responder, y los cambios pasan por un procedimiento controlado. El camino de Aether (scripts + `tmux`) es el mínimo útil para un prototipo, con la ventaja de ser explicable en una tarde; el salto a systemd, contenedores y monitoreo es la evolución natural (capítulo 26).

Preguntas de repaso: terminal y procesos

1. ¿Qué diferencia hay entre un programa y un proceso, y qué dice `Connection refused` sobre eso?
2. ¿Por qué un programa lanzado por SSH muere si se cierra la ventana y cómo lo evita `tmux`?
3. ¿Qué se pierde, y qué se conserva, al reiniciar el Jetson?
4. ¿Por qué `backend/.env` nunca se sube a Git?
5. Dos comandos que el equipo se prohíbe en el Jetson y la razón de cada uno.
6. ¿Para qué sirve que `iniciar-agente.sh` se pueda repetir sin miedo?

Capítulo 13: Python, FastAPI y la configuración

Por qué casi todo el software de Aether está escrito en Python, cómo un archivo de funciones se vuelve un servidor web, qué es un contrato entre programas y por qué una variable mal escrita debe tumbar el arranque.

Una librería de más que tumbó el arranque

La bitácora de la sesión 4 del Jetson (21 de octubre de 2026) registra algo muy aleccionador: un paquete llamado `ultralytics-thop`, que nadie había pedido directamente, **arrastraba un `torch` de PyPI** (sin soporte de la GPU de NVIDIA) al instalar los requisitos, y con el orden previsto habría pisado el `torch` bueno. Se detectó simulando la instalación antes de ejecutarla. La anécdota resume el capítulo: **el software moderno es una pirámide de dependencias**, y casi todo problema de un entorno Python se resuelve entendiendo *quién instala qué*.

Por qué Python manda aquí

Python es un lenguaje de programación legible y con librerías para casi todo lo que Aether necesita: visión por computadora (OpenCV, `ultralytics`), servidores web (FastAPI), bases de datos (SQLAlchemy) y reconocimiento de voz (Vosk). El equipo lo eligió por una razón práctica: **un solo programador (Mario) debe poder mantener toda la capa de software**, y Python minimiza el código por función. Su costo: es más lento que un lenguaje compilado, por eso la parte de tiempo real (motores) vive en un microcontrolador aparte (capítulo 5).

El proyecto tiene tres grandes zonas de código:

Carpeta	Qué contiene
`backend/`	El servidor de la laptop, los agentes del Jetson y de la Pi, el servicio de voz, la base de datos
`edge/`	Percepción e inventario "de borde": los scripts originales de cámara, YOLO, códigos, ArUco y el motor de agregación
`dashboard/`	El panel web en HTML, CSS y JavaScript (capítulos 11 y 19)

(Además: `firmware/` está vacío por ahora, `contracts/` guarda los contratos de comunicación, `tests/` las pruebas y `scripts/` las herramientas.)

El entorno virtual: una caja de herramientas por proyecto

Si todas las librerías se instalaran en el mismo lugar del sistema, dos proyectos que pidieran versiones distintas de la misma librería chocarían. El **entorno virtual** (**venv**) es una carpeta aislada con su propio Python y sus propias librerías. Aether tiene dos en la laptop (`edge/.venv` y `backend/.venv`) y uno por equipo en el Jetson y la Pi. Para ejecutar con el entorno correcto no hace falta "activarlo": se llama al Python que vive dentro, `[Laptop · PowerShell]`:

```
backend\.venv\Scripts\python.exe -m pytest tests\backend -q
```

(El 24 de octubre ese comando dio **345 pruebas aprobadas y 2 fallidas**; capítulo 22.) El **archivo de requisitos** (`requirements.txt`) lista lo que cada entorno instala, y el proyecto lo parte a propósito:

Archivo	Para quién
`backend/requirements.txt`	La laptop: `fastapi`, `uvicorn`, `sqlalchemy`, `opencv-contrib-python`, `ultralytics`, `pyzbar`, `python-dotenv`
`backend/requirements-dev.txt`	Solo desarrollo: `ruff` (revisor de estilo), `pytest` (pruebas), `httpx` (lo necesita el cliente de pruebas)
`backend/requirements-voice.txt`	Solo la laptop con micrófono: `sounddevice`, `vosk`
`backend/requirements-jetson.txt`	El Jetson: sin OpenCV ni `torch` (vienen del sistema y de NVIDIA), con `numpy<2`
`backend/requirements-pi.txt`	La Pi 4: `fastapi` y `uvicorn`; OpenCV y NumPy salen de `apt`

Es una muestra de disciplina: **"No se instala driver de PostgreSQL todavía porque esa decisión no está cerrada"** (comentario del propio archivo). Es el principio de *no fabricar dependencias* hecho práctica.

FastAPI: convertir funciones en direcciones web

FastAPI es una librería que convierte funciones de Python en **endpoints** (direcciones del servidor que responden a peticiones). Cada función lleva un "decorador" que dice el verbo y la ruta. Ejemplo real de `backend/main.py`:

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

Con eso, `GET /health` responde `{"status": "ok"}`. FastAPI usa **tipos** para validar datos automáticamente: si una ruta espera un cuerpo con un campo `password` de tipo texto y llega otra cosa, responde `422` sin que el programador escriba la validación. El servidor central de Aether tiene más de **25 rutas** (inventario, historial, cámaras, celda, Lumina, llamadas, texto a voz, dataset…); `GET /docs` muestra la lista interactiva.

Uvicorn: el motor que mantiene abierta la puerta

FastAPI **define** qué hacer; **Uvicorn** es el programa que **escucha** el puerto y le pasa cada petición. Por eso se arranca así, `[Laptop · PowerShell]`, desde `backend/`:

```
.\.venv\Scripts\python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000
```

`main:app` significa "el objeto `app` del archivo `main.py`". `--host 0.0.0.0` acepta conexiones de otros equipos; `--port 8000` es el puerto. (El script `iniciar_aether.ps1` hace todo esto con comprobaciones, capítulo 12.)

Contratos: cuando dos programas deben ponerse de acuerdo

Un **contrato** es un acuerdo escrito sobre qué datos se mandan y con qué forma. Aether tiene varios (`contracts/`), y el más importante es el de los **agentes** (`contracts/agents_api.md`): lo que el Jetson manda a la laptop. Su fuente de verdad es un archivo de Python (`backend/agent_contracts.py`) con **modelos pydantic** (la librería de validación de FastAPI). Un ejemplo, resumido, del modelo `Observation`:

Campo	Tipo y regla
`v`	Siempre `1` (versión del contrato)

Campo	Tipo y regla
`event_id`	Texto de 8 a 64 caracteres (letras, números, `_`, `-`): la identidad única del evento
`timestamp`	Fecha con **zona horaria** obligatoria (UTC)
`source`	Quién la manda: letras minúsculas, números, `_`, `-` (hasta 32)
`detected_class`	Qué se vio (1 a 64 caracteres)
`code_data` / `code_read`	El código leído (o ninguno) y si se leyó
`confidence`	Número entre 0 y 1
`location_marker_id`	El ArUco visible, o ninguno

Reglas estrictas del contrato: **se prohíben campos extra** (`extra="forbid"`), un lote admite **hasta 100** observaciones, el cuerpo de una petición no puede pasar de **64 KiB** (si no, `413`), y **todo mensaje lleva la versión** para poder evolucionar sin romper. Ninguna petición ejecuta código: solo se validan y se guardan datos (capítulo 16).

Configuración centralizada: el archivo settings

Casi todo en Aether se configura con **variables de entorno** (capítulo 12), y `backend/settings.py` es la única puerta de entrada para leerlas. Sus funciones (`env_bool`, `env_int`, `env_float`, `env_choice`, `env_url`, `env_cidrs`) **validan al arrancar** y, si algo está mal, lanzan un `ConfigError` que **nombra la variable y dice qué se esperaba**: "*AETHER_DETECT_MAX_HZ debe ser a lo más 120, no 500*". El servidor además se niega a arrancar si una dirección de agente es inválida (`lifespan` en `main.py`). Beneficio: la falla ocurre en el segundo cero, con un mensaje claro, y no a mitad de una demostración.

La **tabla única** de variables está en `docs/variables_entorno.md`. Una selección de las más importantes:

Variable	Defecto	Para qué
`AETHER_PERCEPTION_AGENT_URL`	vacía	Dirección del agente del Jetson; sin ella las páginas usan la cámara de la laptop
`AETHER_LAN_CIDRS`	rangos privados	Redes de las que se aceptan agentes y voz
`AETHER_DETECT_MAX_HZ`	`10`	Máximo de inferencias por segundo del agente (valor de diseño, no medido)

Variable	Defecto	Para qué
`AETHER_OUTBOX_INTERVAL_S`	`1`	Cada cuánto se juntan y mandan las observaciones
`AETHER_VOICE_VAD_THRESHOLD`	`500`	Umbral de energía de la voz
`CELDA_HMI_CLAVE`	—	**Secreto**: contraseña de la celda
`ADMIN_ACTION_PASSWORD`	`aether-dev`	**Secreto** de desarrollo para retirar inventario

Pruebas automáticas: un guardia que revisa por uno

Una **prueba automática** es un programa que ejecuta otra parte del código y comprueba que haga lo esperado. Aether usa **pytest**. Un patrón del proyecto: las pruebas **no necesitan cámara ni red**, porque las dependencias pesadas (la cámara, el modelo, el reloj) se pasan **por parámetro** y las pruebas usan **dobles** (versiones falsas). El capítulo 22 las detalla.

Qué leer primero cuando uno se pierde en el código

Un recorrido sugerido para entender el servidor:

1. `backend/main.py`: la lista de rutas y qué hace cada una (el principio del archivo explica que es el "punto de entrada mínimo").
2. `backend/agent_contracts.py` y `contracts/agents_api.md`: lo que los agentes pueden mandar.
3. `backend/cell_line.py` y `backend/celda_agent.py`: la celda (capítulo 7).
4. `backend/perception_pipeline.py`: cómo se analiza un cuadro (capítulo 14).
5. `backend/voice/`: el servicio de voz (capítulo 15).
6. `docs/variables_entorno.md` y `docs/guia_arranque.md`: cómo se configura y se arranca todo.

Hacia dónde crece Python en un producto real

En productos mayores se agregan **tipado estático** (`mypy`), **empaquetado** del proyecto, **contenedores** (Docker) para reproducir el entorno exacto, y servidores de producción con varios procesos. Aether ya usa tipos de FastAPI y `ruff` (con `line-length = 100` y

`target-version = py310`, para correr en Python 3.10 del Jetson), y deja Docker como evolución (capítulo 26).

Preguntas de repaso: Python y servidores

1. ¿Qué es un entorno virtual y por qué la Pi usa `--system-site-packages`?
2. ¿Qué hace el decorador `@app.get("/health")`?
3. ¿Cuál es la diferencia entre FastAPI y Uvicorn?
4. ¿Qué pasa si una observación llega con un campo extra no previsto, y por qué es buena idea?
5. ¿Por qué una variable de entorno mal escrita debe detener el arranque?
6. ¿Qué cinco archivos conviene leer primero para entender el servidor?

Capítulo 14: Visión en el borde

Cómo una cámara se convierte en datos: detectar con YOLO, leer códigos, ubicarse con ArUco, entrenar el modelo de las piezas y por qué conviene no detectar más rápido de lo útil.

Una prueba en la que nada se detectó

En la prueba de humo del agente de percepción en una laptop (cámara web, YOLO en CPU, 10 segundos), todo salió bien: **30.0 FPS** de cámara y de entrega, 300 cuadros recibidos, inferencia de 47 ms. Pero la bitácora anota algo que un lector apresurado se salta: **la cámara no vio ningún objeto, así que se generaron 0 observaciones**. El envío de datos se probó con **dobles** (versiones falsas), no con detecciones reales. Esa franqueza resume la mentalidad del capítulo: *funcionar no es lo mismo que detectar*, y las cifras de velocidad dicen poco si la cámara apunta a una pared.

Por qué la inteligencia vive en el robot y no en la nube

Visión en el borde (*edge*) significa procesar las imágenes **en el mismo equipo que tiene la cámara**, no mandarlas a un servidor lejano. Las razones, en el contexto de Aether:

- **Ancho de banda:** un video a 30 cuadros por segundo satura una red WiFi; los datos que salen del análisis son unos cientos de bytes (capítulo 1).
- **Latencia:** el robot debe frenar y decidir en milisegundos, sin esperar a internet.
- **Autonomía:** el sistema funciona aunque no haya conexión (el proyecto lo define como principio: "*el robot funciona standalone*").
- **Privacidad:** el video no sale del equipo.

De la luz a los píxeles

Una cámara digital entrega una **imagen** como una cuadrícula de **píxeles**, cada uno con tres números (rojo, verde y azul, de 0 a 255). Un cuadro de 640 × 480 son 307 200 píxeles y casi un millón de números. La cámara USB del robot entrega varios cuadros por segundo (**FPS**, capítulo 6). **OpenCV** es la librería de visión que lee la cámara, recorta, dibuja y convierte imágenes; el proyecto la usa en el Jetson desde el sistema (versión 4.8.0, **VERIFICADO**) y no desde `pip`.

Detectar con YOLO: una pasada, muchos objetos

YOLO (*You Only Look Once*, "solo miras una vez") es un tipo de red neuronal que, dada una imagen, devuelve **una lista de objetos**: una **caja** (rectángulo), una **clase** ("persona", "silla", "computadora2") y una **confianza** (de 0 a 1). Se llama así porque analiza la imagen completa **de una sola pasada**, en vez de recorrerla por pedazos, lo que lo hace lo bastante rápido para video. Aether usa `ultralytics`, la librería de la versión **YOLOv8**, con el modelo pequeño `yolov8n.pt` (6.5 MB, se descarga solo la primera vez y no se sube a Git).

El flujo completo de una observación, que es el corazón de la percepción (`backend/perception_pipeline.py`, clase `PerceptionEngine`), se resume así:

1. Se busca un **marcador ArUco** en todo el cuadro y se anota su identificador, si hay.
2. YOLO detecta objetos y entrega sus cajas.
3. Dentro de **cada caja** se recorta la imagen y, con la librería `pyzbar`, se intenta **leer un código de barras o QR**.
4. Se arma un registro (una **Observation**) por detección: hora (UTC), clase, código leído (si hubo), si se leyó, confianza y el ArUco visible.
5. Se **dibuja** la caja y la etiqueta sobre la imagen para el video.

Una regla de diseño importante (**Decisión 1**): **ninguna detección se descarta por no tener código legible**. Detectar y contar es independiente de identificar. Y una advertencia de terminología (principio 4.2): **"conteo visible detectado" no es "conteo real de inventario"**; el sistema cuenta lo que ve, no lo que existe.

Formatos de modelo: .pt, ONNX y por qué importa

Un modelo entrenado se guarda en un archivo. Dos formatos aparecen en Aether:

Formato	Qué es	Dónde se usa
`.pt`	Formato de **PyTorch** (la librería de redes neuronales); necesita `torch` y `ultralytics` para correr	Laptop y Jetson (`yolov8n.pt` y `parts.pt`)
ONNX (`.onnx`)	Formato **neutral** de modelos; lo puede leer OpenCV (`cv2.dnn`) sin `torch`	La Raspberry Pi 4 (`parts.onnx`)

La Pi no tiene `torch`, así que el modelo de piezas se **exporta** a ONNX en la laptop y se copia (capítulo 8). Una verificación real: el modelo ONNX juzgó **igual que el original en 19 de 19 fotos**, con un IoU medio de 0.994 (el **IoU**, *Intersection over Union*, mide cuánto coinciden dos cajas: 1.0 es idéntica). Una validación al arrancar (`line_model.py`) comprueba que el modelo trae las clases de `classes.txt` y, si no, se niega a iniciar con un mensaje que pide volver a exportar.

El límite de frecuencia: no detectar más rápido de lo útil

Aunque la cámara entregue 30 cuadros por segundo, **no hace falta correr YOLO 30 veces por segundo**: consume CPU o GPU y casi no aporta. El agente separa las dos cosas con dos piezas (`perception_pipeline.py`):

- **El video** se entrega a su ritmo, con **las últimas cajas conocidas** dibujadas encima.
- `AsyncDetector` corre YOLO en **otro hilo**, siempre sobre **el cuadro más reciente**; si sigue ocupado, **descarta** el cuadro nuevo (no los acumula), y su frecuencia máxima se limita con `AETHER_DETECT_MAX_HZ` (por defecto **10 por segundo**; es un **valor de diseño, no medido**).

Resultado en el Jetson (**MEDIDO**, reportado): 29.0 FPS de cámara y 29.1 de entrega con una inferencia de **68.0 ms** de promedio, y en la prueba de laptop 47 ms. Conviene recordar que el modo `sync` (analizar cada cuadro) era el comportamiento histórico de la laptop y sigue disponible.

Lo que se ve en pantalla y lo que se calcula

Dos cosas distintas, que a veces se confunden:

- **Lo que se ve**: el video con cajas y etiquetas.
- **Lo que se calcula y se manda**: observaciones en JSON (capítulo 16).

Por ejemplo, el letrero de la cámara de la celda dice `PIEZA REAL · computadora2 · PASS` o `NO ES PIEZA REAL · FAIL` o `SIN PIEZA`: esa es la **vista**; lo que decide al pistón son cinco juicios iguales seguidos (capítulo 7).

El modelo de las piezas: entrenar con pocas fotos

Para la celda, el equipo entrenó un modelo propio, `parts.pt`, que reconoce **tres piezas registradas** (`computadora1`, `computadora2`, `computadora3`, las ECU). El flujo:

1. En la página `dataset.html` (capítulo 19) se **suben fotos** y se **dibujan cajas** sobre cada pieza, con su clase. El servidor las guarda en `edge/perception/parts/` (`POST /dataset/samples`); rechaza fotos casi idénticas a una ya guardada (para no inflar el dataset con duplicados).
2. Las fotos se reparten **4 de cada 5 a entrenamiento y 1 a validación**, con una regla fija calculada a partir del nombre de la foto (para que la misma foto no cambie de conjunto entre ejecuciones).
3. `edge/perception/train_parts.py` entrena. Si ya existe un `parts.pt`, **continúa desde ahí** y **congela las primeras 10 capas** de la red: en CPU solo se ajusta la "cabeza", que es lo que distingue las piezas nuevas. **Publica el peso nuevo solo si su mAP50 supera al anterior.**
4. Se escribe `parts_report.json` con el resultado.

El **mAP50** de la última corrida registrada es **0.995** (entrenamiento del 15 de octubre de 2026). Es una nota muy alta, y por eso conviene dudar de ella: con unas decenas de fotos de tres piezas, el modelo "se aprende" el conjunto. El repositorio conserva 19 fotos en la bandeja de entrada y **no se probó con objetos que no sean pieza**: el FAIL real **no está medido**. Los pesos (`.pt`, `.onnx`) y las fotos **no se suben a Git** (están en `.gitignore`).

Errores típicos de visión y cómo leerlos

Síntoma	Causa probable	Dónde se vio en el proyecto
0 observaciones aunque corre	La cámara no ve ningún objeto conocido	Prueba de humo del agente en laptop
Video a 2304 × 1536 y lento	OpenCV abrió la cámara por GStreamer en vez de V4L2	Backend de prueba en el Jetson
FPS bajos	Poca luz: la cámara alarga la exposición	Pruebas del 21 de octubre
ArUco con un identificador "válido pero equivocado"	Reflejo de luz sobre el marcador	Informe integrador (hallazgo del módulo de percepción)
Dos cámaras, un solo dispositivo	Percepción y visión de línea usan ambas `/dev/video0` por defecto	Bitácora, sesión 5: no deben abrirse a la vez
FAIL falso que dispara el pistón	Umbral de estabilidad demasiado bajo	Documentación: no bajar `LINE_ESTABLES` de 3 sin probar

Cuánto acierta: lo que falta medir

Los **criterios de aceptación** del proyecto (`docs/architecture/decisions/criterios_aceptacion.md`) dejan **pendientes** los umbrales numéricos de percepción: la tasa mínima de detección de YOLO y de lectura de código, el error de posición por ArUco y los FPS mínimos. Existe un script manual (`edge/perception/test_detection_rate.py`) para medir la tasa de detección con un conjunto fijo de objetos conocidos; **no se ha corrido y registrado un resultado**. Qué umbral se exigirá lo define el equipo con datos reales: **PENDIENTE (P-22)**.

Quién lo hace en grande

En la industria, la visión por computadora de planta corre en **cámaras inteligentes** o computadoras industriales junto a la línea, con iluminación **controlada** (el factor que más mejora la fiabilidad), modelos optimizados con **TensorRT** o equivalentes y conjuntos de datos de **miles** de ejemplos con defectos reales. Lo que Aether hace con tres piezas y unas decenas de fotos es la versión de laboratorio de ese flujo: datos → entrenar → exportar → desplegar → medir.

Preguntas de repaso: visión

1. ¿Qué devuelve YOLO sobre una imagen y por qué se dice que "mira una sola vez"?
2. ¿Por qué ninguna detección se descarta por no tener código legible?
3. ¿Por qué la Raspberry Pi usa un modelo ONNX y no el `.pt`?
4. ¿Qué hace `AsyncDetector` cuando llega un cuadro y el detector sigue ocupado, y por qué?
5. ¿Qué significa un mAP50 de 0.995 con un dataset chico y qué prueba falta?
6. Citar dos causas de bajos FPS que **no** son defectos del modelo.

Capítulo 15: La voz de Lumina

Quién es Lumina, cómo oye y cómo habla, por qué un reconocedor de voz no pudo escribir su nombre, qué se midió y qué no, y cómo diagnosticar cuando no escucha.

Un reconocedor que no sabe escribir "Lumina"

El 23 de octubre de 2026 se descubrió que el asistente del proyecto no despertaba. Al decir "Lumina", el reconocedor de voz local escribía **"nómina"**, **"luz mina"**, **"lumínica"**, **"luminoso"** o **"mina"**, y el programa que esperaba la palabra exacta nunca la veía. El diagnóstico fue una consulta directa al modelo: **"lumina", "lumia" y "luminia" no están en su vocabulario**, mientras que "ilumina", "nómina", "mina", "lumínica" y "luminoso" sí. Un reconocedor solo puede escribir las palabras que conoce; el nombre propio del asistente no era una de ellas.

Es una lección general de la voz por computadora que el resto del capítulo desarrolla: **reconocer voz no es oír, es elegir entre palabras de un diccionario.**

Cuatro piezas, una conversación

Lumina es el nombre del asistente de Aether: una **cara animada** en la pantalla (`robot.html`), con **voz**, **cámara** y un **modelo de lenguaje local**. Se presenta como "el robot móvil de Aether". Para que conteste una frase hay cuatro piezas, que viven en máquinas distintas:

Pieza	Qué hace	Dónde corre
Oído	Capta el micrófono y escribe lo que se dijo	**La laptop** (servicio de voz), o el navegador con reconocimiento propio
Cerebro	Decide qué responder	**La laptop** (servidor FastAPI + **Ollama**, un programa que corre modelos de lenguaje en local)
Voz	Convierte el texto en sonido	**ElevenLabs** (un servicio en la nube) con respaldo de la voz del navegador
Cara	Muestra expresiones, subtítulos y reloj	El navegador (`robot.html`, `robot.js`)

La exclusión mutua importa: en el Jetson, correr un **modelo de lenguaje** y la **percepción con YOLO** a la vez degradaría a ambos, porque comparten un solo cómputo; por eso `robot.html` tiene un **Modo Conversación** (la cámara y YOLO apagados) y un **Modo**

Cámara, que nunca se activan juntos, y el modelo de lenguaje se queda en la **laptop**.

El cerebro: tres capas de respuesta

Cuando llega una pregunta a `POST /robot/ask`, el servidor la resuelve en este orden:

1. **Comandos de escritorio** (`lumina_desktop.py`): abrir un programa, poner música en Spotify, guardar un archivo, calcular. Se reconocen con **patrones de texto**, **sin modelo de lenguaje**. Lo delicado (cerrar una app, guardar un archivo) **pide confirmación por voz** ("sí" o "no").
2. **Respuestas precargadas**: si la pregunta coincide con alguna típica de una presentación, se contesta al instante con una respuesta fija.
3. **Ollama** (`llama3.2:1b`, un modelo de 1 000 millones de parámetros que corre en la laptop), con un **tope duro** de 60 tokens de respuesta, **elegido probando** (con 40, una respuesta real se cortó a media palabra) y una **limpieza** posterior: se descartan listas (no sirven para decirse en voz alta) y se quitan oraciones de "meta-comentario" donde el modelo describe cómo debe responder.

Si Ollama no responde, el servidor devuelve **503** con un mensaje que explica cómo arreglarlo (`ollama serve`, `ollama pull llama3.2:1b`); **nunca inventa** una respuesta cuando el modelo real no contestó (principio de honestidad).

El **mensaje de sistema** de Lumina (su "personalidad") le exige responder **en una sola oración corta, tierna y cálida**, decir con honestidad lo que no sabe, **no fingir** capacidades que no tiene (como ver, si la cámara está apagada) y **aclarar que la línea de manufactura todavía no existe físicamente** como parte de lo que ya hace. Es una aplicación directa del principio de **honestidad de producto**.

Llamadas telefónicas, con candado

Lumina puede "llamar" a un contacto (`POST /robot/call`, mediante un servicio externo de mensajería). Las salvaguardas del código, dignas de copiar: **solo marca números de una lista local** (`backend/contactos.json`, fuera de Git) con formato internacional válido, **nunca un número que llegue por la red** (cualquiera en la red puede llamar al endpoint), y hay un **límite de 3 llamadas por ventana de 10 minutos**. Como el servicio lee el texto una sola vez al contestar, el mensaje se repite dos veces y lleva una introducción corta.

La voz de salida y su respaldo

`POST /robot/tts` manda el texto a **ElevenLabs** (modelo `eleven_multilingual_v2`) y devuelve un **MP3**. La clave vive solo en `backend/.env`; si falta o falla, responde 503 y el navegador **cae a** `speechSynthesis`, la voz integrada del navegador. El 23 de octubre se descubrió que el MP3 se descargaba bien pero el navegador **no lo reproducía** por la política de contenido (`media-src`); el diagnóstico está en el capítulo 11. Si la clave de ElevenLabs es válida y tiene créditos **no se verificó**.

El oído: del micrófono al texto

El micrófono de Lumina **es el de la laptop**, atendido por un proceso aparte: `python -m voice.service` (carpeta `backend/voice/`). La ruta de una frase:

1. `audio.py`: captura el micrófono con `sounddevice` (audio **PCM**, 16 kHz, mono, 16 bits).
2. `core.py`: un **VAD** (*Voice Activity Detector*, "detector de actividad de voz") de **energía** decide si hay voz: mide el volumen (RMS) y lo compara con un **umbral** (`AETHER_VOICE_VAD_THRESHOLD`, 500 por defecto). Guarda solo un anillo corto en memoria para no cortar el principio de la frase.
3. `engine.py`: el reconocedor, un adaptador intercambiable: **Vosk** (por defecto, ligero, 100 % local) o **Whisper** (más preciso y pesado, opcional).
4. `wake.py`: si la frase **empieza** con algo que suena a "Lumina", se reescribe como "lumina" (ver abajo).
5. `hublink.py`: manda **solo el texto** por WebSocket al servidor (`/voice/ws`), que lo reenvía a las páginas de Lumina que no tienen reconocimiento propio (por ejemplo Firefox en el Jetson).

Reglas de privacidad del servicio, **VERIFICADAS por pruebas automáticas**: **el audio nunca se guarda en disco, no se escribe en los registros y no sale del proceso**; hay un **aviso persistente** en la ventana (`● MICRÓFONO ACTIVO · 15:04:05`) y una **notificación de Windows** al activar, pausar y apagar; la tecla `p` **pausa y suelta el micrófono** (el icono de micrófono de Windows se apaga); `q` sale; `w` despierta a Lumina sin hablar.

Anti-eco: cuando Lumina habla, la página avisa al servicio (`speaking`) y este ignora el audio hasta **0.8 s después** (`AETHER_VOICE_ECHO_TAIL_S`), para que Lumina no se oiga a sí misma.

La palabra de activación y su problema de vocabulario

La solución a "nómina" está en `backend/voice/wake.py`: **si la frase empieza** (o empieza tras "hola"/"oye") con una forma que suena a "Lumina", esa palabra se reescribe como "lumina" **antes de mandar el texto**, y el código de la página no cambia. Acepta una lista corta de formas observadas ("nómina", "mina", "lumínica", "luminoso/a"), palabras que empiezan con "lu", tienen 5 letras o más y están a **una letra** de "lumina" (según la **distancia de Levenshtein**, que cuenta cuántas letras hay que cambiar, quitar o poner), y pares como "luz mina" o "lo mina". **No** acepta "la mina", "una mina", "lámina", "lucía", "luna", "camina" ni "elimina".

MEDIDO con **voz sintética de Windows** (una sola voz, ruido simulado), 30 frases con "Lumina" y 25 sin ella:

Condición	Regla exacta (antes)	Regla tolerante (ahora)	Falsos positivos
Silencio	0/30	**28–30/30**	1/25 → 4/25
Ruido blanco +10 dB	27/30	28/30	1/25 → 4/25
Murmullo (3 voces) +10 dB	2/30	**11/30**	2/25 → 4/25

Tres lecciones: en silencio la regla **resuelve** el problema; el **nivel de voz no era el problema** (a RMS 300 y 1000 Vosk acierta igual); y con **conversaciones de fondo** el límite deja de ser la regla (Vosk escribe lo que dicen las voces de fondo y ninguna lista de variantes lo arregla). El costo: frases reales que **empiezan** con "nómina…" o "mina…" activarán a Lumina; los falsos positivos conocidos quedan **a la vista como pruebas**. Se apaga con `AETHER_VOICE_WAKE_FUZZY=0`.

Se probaron **10 palabras alternativas**; las que sí están en el vocabulario ("atenea", "aurora") se reconocen mejor en silencio pero **con murmullo se degradan igual**, así que **no se cambió el nombre**. Y Vosk cierra la frase por su propio detector de silencio (antes, con ruido constante, una frase tardaba hasta 20 s): latencia desde que termina la voz, **mediana 0.07 s en silencio** y **0.48 s con ruido blanco a +5 dB** (**MEDIDO**).

Vosk contra Whisper: lo medido y lo no medido

Dato	Vosk (modelo pequeño `vosk-model-small-es-0.42`)	Whisper
Carga del modelo	1.0–1.6 s	**No medido**

Dato	Vosk (modelo pequeño `vosk-model-small-es-0.42`)	Whisper
Memoria añadida	165 MB (silencio) a 288 MB (ruido)	**No medido**
Factor de tiempo real	0.06–0.28 (más de 3 veces más rápido que el audio)	**No medido**
Latencia al cerrar la frase (p95)	~25–30 ms (silencio), hasta ~200 ms (ruido)	**No medido**
CPU	6 % de un núcleo (silencio), hasta 22 % (ruido)	**No medido**

(Cifras medidas en la laptop; algunas tandas corrieron junto a otras pruebas, así que lo alto del rango puede estar inflado.) **Whisper no se midió porque instalarlo es decisión del equipo**; su ventaja de diseño es que escribe letra por letra y **no tiene límite de vocabulario**, pero con ruido o silencio puede **escribir el texto de orientación (prompt) como si lo hubiera oído**, por eso ese ajuste (`AETHER_VOICE_WHISPER_PROMPT`) es opcional y vacío por defecto.

Nada se ha medido con la voz ni el ruido reales del equipo (verificación V12 pendiente): **PENDIENTE (P-27)**. Esto incluye el porcentaje real de falsos positivos con habla humana, que las 25 frases sintéticas no estiman.

El ruido: por qué el VAD no basta

El ruido de un laboratorio (en la medición del equipo: voz máxima ~1000, picos de ruido ~3000) puede superar a la voz. Un **umbral de energía no puede separarlas**, por eso se confía el cierre de frase al detector acústico de Vosk. Lo que más ayuda es **físico**: un **micrófono de diadema** o muy cerca de la boca sube la voz respecto del ruido de golpe; y la tecla `w` o el control remoto del expositor (`/show/ws`, mensaje `wake`) son vías que no dependen de entender la voz.

Audio en el Jetson: por qué no sonaba

Dos cosas distintas: **voz de entrada** y **voz de salida**. La entrada no funciona en Firefox del Jetson porque `robot.html` usa `SpeechRecognition` del navegador (no activo en Firefox): por eso existe el servicio de voz de la laptop, que manda texto. La salida falló por la CSP (capítulo 11). Si el Jetson tendrá micrófono y bocina propios: **PENDIENTE (P-20)**.

Las variables que controlan la voz

Variable	Defecto	Qué cambia
`AETHER_VOICE_MODEL`	— (obligatoria con Vosk)	Carpeta del modelo en español
`AETHER_VOICE_ENGINE`	`vosk`	`whisper` exige instalar `faster-whisper`
`AETHER_VOICE_DEVICE`	micrófono por defecto	Qué micrófono usar
`AETHER_VOICE_START`	`on`	`off` arranca pausado, sin abrir el micrófono
`AETHER_VOICE_VAD_THRESHOLD`	`500`	Umbral de energía, se calibra con `scripts/probar_voz.py --mic 8`
`AETHER_VOICE_ECHO_TAIL_S`	`0.8`	Silencio tras hablar Lumina
`AETHER_VOICE_NOTIFY`	`1`	Notificación de Windows
`AETHER_VOICE_WAKE_FUZZY`	`1`	La regla tolerante de "Lumina"

Para calibrar el micrófono (4 s en silencio y 4 s hablando), `[Laptop · PowerShell]`, desde la raíz del proyecto:

```
backend\.venv\Scripts\python.exe scripts\probar_voz.py --mic 8
```

El script sugiere un valor para `AETHER_VOICE_VAD_THRESHOLD`. (Una corrección propia registrada: la versión anterior mostraba siempre "memoria del proceso: 0.0 MB" en Windows por un manejador truncado; se arregló y se probó.)

Si Lumina no escucha: diagnóstico en orden

1. **¿Corre el servidor?** `[Laptop · PowerShell]`: `.\iniciar_aether.ps1 -SoloComprobar`.
2. **¿Está el servicio de voz con "MICRÓFONO ACTIVO"?** En su ventana. Si dice pausado, `p`.
3. **¿Conecta con el servidor?** La consola dice `sin conexión con el servidor` si no; revisar `backend\logs\voice.log`.

4. **¿La página recibe?** El punto de voz de `robot.html` se pone "escuchando" cuando conecta.
5. **¿Qué escribió el reconocedor?** Con `AETHER_VOICE_LOG_TEXT=1` el texto queda en `backend/logs/voice.log` (el audio, nunca). Si lo escrito no se parece a "Lumina", es reconocimiento (ruido), no la regla.
6. **Eco:** si usa bocinas, probar audífonos o subir `AETHER_VOICE_ECHO_TAIL_S`.

Asistentes de voz en el mundo real

Los asistentes comerciales usan una **palabra de activación** detectada por un modelo pequeño y siempre atento ("wake word"), un reconocedor de voz mucho mayor en la nube o en el dispositivo, y respuestas generadas por un modelo de lenguaje grande. Aether replica la arquitectura con piezas de laboratorio, y deja claro lo que no tiene: un detector de activación dedicado, medidas con voces reales y un micrófono con cancelación de ruido.

Retos de voz para repasar

1. ¿Por qué un reconocedor no puede escribir una palabra que no está en su vocabulario?
2. ¿Qué hace `wake.py` y qué riesgo introduce?
3. ¿Por qué el audio nunca se guarda y cómo lo comprueban las pruebas?
4. ¿Qué tres capas de respuesta tiene Lumina y en qué orden se consultan?
5. ¿Qué medidas de Vosk existen y cuáles de Whisper?
6. Si Lumina no despierta con ruido de fondo, ¿qué dos salidas existen que no dependen de entender la voz?

Capítulo 16: Sistemas distribuidos

Qué cambia cuando un programa vive repartido en varias máquinas que pueden fallar por separado: entregar al menos una vez, no contar dos veces, juntar mensajes en lotes y decir siempre qué tan seguro es cada dato.

Cuando cada máquina puede fallar sola

Un programa en una sola computadora falla entero o no falla. Aether, en cambio, reparte el trabajo entre tres máquinas con **tres formas distintas de fallar**:

Máquina	Qué puede pasar	Qué ocurre con lo demás
Laptop (servidor)	Se duerme, se reinicia, se apaga el WiFi	Los agentes siguen trabajando y **acumulan** lo que no pueden entregar
Jetson (agente de percepción)	Se cae la cámara, se cuelga YOLO	El video se pierde, la laptop sigue sirviendo el resto
Raspberry Pi 4 (celda)	Pierde la red, la cámara se desconecta	**La celda sigue decidiendo y actuando por su cuenta**

Esa independencia es una decisión deliberada (ADR 0012): "*la celda decide y actúa aunque la laptop o la red se caigan.*" A cambio, aparecen problemas nuevos que este capítulo explica. En la jerga: un **sistema distribuido** es un conjunto de computadoras que cooperan por red y deben seguir funcionando cuando alguna **falla de forma parcial**.

Idempotencia: repetir sin dañar

Supóngase que el agente del Jetson manda "vi una `computadora2`" y la laptop **la guarda pero la respuesta se pierde**. El agente no sabe si llegó, así que reintenta. Si la laptop la contara dos veces, el inventario estaría mal. La solución es la **idempotencia**: **una operación que, repetida, produce el mismo efecto que una sola vez**.

En Aether se logra así (**VERIFICADO** en `backend/observation_intake.py` y `observation_outbox.py`, con pruebas):

1. Cada observación lleva un `event_id`, un identificador único y **estable** (el reintento lleva el mismo).
2. La laptop guarda los últimos **20 000** `event_id` vistos (`SeenIds`) y, si recibe uno repetido, lo **descarta y lo cuenta** como `duplicates`.

3. La respuesta dice cuántas aceptó y cuántas eran duplicadas: `{"accepted": 3, "duplicates": 1}`.

Un **límite conocido, declarado**: esa memoria vive en la laptop; **si se reinicia**, un reintento muy antiguo podría contarse otra vez (**persistir los `event_id` en disco** queda como mejora).

El buzón de salida

El agente no manda cada observación en el momento: la mete en un **buzón de salida** (`EventOutbox`), una cola local, y un hilo aparte la entrega. Reglas de la cola (**VERIFICADO** en código y pruebas):

- **Tamaño acotado:** hasta **1000** eventos (`AETHER_OUTBOX_MAX`); si se llena, **se descartan los más viejos** y se cuentan (`dropped_total`). Preferible perder lo antiguo a agotar la memoria.
- **Entrega al menos una vez:** una respuesta 2xx saca el lote de la cola.
- **Reintento con espera creciente** (*backoff exponencial*, de 1 s hasta 30 s) ante errores de red, 5xx, 408 o 429.
- **Descarte de lo inválido:** una respuesta 4xx (salvo 408 y 429) significa que el lote está mal formado; se descarta y se cuenta, porque reintentarlo para siempre **bloquearía la cola**.
- **El video no espera a nadie:** el detector corre aparte; si la laptop no contesta, la cámara y el video siguen.

Lotes: menos mensajes, mismo dato

El 23 de octubre se detectó un desperdicio. Con detecciones continuas (alrededor de **12 por segundo**), **cada observación despertaba al hilo y salía en su propio POST**. El commit `e8c0710` cambió la política: ahora se **juntan las observaciones del intervalo** (`AETHER_OUTBOX_INTERVAL_S`, **1 s** por defecto; `0` equivale al comportamiento anterior, inmediato) en **un solo envío**, con un tope de **50 por lote** (`AETHER_OUTBOX_BATCH`) y 3 s de espera máxima (`AETHER_OUTBOX_TIMEOUT_S`); un lote lleno sale sin esperar. El retraso máximo de entrega es ese intervalo. **No cambia qué se manda ni el inventario.** No se midió todavía el efecto en el Jetson real.

Quién manda: el modelo cliente-servidor

La **laptop es el servidor**: tiene la base de datos, sirve las páginas, aloja a Lumina y **acepta datos**. Los agentes son **clientes** que le **empujan** observaciones. Un detalle de diseño: la laptop **solo recibe y anota** los eventos de la celda; **nunca les manda órdenes de actuación** (el contrato lo dice: "*Solo se anotan: no activa nada ni cambia GPIO*"). Así, un fallo o un ataque en la laptop no puede mover el pistón.

Seguridad en la red local

Los endpoints de los agentes **no llevan autenticación** (contraseña o llave): están protegidos solo por ser **red local** (`AETHER_LAN_CIDRS`) y porque el guardia del dominio público los cierra. La ADR 0012 lo trata como **riesgo aceptado y documentado** (un equipo de la subred podría inyectar observaciones o texto falsos). Las defensas que sí existen (**VERIFICADAS** por pruebas): solo acepta clientes de la red local (403 fuera), valida el formato y rechaza campos extra (422), limita el tamaño (413), exige que el origen de un WebSocket de navegador coincida con el servidor, y limita a 8 las conexiones de voz. Si el proyecto saliera del laboratorio, la **autenticación de los agentes** sería la primera mejora.

Cuatro niveles de diagnóstico

Para saber *qué* falla en un sistema repartido, conviene preguntar de lo general a lo particular. El equipo lo formalizó para los agentes:

1. **¿El servidor está vivo?** `GET /health` → `{"status":"ok"}`.
2. **¿El agente está vivo?** `GET /health` del agente (`localhost:8001`) → incluye `"error"` con el motivo si falla.
3. **¿Qué está haciendo?** `GET /status`: cámara (FPS), inferencia (ms), `laptop.reachable`, `outbox_len`, `sent_total`, `dropped_total`, `last_error`, y recursos (memoria y CPU del proceso).
4. **¿Qué mide una prueba controlada?** `scripts/probar_agente.py` (15 s de medición).

El `/status` es riguroso a propósito: `fps_camera` (lo que entrega el hardware), `fps_delivery` (lo que sale anotado) y los milisegundos de inferencia son **independientes**; así se ve si el cuello de botella es la cámara (poca luz) o el procesamiento. Y el contrato añade una regla de honestidad: "*Ninguna cifra se afirma sin haberla leído de este endpoint en el equipo real.*"

Estados honestos: lo que se vio, lo que se supuso y lo que se midió

Los documentos del proyecto separan tres tipos de afirmaciones, el mismo método del libro:

- **Comprobado en pruebas con dobles** (software correcto sobre datos falsos).
- **Reportado** (Mario lo corrió en el equipo real y lo contó).
- **No verificado** (diseño o suposición).

Un ejemplo literal: la prueba de humo del agente en laptop midió **30.0 FPS** pero con **0 observaciones**, así que el envío de datos se verificó con dobles y se declaró **no verificado con detecciones reales**.

Cuándo esto se vuelve crítico

Todo lo anterior es "criterioso" en un prototipo; en una planta industrial se vuelve **obligatorio**: mensajes confirmados (*acknowledgements*), colas persistentes en disco (las empresas usan brokers como MQTT, RabbitMQ o Kafka), relojes sincronizados, supervisión central y *health checks* automáticos. Aether usa los mismos conceptos con piezas simples; el uso de **MQTT** para telemetría está decidido (Decisión 6) pero **sin implementar** (capítulo 18).

Preguntas de repaso: sistemas repartidos

1. ¿Qué es la idempotencia y qué campo la hace posible en las observaciones?
2. ¿Por qué el buzón de salida descarta los eventos **más viejos** cuando se llena?
3. ¿Qué hace el agente ante una respuesta 5xx y qué ante una 4xx, y por qué?
4. ¿Qué problema resolvió el cambio a lotes del 23 de octubre y qué cifra lo disparó?
5. ¿Por qué la laptop nunca manda órdenes de actuación a la celda?
6. Explicar la diferencia entre `fps_camera` y `fps_delivery`.

Capítulo 17: Del recuadro al inventario

Cómo miles de detecciones sueltas se convierten en una tabla de inventario con historial, por qué una base de datos y no un archivo, y qué promete (y qué no) el estado de cobertura.

Setecientas doce detecciones, diecinueve filas

En una corrida real de prueba registrada en el informe integrador de septiembre, **712 observaciones** crudas se agruparon en **19 entradas** de inventario. La cifra ilustra el trabajo de este capítulo: la cámara y YOLO producen una avalancha de registros ("vi una caja a las 15:04:05", "vi otra a las 15:04:05.1", …) y alguien debe convertirla en algo que un humano pueda leer: "hay 19 cosas distintas, de estas clases, vistas tantas veces". Esa conversión es el **motor de inventario** (*Inventory Engine*).

Detectar no es contar

Una detección dice "en este cuadro hay una caja". Contar exige saber **si es la misma caja** que en el cuadro anterior. Aether lo resuelve a su manera: **no pretende contar objetos físicos, sino agrupar observaciones por identidad cuando existe**. La regla de agrupación (`edge/inventory_engine/aggregate_observations.py`, función pura `aggregate()`):

- Si la observación **tiene un código de barras/QR legible**, la clave de grupo es **ese código exacto**.
- Si **no**, la clave es `unidentified:` seguido de la **clase detectada**.

Por cada grupo se calcula la clase más frecuente, el total de observaciones, cuántas tenían código, la **confianza promedio**, la primera y la última vez vista y el conteo por **ubicación** (el identificador del ArUco visible, o `sin_ubicacion`). Seis pruebas automáticas verifican esta función. La función es **pura**: no lee ni escribe archivos, así que se prueba sin cámara.

Y una advertencia de terminología, obligatoria en el proyecto: **"conteo visible detectado" no es "conteo real de inventario"**. Si el robot no pasó por un estante, el sistema no sabe si ahí hay productos.

Dos niveles de datos: Observations y Declared Inventory

La Decisión 5 separa dos niveles:

Nivel	Qué es	Dónde vive
Observation	El registro crudo de una detección individual	`backend/agent_contracts.py` (contrato), cola en memoria y archivos JSON
Declared Inventory (inventario declarado)	El resumen agrupado: lo que el sistema "declara" saber	Tabla `declared_entries` de la base de datos

Más una tercera tabla, `declared_entry_history`, que es la **bitácora de inspecciones**: cada corrida de carga **agrega** una fila por entrada y **nunca actualiza ni borra** las anteriores (así un cliente puede auditar cómo se llegó a un número). La opción B del alcance del panel (ADR 0007) se justifica así: "*lo que convence a un cliente escéptico no es ver al robot moverse en vivo, sino poder auditar cómo se llegó a un número*".

La ruta del dato, paso a paso

La **tubería manual** original son tres pasos, que se corrían a mano y en orden:

1. **Captura** (`edge/perception/test_combined.py`): abre la cámara, detecta, y al terminar vuelca las observaciones a `observations_output.json`.
2. **Agregación** (`edge/inventory_engine/aggregate_observations.py`): produce `declared_inventory_output.json`.
3. **Carga** (`backend/scripts/load_from_edge.py`): hace un **upsert** (actualiza si existe, crea si no) en `declared_entries` e inserta filas nuevas en `declared_entry_history`.

Desde el 2 de septiembre existe una versión automática (**ADR 0009**, Decisión 9): un **bucle de auto-refresh** que, cada N segundos (15 a 30, configurable), toma las observaciones acumuladas, las agrega y las carga, sin intervención humana. Se activa con `POST /inventory/auto-refresh/start` y se detiene con `/stop`; `GET /inventory/auto-refresh/status` dice si está activo, la hora y cuántas observaciones procesó la última corrida y el último error. Si en una ventana no hubo detecciones, **se omite el ciclo** (no inventa actividad). El panel refleja los cambios con **polling** simple (consultas repetidas), sin WebSockets.

El auto-refresh es una **versión reducida** del monitoreo en vivo y el panel lo dice con honestidad: es la vista de *esta cámara fija*, no "inventario en tiempo real del piso". Con los agentes, cada observación del Jetson llega a la laptop por `POST /perception/observations` (capítulo 16) y alimenta los mismos oyentes.

Qué es una base de datos y por qué no basta un archivo

Un archivo de texto sirve para un dato; no para consultar, actualizar y conservar historial **sin que dos procesos se pisen**. Una **base de datos** ofrece **tablas** (filas y columnas), **consultas** (SQL), **transacciones** (un conjunto de cambios que ocurre completo o no ocurre) y **claves** (una columna que identifica cada fila de forma única). Aether usa **SQLite** (una base de datos en **un solo archivo**, `backend/aether.db`, que viene con Python y no necesita servidor) mediante **SQLAlchemy**, una librería que permite describir las tablas como clases de Python (`DeclaredEntry`, `DeclaredEntryHistory`). La decisión de pasar a **PostgreSQL** está marcada como "candidato a escalar", y por el principio de no fabricar dependencias **no se instaló su driver**. El archivo `aether.db` **no se sube a Git**.

Retirar no es borrar

Una entrada de inventario solo puede dejar de contar como activa por **una decisión humana explícita**: `PATCH /inventory/{identifier}/retire`, que cambia su `status` de `active` a `retired` (nunca borra la fila; sigue consultable con `?include_retired=true`). Nunca ocurre sola por no aparecer en una corrida: **no hay forma de saber hoy si esa corrida cubrió todo el piso**, y retirar automáticamente sería fabricar una garantía que el sistema no puede dar.

La acción exige una contraseña (`ADMIN_ACTION_PASSWORD`, configurada por entorno; el repositorio tiene un valor por defecto solo para desarrollo) y el código mismo aclara que **no es autenticación real**: no hay usuarios, sesiones ni roles; es una "traba contra clics accidentales". Un sistema de permisos real queda como decisión futura explícita.

Confianza del dato: qué se le puede prometer al usuario

El esquema define **tres estados formales de cobertura** de una inspección (`edge/inventory_engine/coverage_states.py`): `PARTIAL` (parcial), `COMPLETE` (completa) e `INVALID` (inválida), con la regla de que **una inspección parcial nunca sobrescribe silenciosamente el inventario declarado previo**. **Pero los estados no se asignan todavía**: dependen del módulo de **navegación**, que decide qué recorrido hizo el robot, y ese módulo no existe. El código lo declara con claridad: "*no se fabrica aquí para no simular una garantía que el sistema aún no puede respaldar*". Y aun cuando se asignen, `COMPLETE` **significa que el procedimiento terminó, no que se vio todo el inventario real**. Los

criterios de aceptación del MVP incluyen probar estos casos (completa, interrumpida e inválida); aún **no se han ejecutado** (**PENDIENTE: P-22**, junto con los umbrales de aceptación).

Lumina y el inventario

El asistente conoce el inventario solo de forma limitada: su mensaje de sistema le dice que la percepción y el motor de inventario "ya funcionan" y que la línea de manufactura todavía no existe físicamente, y que **nunca debe inventar cifras**. Las preguntas típicas de una presentación tienen respuestas fijas precargadas (capítulo 15).

Escala: cuántas observaciones son demasiadas

Una cámara a 10 detecciones por segundo produce 36 000 registros por hora; en una semana, millones. SQLite aguanta cifras así en un prototipo, pero una base real necesitaría **índices**, **limpieza periódica** de observaciones crudas ya agregadas y, a mayor escala, un servidor de base de datos. Esto **no se ha medido** en el proyecto; la cola del agente está acotada a 1 000 eventos y el panel de detecciones recientes, a 50, para evitar el crecimiento sin límite.

Sistemas de inventario industriales

En una bodega real, el inventario se gestiona con un **WMS** (*Warehouse Management System*) conectado a **códigos de barras, RFID** y lectores en puertas, y los robots de inventario (por ejemplo, robots que escanean estantes de tiendas) entregan **reportes con tiempo y confianza**, no "verdades". El diseño de Aether —observaciones crudas, inventario declarado con confianza y bitácora auditable— sigue ese mismo principio.

Preguntas de repaso: inventario y datos

1. ¿Cómo agrupa `aggregate()` las observaciones y qué clave usa cuando no hay código?
2. ¿Qué diferencia hay entre `declared_entries` y `declared_entry_history`?
3. ¿Por qué retirar una entrada no la borra, y por qué nunca ocurre automáticamente?
4. ¿Qué dicen los tres estados de cobertura y por qué todavía no se asignan?
5. ¿Qué es un upsert y dónde se usa?
6. ¿Por qué `COMPLETE` no significa "se vio todo el inventario real"?

Capítulo 18: Todos los idiomas de Aether

REST, WebSocket, MQTT, Modbus, GPIO, RS485, I^2C y USB serie: qué se habla entre cada par de piezas, cuáles ya existen y cuáles son todavía plan.

Doce piezas, ocho idiomas

Una de las confusiones más comunes al conocer Aether es creer que "todo se comunica por MQTT" o "todo por la web". En realidad hay **ocho idiomas distintos**, cada uno elegido para un tramo, y algunos de ellos solo existen en el papel. Este capítulo los ordena. Antes, una definición útil: un **protocolo** es el acuerdo de cómo se manda y se interpreta un mensaje entre dos programas o dispositivos.

El cuadro completo

Tramo	Protocolo	Cuerpo del mensaje	Estado
Navegador ↔ laptop	**HTTP / REST** (JSON)	Peticiones del panel y de Lumina	**VERIFICADO**, corre
Página ↔ laptop (voz, eventos del show)	**WebSocket** (JSON)	Texto reconocido, estados, eventos	**VERIFICADO** en pruebas
Jetson → laptop	**HTTP POST** de observaciones (JSON, con `event_id`)	Lotes de hasta 100 observaciones	Probado con dobles; Jetson real: el agente funcionó, la entrega reportó `laptop.reachable` verdadero
Pi 4 → laptop	**HTTP POST** de eventos de pieza	Eventos con `event_id`	Probado con dobles
Jetson ↔ ESP32	**USB serie** (texto con latido)	Órdenes de movimiento y lecturas	**Diseño**; contrato sin escribir (capítulo 5)
ESP32 ↔ ruedas M0601	**RS485** (half-duplex)	Mensajes del fabricante	**Diseño**; protocolo del motor pendiente
ESP32 ↔ sensores	**I^2C** y señales digitales	Distancias, giro, finales de carrera	**Diseño**

Tramo	Protocolo	Cuerpo del mensaje	Estado
Pi 4 → relevadores → LOGO / pistón	**GPIO** (nivel eléctrico)	Pulsos de 0.5 a 1 s	Código escrito; **sin `gpiod` instalado, no se mueve**
Robot → laptop (telemetría)	**MQTT**	Batería, estado, posición	**Decidido, sin implementar**
Celda ↔ PLC	**Modbus TCP**	Registros de un PLC	**Plan viejo**, no es lo que corre

HTTP/REST: el idioma del panel

REST es un estilo de uso de HTTP donde cada **recurso** (inventario, celda, historial) tiene una dirección y los verbos (`GET`, `POST`, `PATCH`, `DELETE`) expresan la acción. Aether usa REST para todo lo que el panel muestra o pide (capítulo 11). Está en `contracts/rest_api.md`. Su estado, escrito en ese documento: "*Opción B (Decisión 7 / ADR 0007) implementada*"; el estado en vivo del robot (Opción C) **sigue sin implementar**.

Los agentes por HTTP: datos chicos con un identificador

Los dos agentes (el del Jetson y el de la Pi) hablan con la laptop por **HTTP POST con JSON**, y la razón de no usar MQTT todavía es de **simplicidad**: ya existe un servidor HTTP, con validación (capítulo 13), idempotencia (capítulo 16) y pruebas. El contrato (`contracts/agents_api.md`) está **versionado**: todo mensaje lleva `"v": 1`.

WebSocket: cuando el servidor también habla

Dos canales en vivo (capítulo 11): `/voice/ws` (el servicio de voz manda texto, las páginas mandan "estoy hablando") y `/show/ws` (relevo de eventos del show de presentación). Ninguno lleva audio.

MQTT: decidido, todavía sin escribir

MQTT (*Message Queuing Telemetry Transport*) es un protocolo ligero de **publicar y suscribirse**: un programa **publica** un mensaje en un **tópico** (un nombre como `aether/robot/bateria`) y todos los que se **suscribieron** a él lo reciben, a través de un

servidor intermedio llamado **broker**. Se usa mucho en dispositivos industriales y robots porque es pequeño y tolera redes lentas.

La **Decisión 6** (ADR 0006) establece MQTT para la **telemetría** del robot, pero **hoy no hay ni una línea de código MQTT**: `edge/comms/mqtt_client/__init__.py` está **vacío** (**VERIFICADO**) y `contracts/mqtt_topics.md` dice textualmente "*pendiente de definición*": no se fabrican tópicos antes de que exista la interfaz real. Dos usos de MQTT están decididos para el futuro: los eventos de handshake de la línea (para el escucha `line_events_listener`, también vacío) y la telemetría del robot (batería, posición, tarea en curso), que sería la **Opción C** del panel.

Un principio de seguridad atraviesa esta decisión: **los comandos de parada de seguridad no dependen de la red**; se resuelven con un watchdog de hardware y un paro físico, **nunca** con un tópico del que dependa la seguridad (capítulo 4).

El protocolo de handshake de la línea: el contrato que quedó a medias

`contracts/line_handshake_protocol.md` describe, tras el cambio a banda y pistón (ADR 0010), un diálogo mínimo de **dos eventos**:

- **Evento A** (visión → actuador): `resultado` (`PASS` o `FAIL`), hora, `tipo_objeto` (1, 2 o 3 o nulo) y confianza.
- **Evento B** (actuador → escucha): lo que **realmente ocurrió**: `scrap_expulsado`, `pieza_aceptada` o `fallo_actuacion` (nunca reportar aceptada cuando la visión dijo FAIL y el pistón no actuó), con referencia al evento A.

El documento dice que el **medio** de enviar el Evento A (Modbus, E/S digital o serial) quedaba pendiente de investigación técnica. **Hoy** el equipo resolvió esa duda por **GPIO + relevadores** (capítulo 7) y los eventos que el código manda a la laptop son **HTTP POST** de la Pi (`pieza`, `juicio`, `accion`); el MQTT del Evento B **no existe**. En otras palabras, el contrato de papel y la implementación real **no coinciden aún**, y esa diferencia es la misma que se anotó en las correcciones (el contrato menciona "tipo_objeto" como inexistente cuando el código ya lo calcula).

Modbus TCP: el plan con PLC

Si la celda se reconstruyera alrededor de un **PLC**, la comunicación sería por **Modbus TCP**: el PLC expone **registros** (como casillas numeradas) que otro equipo lee y escribe por Ethernet. La hoja de conexión del 5 de octubre definió un mapa de señales (`rechazar`, `banda_marcha`, latidos `rpi_vivo` y `plc_vivo`, `accion`, `pieza_id`) con direcciones de ejemplo y una columna "Real" que nunca se llenó. Es documentación vieja; la carpeta `plc_backend/` está vacía a propósito. Se conserva porque el **diseño de latidos cruzados** (cada lado vigila que el otro siga vivo, y si deja de verlo, pasa a un estado seguro) es una buena práctica general que Aether aplica en otros tramos con el *watchdog* del ESP32.

GPIO, RS485, I²C y USB serie

Los cuatro idiomas del cuerpo físico están explicados en capítulos anteriores: **GPIO** (capítulo 8), **RS485** (5), **I²C** (6) y **USB serie** (5). Un vistazo comparativo de sus rasgos:

Idioma	Cables	Distancia típica	Velocidad (orden de magnitud, regla general)	Dónde
GPIO	1 por señal	Centímetros	Un cambio de nivel	Pi → relevadores
I²C	2 (SDA, SCL) para muchos dispositivos	Decenas de centímetros	Cientos de kbit/s	ESP32 ↔ sensores
USB serie	1 cable USB	Pocos metros	Cientos de kbit/s a Mbit/s	Jetson ↔ ESP32
RS485	2 trenzados	Cientos de metros	Hasta Mbit/s	ESP32 ↔ M0601
Ethernet / WiFi	Red	Edificio	De 10 Mbit/s a Gbit/s	Entre equipos

Qué dice la industria

En automatización industrial conviven **buses de campo** (Modbus, PROFIBUS, CAN, EtherCAT) para sensores y actuadores, y **protocolos de planta** (OPC UA, MQTT) para subir datos a sistemas de supervisión. La arquitectura típica separa **control en tiempo real** (cerca de la máquina) de **supervisión** (en red). Aether imita esa separación: GPIO, RS485 e I²C cerca del hardware; HTTP y WebSocket en la capa de supervisión; MQTT reservado para la telemetría.

Preguntas de repaso: protocolos

1. ¿Qué diferencia hay entre una petición REST y un mensaje MQTT?
2. ¿Por qué los agentes mandan sus datos por HTTP POST y no por MQTT, hoy?
3. ¿Qué dos mensajes define el contrato de handshake de la línea y por qué difiere del código actual?
4. ¿Qué cuatro protocolos se usan "cerca del hardware" y cuáles de ellos son diseño todavía?
5. ¿Qué es un latido cruzado y qué función cumple en Modbus y en el ESP32?
6. ¿Por qué la parada de seguridad no puede depender de MQTT?

Capítulo 19: El panel de Aether

La cara web del proyecto: sus páginas, su sistema de diseño, su cargador, su pantalla de la celda y las reglas de honestidad que se aplican hasta en los colores.

Un panel que prefiere decir "pendiente" a mentir

Entre los detalles más finos del panel de Aether está un color. El sistema de diseño define un **ámbar** (`#FFC14D`) cuyo uso se restringe, por escrito, a cuatro casos: "*sin identificar, pendiente de hardware, retirar producto, errores de formulario. Nunca decorativo.*" Y las tarjetas de módulos **pendientes de hardware** nunca llevan las esquinas de mira que identifican lo activo: llevan un **borde punteado ámbar**. Con eso, un visitante distingue **de un vistazo** lo que funciona de lo que todavía es plan, sin leer una palabra. Es el principio de honestidad de producto llevado al diseño.

Las páginas

El panel (`dashboard/`) es un conjunto de páginas HTML servidas por el mismo servidor (capítulo 11). Qué hace cada una:

Página	Qué muestra	Notas
Portada (`index.html`)	Titular, simulación "De la banda a la mesa", la vista **"Pieza por pieza"** del robot, percepción en vivo, **módulos del sistema** (con su estado), línea de manufactura, arquitectura y alianzas	Se abre con un **cargador** y un campo animado de nodos 3D
Inventario (`inventory.html`)	La tabla del inventario declarado, el historial de inspecciones, el control de auto-refresh y el diálogo de retirar	Muestra el estado de cobertura **con su explicación visible**
Celda (`celda.html`)	Video de la cámara de línea, estado, contadores, últimas piezas, alarmas y Arranque/Paro	La sirve la Pi en la celda real
Lumina (`robot.html`)	La cara, subtítulos, cámara y llamadas	Sin cargador a propósito: arranca negra hasta la voz
Dataset (`dataset.html`)	Subir fotos, dibujar cajas, ver conteos y el modelo activo	Alimenta el entrenamiento (capítulo 14)

Página	Qué muestra	Notas
Armado (`armado.html`)	La hoja de armado 3D del carrito	Capítulo 3
Holo (`holo.html`)	Vista holográfica del carrito	Elemento de presentación

La portada además documenta, en una sección del propio sitio, algo que el equipo decidió no ocultar: el **diseño conceptual** del robot ("Pieza por pieza") está rotulado en la página **como diseño beta, no como el modelo mecánico**, y cada una de sus cinco capas (percepción, cómputo, control, energía, tracción) lleva **el estado real de su decisión**.

El sistema de diseño

Un **sistema de diseño** es un conjunto de reglas que se repiten en todas las páginas para que se vean coherentes. El de Aether (`.interface-design/system.md`) tiene un concepto: **"industrial systems"**, casi negro absoluto con un solo acento. Sus piezas:

Elemento	Valor	Para qué
Fondo	`#03060F` (casi negro)	Base de todo
Acento interactivo	`#2EE6FF` (cian)	Todo lo clicable
Positivo	`#3DFFC2` (verde)	"Funcionando"; nunca decorativo
Atención	`#FFC14D` (ámbar)	Solo los cuatro casos de arriba
Radios	4 px y 8 px	Botones y paneles
Espaciado	4/8/12/16/24/32/48 px	Una escala fija

Tres tipografías con roles fijos: *Big Shoulders Display* solo para títulos grandes, *IBM Plex Sans* para el cuerpo y *IBM Plex Mono* **solo** para anotación técnica (cifras, etiquetas, tags de tabla). La "firma visual" son las **esquinas de mira** (*reticle*): las cuatro esquinas que YOLO dibuja alrededor de una detección, reutilizadas para marcar lo activo.

El cargador y las animaciones (30 de septiembre)

Según `docs/CHANGELOG.md`, el 30 de septiembre se agregó un **cargador global** en la portada y el inventario: un anillo con el **progreso real** de la carga (no un reloj de adorno), un robot que se desarma y se arma en bucle y un logo que se dibuja, con un mínimo de

800 ms y un máximo de 3 s; lo que siga cargando después se ve como **skeleton** (un esqueleto gris que ocupa el lugar del contenido). También se añadieron entradas animadas al hacer scroll, navegación translúcida, cifras que ruedan hasta su valor real y una bitácora animada, todo respetando la preferencia del sistema operativo `prefers-reduced-motion` (que pide menos animación a quien la configuró). Para que el anillo refleje datos reales, toda carga inicial se envuelve en `AetherLoader.track(promesa)`.

Un error corregido ese día es instructivo: `line-vision.js` **redeclaraba constantes globales** de `camera-stream.js`, lo que impedía que corriera, y el botón "Iniciar visión de línea" **no hacía nada**. La solución fue aislar el código de cada archivo en su propio ámbito (una función que se ejecuta de inmediato). Con muchos archivos JavaScript en una misma página, ese es el tipo de choque que ocurre.

La pantalla de la celda

`celda.js` cumple las normas de interfaz industrial en pequeño. Su comentario inicial lo dice: **ISA-101** (color solo ante condición anormal; si no hay controlador, los conteos se pintan como no válidos; Arranque y Paro piden confirmación), **ISA-18.2** (las alarmas vienen del servidor y sobreviven a recargar la página) e **IEC 62443** (solo esta pantalla: el mando exige sesión; "Ingeniería" es quien puede borrar el histórico). Pide `GET /line/cell/status` **cada segundo** y traduce las acciones a frases humanas:

Acción técnica	Texto que ve el operador
`pieza_aceptada`	"Sigue en la banda"
`scrap_expulsado`	"Expulsada por el pistón"
`fallo_actuacion`	"El pistón no disparó"
`pulso_no_enviado`	"El pulso no salió"

Un detalle de honestidad: la hoja de conexión señalaba que la marca `SIM` de la pantalla estaba escrita fija en el HTML y no se apagaba sola cuando llegaran datos reales; por eso debía quitarse en el mismo cambio en que el origen dejara de ser simulado. El alcance de esta pantalla, que diseña **Emmanuel** (HMI/GUI), sigue **PENDIENTE (P-17)**.

Lumina en pantalla: la cara con expresiones

`robot.js` (unas 2 600 líneas, el archivo más grande del panel) implementa la cara de Lumina, un sistema de **expresiones**, los subtítulos, el reconocimiento de voz del navegador (cuando existe), los modos Conversación y Cámara, el guion de la presentación, las llamadas y el oyente del servicio de voz. Su rediseño del 17 de septiembre (estilo OLED de ciencia ficción) incluyó un error corregido el mismo día: una constante declarada dos veces que impedía que corriera.

Ejercicio de lectura de un panel

Para entender cualquier página nueva del panel, el recorrido recomendado es: (1) abrir la página en el navegador; (2) pulsar `F12` y mirar la pestaña **Red** para ver qué direcciones pide; (3) buscar esa dirección en `backend/main.py`; (4) abrir el `.js` de la página y buscar la función que pinta ese dato. Con ese método se entiende en minutos qué parte del servidor alimenta cada elemento.

Cómo se hacen paneles de planta de verdad

Los paneles de supervisión reales siguen normas como **ISA-101** (cómo diseñar pantallas de operador) y **ISA-18.2** (cómo administrar alarmas) para que un operador reaccione bien a las 3 de la mañana: poco color, jerarquía clara y alarmas priorizadas. Aether adopta la idea de **color solo ante condición anormal** en la celda y añade una pantalla pública más "comercial" en la portada, donde la estética pesa más.

Preguntas de repaso: el panel

1. ¿Para qué casos se permite el color ámbar y por qué?
2. ¿Qué diferencia visual hay entre una tarjeta de módulo activo y una pendiente de hardware?
3. ¿Qué problema causó el bug de `line-vision.js` y cómo se evitó?
4. ¿Por qué `robot.html` no lleva cargador?
5. ¿Qué tres normas inspiran la pantalla de la celda y qué aporta cada una?
6. ¿Qué cuatro pasos se siguen para entender una página nueva?

Capítulo 20: El sitio público, el dominio y Cloudflare

Cómo se publica Aether en internet sin abrirle la puerta del laboratorio: un sitio estático, un dominio, un escudo intermedio y una lista de pasos manuales que sigue pendiente.

Un sitio que responde 404 a propósito

Al revisar el dominio público con `curl`, las rutas `/.env`, `/.git/config`, `/perception/stream`, `/line/cell/run`, `/robot/ask` y `/inventory` respondieron todas **404 de GitHub Pages**. No es una falla: **es la prueba de que el backend no está en ese dominio**. El sitio público de Aether es solo **HTML estático**; el servidor, las cámaras y la celda se quedan en la red del laboratorio. Este capítulo cuenta cómo se logra eso y qué falta para endurecerlo.

Qué es el sitio público y dónde vive

Un **sitio estático** es un conjunto de archivos (HTML, CSS, JavaScript, imágenes) que se entregan tal cual, sin programa que "piense" en el servidor. Aether publica el suyo en **GitHub Pages** (el servicio gratuito de GitHub para sitios estáticos), desde la rama `gh-pages`, bajo el dominio `aethermx.online`. Un **dominio** es el nombre legible de la dirección; su configuración (los **registros DNS**) dice a qué IP apunta. Según el runbook de seguridad del 16 de octubre de 2026:

Concepto	Valor
Sitio	HTML estático en GitHub Pages, rama `gh-pages`
Registros A del dominio raíz	Cuatro, hacia las direcciones públicas de GitHub Pages
`www`	Un CNAME (alias) hacia la cuenta de GitHub Pages
Servidores de nombres hoy	Los de Hostinger (el registrador)
Correo	**ImprovMX**, un servicio de reenvío de correo del dominio
Contacto de seguridad	Publicado en `/.well-known/security.txt` (un estándar para decir a quién reportar vulnerabilidades)

El `robots.txt` pide a los buscadores **no indexar** las pantallas del laboratorio (`celda.html`, `dataset.html`, `inventory.html`, `robot.html`, `holo.html`): "en el dominio público no están publicadas".

Lo que se hizo en el repositorio (16 y 17 de octubre)

La rama `feature/hardening-cloudflare` y los commits del 16 de octubre hicieron lo siguiente, **VERIFICADO** en el historial:

- Se **quitó la cámara en vivo** del dominio público y la solicitud de demo quedó como un **correo** (el botón abre Gmail, en lugar de un enlace `mailto`).
- Se **cerraron cámara, pistón y banda** a las peticiones que no vengan del laboratorio (el guardia `public_guard.py`, capítulo 11).
- Se **alineó la política de seguridad del servidor** `nginx` (`deploy/nginx-aether.conf`, un servidor web alternativo no desplegado) con el encabezado que irá en Cloudflare.
- Se limpió la `<meta>` de la política de contenido: se quitó `frame-ancestors` (el navegador lo **ignora** dentro de una `<meta>`) y `connect-src` hacia Google Fonts (las fuentes entran por otras directivas).
- Ajustes del sitio: en iPhone no se carga el título animado, la línea de manufactura queda en tarjetas, y una **simulación de funcionamiento a 2x** debajo de la portada.
- Se agregó la sección **Alianzas** y se escribieron `docs/security/runbook-cloudflare.md` y `docs/security/laboratorio.md`.

Por qué la política del encabezado no es la de la `<meta>`

Un detalle fino del runbook: la `<meta>` de `index.html` solo aplica a la portada, pero **el encabezado HTTP aplica a todas las páginas**, incluida `armado.html`, que lleva un módulo 3D enorme **en línea** y un *import map* (un mapa de módulos de JavaScript que no puede ir en archivo externo) y carga Three.js desde jsDelivr. Para que el carrito 3D no se quede en blanco, el encabezado necesita `'unsafe-inline'` en `script-src` y `style-src` y permitir `https://cdn.jsdelivr.net`. **No** necesita `'unsafe-eval'`. Es un **riesgo declarado**: `'unsafe-inline'` es una directiva amplia, y una falla de inyección de scripts dentro de `armado.html` no la frenaría la política. La `<meta>` de la portada queda más estricta (`script-src 'self'`), y el navegador aplica **ambas** a la vez.

Cloudflare: un escudo entre internet y el sitio

Cloudflare es un servicio que se coloca **entre los visitantes y el sitio**: recibe cada petición, la filtra y la reenvía. Con su plan gratuito se puede añadir **cifrado HTTPS estricto**, **encabezados de seguridad**, **filtros de bots** y **límites de peticiones**. Para usarlo hay que **cambiar los servidores de nombres** del dominio (de los de Hostinger a los de Cloudflare). El runbook, escrito para el plan gratuito, ordena los pasos así:

1. **Copiar el DNS dentro de Cloudflare** sin cambiar nada todavía.
2. **Bajar el TTL** (el tiempo que los DNS recuerdan una respuesta) y **cambiar los servidores de nombres** en Hostinger.
3. **HTTPS:** modo **Full (strict)** (nunca Flexible, que provoca un bucle de redirecciones), "Always Use HTTPS", TLS mínimo 1.2 y **HSTS** a 6 meses con subdominios y **sin preload**.
4. **Encabezados HTTP**, primero en **modo reporte** (la consola avisa, no bloquea) y después en modo **forzado**: `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, `Cross-Origin-Opener-Policy: same-origin` y `X-Frame-Options: SAMEORIGIN`.
5. **Bot Fight Mode**, **Browser Integrity Check**, la regla de límite de peticiones (en el plan gratuito, **un solo conteo de 10 segundos**) y una regla de WAF para rutas sensibles.
6. **Correo:** SPF (ya existía), DKIM (copiado del panel de ImprovMX, "no de memoria") y **DMARC** en vigilancia.

HSTS (*HTTP Strict Transport Security*) le dice al navegador que **solo use HTTPS** con ese dominio durante el tiempo indicado; una vez enviado, **no se puede deshacer** en navegadores que ya lo guardaron, por eso se eligen seis meses y no se activa el *preload*.

Estado: manual y pendiente

El reporte del 16 de octubre es explícito: **no hay token de Cloudflare en el entorno y no se pidió por el chat.** Los pasos del panel **no están ejecutados**: **PENDIENTE (P-29)** (alta del dominio, cambio de servidores de nombres, SSL Full strict, HSTS, reglas de encabezados, Bot Fight, límite de peticiones, DMARC y DKIM). Hoy, `curl -I https://aethermx.online` responde `Server: GitHub.com` **sin** HSTS, CSP, `nosniff`, `Referrer-Policy` ni `Permissions-Policy` (**VERIFICADO** antes del cambio). Hasta que se

cambien los servidores de nombres, el dominio sigue sin esos encabezados. Las comprobaciones finales (securityheaders.com, SSL Labs, un `iframe` desde otro origen, mail-tester) **no pueden cerrarse hasta entonces**.

Riesgos que se quedan, con nombre

- El certificado de GitHub podría no renovarse con la "nube naranja" (el proxy de Cloudflare) activa: cada ~90 días hay que poner la nube en gris un rato.
- **Bot Fight Mode** puede mostrar un reto a un visitante legítimo; el runbook sugiere apagarlo si un inversionista ve la página de Cloudflare en vez de la portada.
- El límite de peticiones gratuito es una ráfaga de 10 segundos, no "60 por minuto"; un atacante paciente podría quedarse por debajo.
- SPF con `~all` marca falsificaciones como sospechosas, no las rechaza; el rechazo llega al pasar DMARC a `p=reject` con DKIM firmando.
- **Quien tenga el puerto 8000 reenviado en el router salta todo esto**: por eso existe la lista de `docs/security/laboratorio.md` (capítulo 21).

Cómo se hace en una empresa

Una empresa pondría un **CDN** (red de distribución de contenido) con WAF, certificados administrados y monitoreo, y separaría el sitio público de las herramientas internas con **acceso por identidad** (como un VPN o Cloudflare Access) en lugar de "ocultar" páginas con `robots.txt`. Aether adopta el mismo principio de **mínimo expuesto**: lo público es estático y lo operativo solo vive en la red local; si algún día la celda se mostrara en vivo, el documento del laboratorio propone **Cloudflare Tunnel con Access** o **Tailscale**, ambas sin abrir puertos del router.

Preguntas de repaso: lo público

1. ¿Por qué un 404 de GitHub Pages en `/inventory` es una buena señal?
2. ¿Qué diferencia hay entre una política de contenido en `<meta>` y en el encabezado?
3. ¿Por qué el encabezado necesita `'unsafe-inline'` y cuál es el riesgo?
4. ¿Qué es HSTS y por qué se evita el *preload*?
5. ¿Qué pasos del runbook siguen sin ejecutarse y por qué no se pueden cerrar las pruebas?
6. ¿Qué dos formas de mostrar la celda en vivo propone el equipo sin abrir puertos?

Capítulo 21: Seguridad

Un repositorio público, una red compartida y un robot con relevadores: qué amenazas se consideraron, qué capas existen, qué se admitió que no está resuelto y qué hábitos protegen los secretos.

Una contraseña que dejó de estar en el código

El 21 de octubre, al leer el código de la celda con ojos de revisor, se encontró una cuenta **fija** (`admin` / `admin`) escrita dentro de `cell_hmi.py`, con un comentario que la llamaba "cuenta fija de laboratorio". El repositorio es público y el servidor escucha en todas las interfaces: **cualquiera en la red podía iniciar sesión y usar Arranque, Paro y "Borrar histórico"**, y el estado de la celda no pedía sesión. El hallazgo (bitácora, sesión 5) se resolvió así: la contraseña **salió del código** y ahora viene de la variable de entorno `CELDA_HMI_CLAVE`; **sin esa variable nadie inicia sesión** y los botones no responden. Es el ejemplo perfecto de la regla número uno de este capítulo: **en un repositorio público, un secreto en el código es un secreto de todos.**

El modelo de amenazas, en un párrafo

El documento de la ADR 0012 lo resume sin adornos: el repositorio es público, la red de la universidad es compartida, el backend no tiene usuarios y el WebSocket del *show* no está autenticado. **Cualquier equipo de la subred puede intentar hablarle a los endpoints.** Las defensas ya existentes son: solo aceptar clientes de `AETHER_LAN_CIDRS`, validar cada campo con límites de tamaño, **no ejecutar nada de lo recibido**, exigir `Origin` en los WebSocket de navegador, cerrar `/config` y `/voice` al dominio público, abrir el firewall de Windows solo en perfil Privado y no guardar secretos ni audio. **Y un riesgo aceptado, dicho tal cual:** un atacante dentro de la subred **podría inyectar observaciones falsas o texto a Lumina** porque no hay autenticación; se acepta como riesgo de laboratorio y "*un jurado debe oírlo así*".

Siete capas, de la más externa a la más interna

#	Capa	Dónde está	Estado
1	**Dominio público estático**: no hay backend en `aethermx.online`	GitHub Pages	**VERIFICADO** (rutas del laboratorio responden 404)

#	Capa	Dónde está	Estado
2	**Cloudflare** delante del sitio (encabezados, bots, límites)	Panel de Cloudflare	**Pendiente** (capítulo 20, **P-29**)
3	**Guardia público** (`public_guard.py`): cierra rutas peligrosas y limita a 120 peticiones por minuto	Servidor	**VERIFICADO** en pruebas
4	**Guardia de red local** (`lan_guard.py`): agentes y voz solo desde `AETHER_LAN_CIDRS`	Servidor	**VERIFICADO** en pruebas
5	**Validación estricta de contratos** (campos extra prohibidos, tamaños, versiones)	`agent_contracts.py`	**VERIFICADO** en pruebas
6	**Firewall de Windows** en perfil Privado	Laptop	Regla escrita; **NO verificada** de punta a punta con el Jetson
7	**Sin** `gpiod` : ningún botón mueve un pin	Código de la celda	**VERIFICADO** (capítulo 7)

Y, **fuera del software**, la capa que no se puede reemplazar: el **paro de emergencia físico** (capítulo 4), que en el robot es un diseño y en la celda **no está documentado** (**PENDIENTE (P-16)**).

Lo que el guardia público permite

En modo restringido (petición del dominio público o de una IP no local) solo se permiten `GET /health` , `GET /inventory` y `GET /inventory/history` . Todo lo demás responde 403: cámaras (`/perception`), celda (`/line/`), robot (`/robot`), dataset, Spotify, show, voz, configuración y el auto-refresh. Las rutas ocultas (`/.env` , `/.git` , `*.db` , `*.bak` , `*.sqlite`) responden 404. Los encabezados de seguridad se añaden siempre. El modo se elige con `AETHER_PUBLIC_MODE` : `auto` (por defecto), `public` (siempre restringido) o `local` (**solo para el laboratorio; nunca en un servidor alcanzable desde internet**). Un detalle de ingeniería: **no se lee** `X-Forwarded-For` , porque cualquier cliente puede escribir "127.0.0.1" ahí y saltarse el cierre; solo se confía en `CF-Connecting-IP` o `X-Real-IP` si `AETHER_TRUST_PROXY` está activo y un proxy confiable los pisa.

Secretos: dónde viven y dónde nunca

Reglas verificadas en `.gitignore` y en la documentación:

Qué	Dónde vive	En Git
Claves (ElevenLabs, mensajería, celda, Spotify)	`backend/.env`	**No** (ignorado)
Token de Spotify	`backend/spotify_token.json`	**No**
Agenda de contactos para llamadas	`backend/contactos.json`	**No**
Registro de acciones de Lumina	`backend/lumina_actions.jsonl`	**No**
Base de datos	`backend/aether.db`	**No**
Grabaciones de voz de pruebas	`pruebas_voz/`	**No**
Pesos de modelos y fotos del dataset	`edge/perception/weights/`, `parts/images/`	**No**
Configuración del agente del Jetson	`~/.aether-agent.env`	Fuera del repositorio (hay plantilla sin secretos)

La regla práctica del equipo (documentada en `COMANDOS_LINUX.md`): **revisar `git diff --stat` antes de cada `push`**, y **nunca** pegar claves en el chat ni en la documentación. Dos hechos honestos: (1) la contraseña de acción administrativa (`ADMIN_ACTION_PASSWORD`) tiene un **valor por defecto de desarrollo** en el código, que debe sustituirse por entorno en cualquier uso real; (2) en una sesión previa, un secreto de la integración de música se pegó una vez en un chat y se recomendó **rotarlo** (cambiarlo por uno nuevo): que ya se haya rotado **no está confirmado**: **PENDIENTE (P-28)**. La palabra clave de esa lección: **un secreto expuesto una vez debe tratarse como expuesto para siempre y rotarse.**

El equipo además mantiene una rama de limpieza (`jetson-agente-limpia`) en la que, por ejemplo, **la dirección IP de la universidad se saca del repositorio** y se guarda en un archivo local del Jetson (`~/.aether-host`). Es el mismo criterio: lo que describe un lugar real no tiene por qué ser público.

El laboratorio no sale a internet

El documento `docs/security/laboratorio.md` es una lista de ocho comprobaciones **que nadie ha ejecutado todavía en el router** (se dice así en el propio archivo):

1. No debe haber **reenvío de puertos** hacia el 8000 (API), el 1883 y 8883 (MQTT) ni el de la cámara.

2. **UPnP apagado** (deja que un programa abra un puerto sin avisar).
3. El permiso de Python en el **firewall de Windows**, si existe, **solo para red privada**.
4. Comprobar con `netstat -ano | findstr :8000` dónde escucha el API: la dirección local no debe ser una IP pública.
5. El broker MQTT, si corre, escucha en `127.0.0.1`.
6. **Prueba con datos móviles**: desde el celular (sin WiFi) abrir `http://<IP pública>:8000/health` **debe fallar**.
7. `AETHER_PUBLIC_MODE=local` solo en la PC del laboratorio.
8. `AETHER_TRUST_PROXY` sin definir mientras no haya un proxy que pise las cabeceras.

Su estado: **PENDIENTE (P-30)** (revisar y ejecutar la lista).

Dos cuentas de seguridad que conviene no confundir

El proyecto distingue la **seguridad de información** (que nadie lea ni cambie datos) de la **seguridad funcional** (que una máquina no lastime a nadie). Los guardias y la CSP pertenecen a la primera; el paro de emergencia, el watchdog y el hecho de que **el software de la celda no pueda mover pines sin** `gpiod` pertenecen a la segunda. El error clásico es creer que una contraseña es una protección contra un movimiento peligroso: **no lo es**. El propio `cell_hmi.py` lo avisa: "*No es una certificación del sistema. El paro de emergencia no pasa por aquí.*"

Qué hacen las empresas

Las organizaciones aplican **defensa en profundidad** (varias capas, ninguna imprescindible), **mínimos privilegios** (cada programa solo puede lo que necesita), **gestores de secretos** (en lugar de archivos `.env`), **auditorías** periódicas y **segmentación** de redes industriales (IT/OT). Para plantas se aplican normas como **IEC 62443** (ciberseguridad de sistemas de control industriales). Aether es un **prototipo de laboratorio** que adopta las ideas con medios mínimos y declara con claridad lo que no cubre.

Preguntas de repaso: seguridad

1. ¿Qué pasaba con la cuenta de la celda antes del 21 de octubre y cómo se corrigió?
2. ¿Qué tres rutas permite el guardia público y qué responde a todo lo demás?
3. ¿Por qué no se confía en `X-Forwarded-For`?
4. ¿Qué archivos con secretos o datos personales están excluidos de Git?

5. ¿Qué riesgo acepta el equipo explícitamente y cómo lo diría a un jurado?
6. ¿Por qué una contraseña no es una protección contra un movimiento peligroso?

Capítulo 22: Pruebas y calidad

Qué prueba el proyecto, qué no puede probar sin hardware, qué hacer cuando dos archivos de prueba se llaman igual y cuánto aporta cada tipo de comprobación.

Un error de colección que enseñó más que una prueba

El 24 de octubre de 2026, al preparar este libro, se intentó correr **todas** las pruebas del repositorio con un solo comando. Falló antes de ejecutar ninguna: "*import file mismatch: imported module 'test_setup' has this* `__file__` *attribute:* `tests\backend\test_setup.py` *which is not the same as the test file we want to collect:* `tests\edge\test_setup.py`". Hay **dos archivos con el mismo nombre** (`test_setup.py`) en carpetas distintas, y **pytest** (la herramienta de pruebas) los confunde cuando se corren juntos sin paquetes. La solución del proyecto es sencilla y ya está en la práctica: **se corre cada conjunto por separado**, con **su propio entorno virtual**, porque la parte del servidor y la de borde tienen dependencias distintas.

Qué es una prueba automática y qué aporta

Una **prueba automática** es un programa que ejecuta otra parte del código y comprueba que haga lo esperado, **sin que nadie lo mire**. Su valor es triple: avisan **en segundos** si un cambio rompe algo, documentan **lo que el código promete** y permiten **cambiar con confianza**. Un principio del proyecto (módulos independientes y probables) hace que casi todo se pueda comprobar sin cámara, sin red y sin hardware.

Cómo se ejecutan, `[Laptop · PowerShell]`

Desde la raíz del repositorio:

```
backend\.venv\Scripts\python.exe -m pytest tests\backend -q
edge\.venv\Scripts\python.exe -m pytest tests\edge -q
```

Resultados de la ejecución real del **24 de octubre de 2026** (**MEDIDO**):

Conjunto	Resultado	Tiempo
`tests/backend`	**345 aprobadas, 2 fallidas**	34.9 s
`tests/edge`	**7 aprobadas**	0.04 s

Las **dos fallidas** son de Lumina (`test_robot_ask_conserva_varias_oraciones_reales_tras_filtrar` y `test_robot_ask_lista_numerada_tambien_se_detecta`, en `tests/backend/test_main.py`). **Ya fallaban antes de los cambios recientes** y no se tocaron: son una deuda conocida y declarada, no un hallazgo nuevo. (Se corrobora que el resultado es el mismo que el registrado en la bitácora del 22 de octubre: "264 pruebas, 262 pasan", y el de la palabra de activación: "346 pruebas, 344 pasan"; las cifras crecen con los cambios.)

Qué prueba cada archivo

La suite tiene **347 pruebas** del servidor repartidas en 24 archivos. Los cinco más grandes, para ver el reparto:

Archivo	Pruebas	Qué cubre
`test_voice_wake.py`	58	La regla tolerante de "Lumina": formas observadas, frases que **no** deben activar, falsos positivos conocidos a la vista, tecla `w`
`test_main.py`	51	Las rutas del servidor: inventario, retiro, historial, guardia público, CSP, Lumina
`test_voice_service.py`	27	El servicio de voz, con un servidor y un WebSocket reales
`test_agent_routes.py`	26	Las rutas de los agentes: 403 fuera de red, 413 por tamaño, 422 por contrato, origen ajeno, tope de conexiones
`test_agent_contracts.py`	23	Los modelos de datos y sus límites

Otros archivos con propósito claro: `test_observation_outbox.py` (reintento, cola acotada, laptop que se cae y vuelve), `test_cell_line.py` y `test_celda_agent.py` (los pulsos, con relevadores falsos; que el agente de la celda **no importe** `main`, Lumina, `torch` ni `ultralytics`), `test_dataset_store.py`, `test_settings.py` (variables de entorno inválidas), `test_scripts_arranque.py` y `test_scripts_voz.py`, y `test_dashboard_js.py`, que ejecuta con **Node** (el motor de JavaScript) las pruebas del código del panel en `tests/dashboard/`: el puente de voz de `robot.js`, la reconexión y los tres guiones de video (agente configurado, sin configurar, esquema peligroso, versión desconocida).

Dobles y relojes falsos: cómo probar sin hardware

Casi toda la lógica del proyecto está escrita para recibir sus dependencias **por parámetro**: la cámara, el modelo, el reloj, la función de envío por red, los pines. En las pruebas se pasan **dobles** (versiones falsas):

- `observation_outbox.py` recibe `send(url, payload, timeout)`; las pruebas le pasan una función que simula caídas.
- `voice/core.py` recibe un **reloj** inyectable: las pruebas "avanzan el tiempo a mano" sin esperar de verdad.
- `cell_line.py` recibe una función de espera y una clase de salidas: las pruebas usan **relevadores falsos**.

El resultado es que una lógica compleja (reintentos, anti-eco, temporizadores del pistón) se prueba **en milisegundos y sin riesgo**.

Una prueba que falla antes de la corrección

El equipo aplica una regla valiosa: **las pruebas de un error deben fallar con el código viejo y pasar con el nuevo.** Se aplicó al video bloqueado por la CSP del 23 de octubre ("Las pruebas nuevas fallan con el código anterior y pasan con el nuevo"). Una prueba que nunca pudo fallar no demuestra nada.

Pruebas de extremo a extremo y revisión estática

Algunas pruebas levantan **un servidor real** (`uvicorn`) y un **WebSocket real**: es el caso del servicio de voz con una página simulada y del agente del Jetson con una cámara falsa. Otras son **revisiones estáticas**: por ejemplo, una prueba lee el código del servicio de voz y comprueba que **no contenga** ninguna llamada a abrir archivos de audio (`open(`, `wave`, `soundfile`), de modo que la promesa "el audio nunca se guarda" se verifica **en el texto del programa**.

Lo que ninguna prueba automática puede decir

El documento de trazabilidad (`docs/auditoria/TRAZABILIDAD.md`) tiene una columna honesta para cada requisito: **COMPROBADO** (hay prueba que pasó o salida real) contra **NO COMPROBADO** (depende de hardware o voz reales). Ejemplos: **el micrófono real, Vosk con español real, el aviso de Windows, el firewall Privado, Firefox con monitor HDMI, el Jetson a 30 FPS, la Pi real**. Para esos casos existen **13 verificaciones manuales**

(V1 a V13) con comando, salida esperada y qué hacer si falla: FPS del agente (V1), página que muestra la cámara del Jetson (V2), observaciones que llegan (V3), firewall (V4), micrófono (V5), latencia del reconocedor (V6), voz de punta a punta (V7), aviso y privacidad (V8 y V9), eco (V10), Firefox con HDMI (V11), "Lumina" con la voz real (V12) y Whisper contra Vosk (V13). **La mayoría sigue sin ejecutarse**; la primera prueba del agente en el Jetson (23 de octubre) cubrió una parte de V1 y V2. Pendientes: **P-27** (las de voz) y **P-22** (las de medición del Jetson).

Calidad del código: ruff

`ruff` es una herramienta que revisa estilo y errores comunes del código Python. El proyecto la usa con `line-length = 100` y `target-version = py310`, y la bajó a Python 3.10 porque el Jetson trae esa versión (el cambio de `datetime.UTC` por `timezone.utc` también salió de ahí). Al correrla el 24 de octubre con la configuración que encuentra la laptop, `ruff check backend tests` reportó **243 observaciones**: **111** de una regla sobre el alias de banderas de expresiones regulares, **46** `noqa` sin efecto, **15** importaciones sin ordenar y **14** `except` genéricos, entre otras; **178** son corregibles automáticamente. Ninguna es un error de funcionamiento (las pruebas pasan), pero es **deuda de limpieza**, y no se ha limpiado todavía.

Una nota sobre tiempos y hábitos

El proyecto sigue una regla de trabajo que conviene copiar: **cada cambio de comportamiento viene con su prueba, o con la explicación de por qué no se puede probar**; **cada medición dice con qué hardware y en qué modo se hizo**; y los informes distinguen siempre lo que se comprobó con dobles de lo que se vio en el equipo real.

Cómo lo hace una empresa

Las empresas ejecutan las pruebas **automáticamente en cada cambio** (**integración continua**, con servicios como GitHub Actions), añaden **pruebas de integración** con hardware real en un banco ("hardware-in-the-loop"), miden la **cobertura** (qué porcentaje del código recorren las pruebas) y bloquean un cambio que rompa la suite. Aether no tiene integración continua configurada ni medición de cobertura: **NO VERIFICADO** que existan (no se encontraron en el repositorio).

Preguntas de repaso: pruebas

1. ¿Por qué falló la colección de pruebas cuando se corrieron juntas las del servidor y las de borde?
2. ¿Qué significa que una lógica reciba su reloj "por parámetro" y qué ventaja da?
3. Explicar con un ejemplo qué es una prueba de revisión estática.
4. ¿Por qué se dice que una prueba de un error debe fallar con el código viejo?
5. Citar tres cosas que ninguna prueba automática puede decir y quién las comprueba.
6. ¿Qué dos pruebas fallan hoy y por qué no se han corregido?

Capítulo 23: Git, documentación y auditoría

Cómo se guarda la historia del proyecto, cómo se registran las decisiones, por qué la documentación envejece más rápido que el código y qué hábitos permiten defender cada afirmación.

Cincuenta y nueve commits, una máquina del tiempo

Al 24 de octubre de 2026 el repositorio de Aether tiene **59 commits** (puntos de guardado) y varias **ramas**. **Git** es el programa que guarda **cada versión** de los archivos con quién, cuándo y por qué se cambió; un **repositorio** es la carpeta con su historia; un **commit** es un punto de guardado con un mensaje; una **rama** es una línea paralela de trabajo. **GitHub** es el servicio donde el equipo guarda una copia en línea; el repositorio es **público** a propósito (los comandos de Linux lo clonan sin contraseña).

Las ramas del proyecto, según el repositorio:

Rama	Para qué
`main`	La rama principal
`feature/robot-lumina-camera-sharing`	Lumina y la cámara compartida
`feature/hardening-cloudflare` y `feature/seguridad-y-alianzas`	Endurecimiento del sitio público y sección Alianzas
`jetson-setup`	Preparación del Jetson
`jetson-agente`	Los agentes de campo y la voz (la rama actual de trabajo)
`jetson-agente-limpia`	Una versión de la anterior depurada de datos específicos del lugar
`gh-pages`	El sitio público estático

Mensajes de commit que cuentan una historia

Los mensajes del proyecto son **descriptivos y en español** y suelen explicar el motivo. Ejemplos reales: "*Corrige el video del agente bloqueado en index.html: el* `<meta>` *de CSP no llevaba los orígenes del agente*"; "*Observaciones: se juntan en lotes (*`AETHER_OUTBOX_INTERVAL_S`*) en vez de un POST por observación*". El trabajo grande se partió en **bloques pequeños y reversibles** (R0 a R6 en la rama `jetson-agente`); el

documento de trazabilidad da para cada bloque su commit y **cómo se revierte** (`git revert <commit>`): una práctica de ingeniería profesional, porque permite arrepentirse sin perder el resto.

Antes de subir: la revisión de secretos

El repositorio es público, así que una clave subida **se considera filtrada para siempre**. Los hábitos del proyecto (capítulo 21): `.gitignore` con los archivos sensibles, **`git diff --stat` antes de cada `push`** para ver qué sube, ninguna clave en el chat ni en los documentos, y una rama de limpieza para sacar datos específicos del lugar. Dos comandos útiles, `[Laptop · PowerShell o Git Bash]`, desde la raíz:

```
git status --short       # qué cambió, sin subirlo
git diff --stat          # cuánto cambió en cada archivo
```

La pirámide de documentos

El proyecto tiene mucha documentación y, por eso, **un orden de autoridad**. De más a menos confiable:

Nivel	Qué es	Ejemplos
Código y pruebas	La verdad de lo que corre	`backend/*.py`, `tests/`
Bitácora de hardware real	Lo que se hizo y vio en el Jetson, con estados `[reportado]` y `NO verificado`	`docs/jetson/BITACORA_JETSON.md`, `SETUP_JETSON.md`
Contexto maestro	Decisiones de arquitectura, principios y reglas	`aether_context_docs.md`, `CLAUDE.md`
ADR (*Architecture Decision Records*)	Una decisión, sus alternativas y su estado	`docs/architecture/decisions/0001…0012`
Contratos	Qué datos intercambian los programas	`contracts/*.md`
Guías operativas	Cómo arrancar y operar	`docs/guia_arranque.md`, `COMANDOS_LINUX.md`, `URLS_Y_COMANDOS.md`
Documentos viejos o superados	Conservados como historia	`docs/academic/AI_CONTEXT.md` (marcado SUPERADO), ADR 0008

Regla de oro del libro (y de buena parte del equipo): **cuando dos niveles se contradicen, gana el de arriba.** Y un hallazgo que la auditoría del 22 de octubre documentó: "*ADR 0006 está 'Aceptada' y dice MQTT, pero* `edge/comms/mqtt_client/` *está vacío: decisión aceptada, sin una sola línea de código*". Una ADR aceptada no es una función entregada.

Las ADR: una decisión cabe en una página

Una **ADR** tiene: contexto, decisión, alternativas evaluadas, consecuencias y **estado** (borrador, aceptada, superada). En el proyecto hay **12**; algunas, como la 0004 (hardware de cómputo) y la 0006 (comunicaciones), llevan una nota honesta: "*Opciones evaluadas, ventajas/desventajas e impacto en tiempo/costo no están documentados en el contexto fuente*". Las **superadas** no se borran: la ADR 0008 (brazos robóticos) sigue en el repositorio, marcada como histórica y reemplazada por la 0010. El **proceso de cambio** (sección 9 del contexto maestro) obliga a registrar cada cambio de alcance con fecha y motivo, reflejarlo en la ADR y propagarlo a las carpetas y documentos afectados.

El registro de cambios y la bitácora

Dos documentos cuentan la evolución día a día:

- `docs/CHANGELOG.md`: solo decisiones que afectan a más de una persona (el cambio de la línea del 16 de septiembre; el cargador y la vista por capas del 30 de septiembre), con "qué cambió", "por qué", "qué deben tener en cuenta" y "qué NO cambió".
- `docs/jetson/BITACORA_JETSON.md`: **diez sesiones** entre el 21 y el 23 de octubre, cada una con *quién ejecutó qué* (Mario corría los comandos en el Jetson y pegaba la salida; el asistente de programación analizaba), lo comprobado, lo no verificado y el siguiente paso. Es el modelo de cómo se documenta hardware real: **con las salidas pegadas y las etiquetas honestas**.

Auditoría y trazabilidad

Una **auditoría** revisa qué hay, qué se afirma y si coinciden. El equipo hizo la del **22 de octubre** (`docs/auditoria/`) en fases: A (qué hay: procesos, quién inicia a quién, cómo viaja cada dato, redes, ADR vivas y muertas, contradicciones entre documentos, código y mensajes) y B (qué se sabe de fuera, con **fuentes externas citadas como referencia, no como evidencia**). De ahí salió la decisión final de agentes (`DECISION_FINAL_AGENTES.md`) y la **matriz de trazabilidad** (`TRAZABILIDAD.md`): cada requisito (RF-01 a RF-09, RNF-01 a

RNF-08) enlazado a sus archivos, a su prueba o verificación y a su estado (**COMPROBADO** o **NO COMPROBADO**). Es una herramienta poderosa para un jurado: "*¿cómo se sabe que el audio no se guarda?*" → *RF-06* → `test_voice_core.py` *(revisión estática)*.

Para comparar alternativas de arquitectura, el equipo también guardó el material de un procedimiento de **torneo de soluciones** (carpeta `.arena/` y `docs/auditoria/TAREA_BASE_ARENAS.md`), con una tarea base, una rúbrica de puntuación y rondas, para decidir con criterios explícitos en lugar de intuición.

El libro mismo es un producto de este método

Este libro respeta la pirámide: lo que no está en el código ni en la bitácora se marca **PENDIENTE**, las correcciones entre documentos están en `docs/libro/NOTAS_CORRECCIONES.md` y cada afirmación lleva su etiqueta de certeza.

Una red de conocimiento sobre el proyecto

El repositorio incluye un **grafo de conocimiento** generado con la herramienta **Graphify** (`graphify-out/`): una red de **2 336 nodos y 4 062 conexiones** agrupados en **182 comunidades** (por ejemplo "Motor de audio", "Robot Lumina (UI)", "Celda real (PLC/Modbus)", "Publicador de cámara"), con un archivo `graph.html` interactivo y un informe (`GRAPH_REPORT.md`). Sirve para ubicar rápido qué partes del proyecto se relacionan. El 94 % de sus conexiones son **extraídas directamente** del código y el 6 % **inferidas**.

Cómo lo hacen las empresas

El flujo profesional añade **revisión de código** (otra persona aprueba cada cambio, con *pull requests*), **integración continua**, **versionado de lanzamientos** y documentación **generada** a partir del código para que no envejezca. Aether tiene el germen: ADR, bitácora, trazabilidad y un paquete que **genera el informe integrador desde código** (`aether_report_package/`, con los capítulos como archivos fuente y un generador que produce el documento de Word). Lo que falta es automatizar: ninguna de esas piezas se ejecuta sola.

Preguntas de repaso: historia y documentos

1. ¿Qué diferencia hay entre un commit y una rama?

2. ¿Por qué un secreto subido a un repositorio público se considera filtrado "para siempre"?
3. Ordenar de más a menos confiable: guía de construcción, código, bitácora del Jetson, ADR.
4. ¿Qué dice la ADR 0006 y por qué "aceptada" no significa "implementada"?
5. ¿Qué hace la matriz de trazabilidad y cómo ayuda en una defensa?
6. ¿Cómo se revierte un bloque de trabajo sin perder los demás?

Capítulo 24: El video y la presentación en vivo

Cómo se sincroniza un video de cuatro minutos con un robot que habla en vivo, qué se midió de su voz y qué falta medir en el escenario.

Un video que espera a que el robot hable

Un video de presentación normal se reproduce y listo. El de Aether tiene un problema distinto: **el robot Lumina habla en vivo** (su voz sale del servicio de ElevenLabs, con una latencia que varía) y el video debe acompañarla con imágenes, música y efectos **sin adelantarse ni quedarse atrás**. El proyecto `aether-video/` resuelve esa sincronía con un método que sirve para cualquier proyecto que mezcle un guion humano y uno automático: **primero se mide, después se diseña.**

Qué es el entregable

`aether-video/` contiene un video de **~4:25** hecho con **Remotion** (una librería que genera video a partir de código React/TypeScript, de modo que cada escena es un componente editable) que **acompaña en vivo la presentación**: Lumina habla, el expositor habla y el video pone la narrativa visual, la música y los efectos. **El video no genera voz.** Hay dos entregables:

- **A.** Los **MP4** de ensayo (con la voz de Lumina en sus tiempos medianos, para revisar) y de escenario (sin voz). Ambos miden **265.045 s (4:25)**, 1920 × 1080, 30 cuadros por segundo, H.264 + AAC (**VERIFICADO** con `ffprobe`; el máximo pedido era 5:30).
- **B.** El **reproductor en vivo** (`player/`), que reacciona a los eventos reales de Lumina.

Fase 0: medir la voz real

Antes de escribir una escena, se midió a Lumina. El reporte (`phase0/REPORT.md`, 29 de septiembre) explica cómo habla: `speak(text)` pide un MP3 al servidor, que lo pide a ElevenLabs (**sin streaming ni caché**); cada frase se pide **después** de que terminó la anterior más su pausa, de modo que el hueco real es *la pausa diseñada más la latencia del servicio*. Si algo falla, cae a la voz del navegador (en esa máquina, "Microsoft Raul es-MX").

MEDIDO (Chrome 154, laptop con procesador AMD Ryzen 5 4500U y 4 GB de RAM, WiFi doméstico, 128 frases sin límite de red, 4 corridas):

Qué	Mediana	p95	Máximo
Latencia pedido → audio (ElevenLabs)	**1 428 ms**	2 696 ms	8 988 ms
Hueco real − pausa diseñada	1 438 ms	2 643 ms	6 213 ms
Inicio del guion → primer audio	2 057 ms	3 036 ms	3 198 ms
Relevo por WebSocket robot → video (misma máquina)	**4 ms**	38 ms	304 ms

(**p95** es el valor que el 95 % de las mediciones no supera: una forma más honesta de decir "casi siempre".) Con redes lentas la latencia mediana sube (**3G: 2.9 s**; **Slow 4G**: p95 de 5.2 s) y **sin internet** cae a la voz local con una latencia de unos 50 ms. **Lo que no se midió (PENDIENTE: P-31):** el reconocimiento de "Lumina" con el micrófono real (el reconocedor de Chrome ignora un micrófono falso), el hardware del escenario y la red del escenario. El propio reporte lo dice: "*Las cifras valen solo para este equipo y esta red.*"

El guion se hizo esperar al expositor

Una decisión de diseño que salió de la medición: tras la pregunta "¿Quieren ver cómo funciono?", el guion **espera a oír "Lumina"** en lugar de un temporizador de 70 segundos que **podía hablar encima del expositor**. Para ello se añadió la variante "lumia" al patrón de reconocimiento (la raíz del problema que el capítulo 15 explica con más detalle).

El timeline: cada escena atada a un evento real

`timeline.json` es la **fuente de verdad**: cada *cue* (señal de escena) con su **disparador real** (un evento de Lumina o un clic del expositor), su tiempo y de dónde sale ese número; se regenera desde la medición con `npm run timeline`. Los eventos que el robot publica por **WebSocket** son: `wake`, `script-start`, `line-request`, `line-start`, `line-end`, `await-lumina`, `heard-lumina` y `camera-on`. Se eligió WebSocket (y no `BroadcastChannel`) porque **son equipos distintos**; el relevo del servidor (`/show/ws`) mide **4 ms de mediana** en la misma máquina.

El clicker y el modo ensayo

El expositor controla el video con un **clicker** (un mando de diapositivas): `→` o `AvPág` avanza, `←` retrocede, la tecla `F` pone pantalla completa, `H` retiene o reanuda el **watchdog** (el temporizador de seguridad que evita que el video se quede esperando a un evento que nunca llega) y `E` abre el **panel de ensayo**. El **modo ensayo** (`?ensayo=1`) simula a Lumina con la distribución de tiempos **medida** de cada perfil de red, con retrasos

añadidos, eventos perdidos y la duración del expositor; no necesita red. Existe además un modo **solo clicker** (`?clicker=1`) que reproduce el mismo video sin robot, y en la diapositiva de cámara pide el MJPEG real (`GET /perception/stream`) y **no deja seguir hasta confirmar EN VIVO o SIN CÁMARA**.

Resultados verificados (en la misma laptop)

Una corrida real de extremo a extremo (con `robot.js` real, voz real de ElevenLabs, el servidor real y el reproductor real, en contextos separados de Chrome) dio, **MEDIDO**:

Qué	Resultado
Sincronía de 32 frases (error entre el cue y el inicio de la voz)	mediana **+3 ms**, p95 **75 ms**, máx. **91 ms** (cumple ±250 / ±500 ms)
Relevo robot → video	mediana 2 ms, p95 6 ms
Watchdog, duplicados, errores de JavaScript	0 / 0 / 0

Con la voz local de respaldo, la mediana fue +56 ms y el p95, 71 ms. **Nada de esto se midió en el hardware ni en la red del escenario.** Un primer intento con ElevenLabs se quedó a medias porque la máquina se congeló por falta de memoria: una nota de honestidad que el propio documento de verificación conserva.

Música y efectos generados por código

La música, las camas por sección, los golpes por cue y los efectos se **sintetizan con Web Audio** (`audio/engine.js`, la partitura y la síntesis) y se renderizan con scripts (`npm run music`, unos 20 minutos en esa laptop). Hay medidores de **niveles** de voz y música (`scripts/levels.mjs`) para comprobar que la música no tape la voz (el efecto *ducking*, que baja la música cuando alguien habla). El texto en pantalla del video está en **español** (decisión del equipo).

Montaje en el escenario

Tres equipos: la **laptop del servidor** (`uvicorn main:app --host 0.0.0.0 --port 8000`), la **pantalla del robot** (`http://<IP>:8000/robot.html`, en primer plano y pantalla completa porque un navegador congela los temporizadores de una pestaña en segundo plano) y el **equipo del video** (`npm run show`, en `http://localhost:5055/player/?backend=<IP>:8000`). Un clic o tecla "arma" el audio (política de *autoplay*, capítulo 11).

También el informe y los dictados

El mismo espíritu de "generar desde código" se aplicó a otros entregables: el **informe integrador** (un documento de Word generado desde archivos fuente en `aether_report_package/`), el **dictado sin símbolos para ElevenLabs** (`AETHER_DICTADO.txt`) y el **compendio** del proyecto (`AETHER_COMPENDIO.md`), que explica también cómo se prepara el texto para que una voz sintética lo lea sin tropezar.

Cómo se hacen las presentaciones técnicas serias

En conferencias y ferias se ensaya con **cronometraje**, se prepara un **plan B** sin red (aquí: el modo solo clicker y la voz local), y se mide el **retraso de extremo a extremo** de lo que depende de internet. Aether sigue el mismo plan con un ingrediente poco común: las cifras de latencia están **medidas y publicadas** con sus límites.

Preguntas de repaso: la presentación

1. ¿Por qué el video no genera la voz?
2. ¿Qué mide el p95 y por qué se reporta junto a la mediana?
3. ¿Qué problema evita el cambio de "temporizador de 70 s" a "esperar a oír Lumina"?
4. ¿Por qué se usa WebSocket y no `BroadcastChannel` entre la pantalla del robot y el video?
5. ¿Qué hace el watchdog del reproductor?
6. ¿Qué dos cosas **no** se midieron y deben medirse antes de la presentación real?

Capítulo 25: Manual para cuando todo falla

Un método de diagnóstico y un catálogo de síntomas reales de Aether, ordenados por tipo, con la causa que tuvo cada uno y qué hacer.

Un método antes que una lista

Casi todas las fallas del proyecto se resolvieron con el mismo hábito, que conviene aprender antes de memorizar síntomas: **partir el problema en capas y probar cada capa por separado.** El video no aparece en la página: ¿falla la **cámara**? ¿el **agente**? ¿la **red**? ¿el **navegador**? Cada pregunta tiene un comando que la responde (capítulos 10, 11 y 16). Dos reglas más:

1. **Copiar el mensaje exacto.** "*Connection refused*", "*timeout*" y "*No such host*" son tres fallas distintas con tres causas distintas.
2. **Cambiar una sola cosa por vez** y volver a probar. Con dos cambios a la vez, no se sabe cuál arregló o rompió.

Fallas de arranque y de configuración

Síntoma	Causa más probable	Qué hacer
El servidor se detiene al arrancar con un mensaje que nombra una variable (por ejemplo `AETHER_DETECT_MAX_HZ debe ser a lo más 120`)	Variable de entorno mal escrita; es **a propósito** (capítulo 13)	Corregirla en `backend/.env` y arrancar de nuevo
`No module named 'sqlalchemy'` al arrancar	Se usó el Python del sistema, no el del entorno virtual	Usar `backend\.venv\Scripts\python.exe` (`[Laptop · PowerShell]`)
`Address already in use` / puerto 8000 u 8888 ocupado	Hay otro proceso del servidor vivo	Cerrarlo (`[Jetson · SSH]`: `ps aux | grep uvicorn`, luego `kill <PID>`); en la laptop, cerrar la otra ventana
`.\iniciar_aether.ps1` dice `[FALTA]` y un comando	Falta algo (clave de la celda, regla de firewall, perfil de red)	Hacer exactamente lo que dice debajo de la flecha `->`

Síntoma	Causa más probable	Qué hacer
El agente del Jetson se niega a arrancar diciendo "el gpiod de Python está instalado"	Regla de seguridad (capítulo 9)	**No continuar**; confirmar el cableado de la celda antes de instalarlo
La celda o `celda_agent` no arranca y dice que el modelo "trae N clases y classes.txt tiene M"	El `.onnx` no corresponde a las clases actuales	Volver a exportar `parts.pt` a ONNX (capítulo 8)
`import file mismatch` al correr todas las pruebas	Dos `test_setup.py` en carpetas distintas	Correr `tests/backend` y `tests/edge` por separado (capítulo 22)

Fallas de red

Síntoma	Causa más probable	Qué hacer
`curl` desde el Jetson a la laptop: **timeout**	El servidor no corre, el firewall lo bloquea o están en subredes distintas	`.\iniciar_aether.ps1 -SoloComprobar`; revisar el perfil de red (Privado) y la regla del puerto 8000
`Connection refused`	La máquina responde pero nadie escucha	Arrancar el servidor o el agente
`No such host is known` con un nombre `.local`	mDNS no funciona en esa red	Usar la IP
SSH se corta con `Connection reset`	Dos interfaces de red activas en el Jetson (WiFi y cable)	Apagar el WiFi (`nmcli radio wifi off`) y usar `tmux`
Funciona unos minutos y se cae	La laptop se suspende o cambió la IP	Configurar energía "Nunca" al estar conectada; revisar la IP actual

Fallas de video, navegador y página

Síntoma	Causa más probable	Qué hacer
El video del agente no aparece pero `/stream` da 200	**CSP** del navegador (`img-src`) bloqueó la imagen (capítulo 11)	Mirar la consola (`F12`); reiniciar el servidor con `AETHER_PERCEPTION_AGENT_URL` y recargar con `Ctrl+F5`
La página dice "Bloqueado por la política de seguridad (CSP)"	Mismo caso; el mensaje ya lo distingue	Igual

Síntoma	Causa más probable	Qué hacer
"No se pudo cargar el video… el agente no responde"	El agente no corre o su cámara no abre	Probar `.../health` del agente
Video a 2304 × 1536 y lentísimo	OpenCV abrió la cámara por GStreamer (capítulo 6)	Poner `*_CAMERA_BACKEND=v4l2`, `FOURCC=MJPG`, 640 × 480
Pocos FPS	Poca luz (la cámara alarga la exposición) o modo de energía bajo	Más luz; medir con el modo anotado
Firefox del Jetson muy lento	Renderizado por software a través de VNC (no medido con HDMI)	Probar con un monitor conectado
Los navegadores en snap no abren	`snap-confine` falla (capítulo 9)	Usar Firefox del PPA de Mozilla; no tocar `snapd`

Fallas de voz y de Lumina

Síntoma	Causa más probable	Qué hacer
Lumina no despierta	Vosk no escribe "Lumina"; o hay ruido/murmullo	Ver el texto reconocido (`AETHER_VOICE_LOG_TEXT=1`); usar la tecla `w`; micrófono de diadema
La consola de voz dice `sin conexión con el servidor`	El servidor no corre o la dirección es otra	Arrancarlo; revisar `AETHER_VOICE_SERVER`
Lumina se contesta a sí misma	Eco	Audífonos o subir `AETHER_VOICE_ECHO_TAIL_S`
"ElevenLabs no disponible, uso voz del navegador"	La clave falta o la CSP (`media-src`) bloqueó el audio	Revisar `backend/.env` y la consola del navegador
`503` "Lumina no pudo pensar una respuesta"	Ollama no responde	`ollama serve` y `ollama pull llama3.2:1b`
El servicio de voz dice que falta el modelo	No está `AETHER_VOICE_MODEL`	Descargar el modelo `vosk-model-small-es` y apuntar a su carpeta

Fallas de la celda

Síntoma	Causa más probable	Qué hacer
Estado **DETENIDA** con motivo "Cámara apagada"	La visión no ha juzgado un cuadro en 2 s	Abrir la cámara de la celda; revisar `/health` de `celda_agent`
Alarma "Enlace PLC perdido"	`sources.plc` es `false` porque `gpiod` no está (capítulo 7)	Es **esperada** hoy; no es una falla del PLC
Arranque y Paro no responden	Falta `CELDA_HMI_CLAVE` (nadie inicia sesión)	Definirla en el entorno de la celda
Piezas contadas como `pulso_no_enviado`	`gpiod` ausente: nada se mueve a propósito	Correcto y seguro; para actuar real hay que cumplir la lista del capítulo 8
Un FAIL falso	Umbral de estabilidad bajo	No bajar `LINE_ESTABLES` de 3 sin probar
Percepción y celda se estorban	Ambas cámaras usan `/dev/video0`	No abrirlas a la vez

Fallas de instalación en el Jetson

Síntoma	Causa más probable	Qué hacer
`torch.cuda.is_available()` es `False`	`pip` instaló el torch de PyPI (sin GPU)	Reinstalar la rueda de NVIDIA con `--force-reinstall --no-deps`
Aparece `Uninstalling torch` durante una instalación	Una dependencia quiere cambiar el torch	**Cancelar** con `Ctrl+C` antes de que termine
`unable to allocate CUDA0 buffer`	Posible efecto de actualizar paquetes de Ubuntu (reporte de un foro, **no verificado** aquí)	No usar `apt upgrade` en ese equipo
`ultralytics` intenta instalar `opencv-python`	Falta `YOLO_AUTOINSTALL=False`	Exportar esa variable
`numpy` demasiado nuevo	Conflicto con la versión que piden otras librerías	Mantener `numpy>=1.23,<2`

Fallas eléctricas y de hardware (las del carrito, en diseño)

Estas aparecen en la hoja de armado como "error típico" de cada etapa; ningún síntoma de esta tabla se ha reportado todavía en un carrito armado:

Síntoma	Causa más probable	Qué hacer
El multímetro **pita** entre + y − con la batería desconectada	Cortocircuito	**No conectar nada**; buscar el corto
Un A4988 se quema	Motor conectado o desconectado con el driver encendido, VDD a 5 V o sin capacitor	Revisar la receta de la Etapa 11
El ESP32 se reinicia al mover el servo	El servo se alimenta del ESP32	Alimentarlo del XL4005
El ESP32 "no se deja programar"	Un pin con trampa de arranque conectado, o cable USB de solo carga	Desconectar esas líneas; usar un cable de datos
Los motores-rueda no contestan por RS485	A y B cruzados	Invertir A y B
Los sensores VL53L0X o el RS485 fallan "al azar"	Cables de señal junto a los de potencia	Separar y trenzar
Una llanta no toca el piso	Suspensión descompensada	Revisar la precarga del resorte

Una regla para todos los casos

Si después de revisar su tabla algo sigue fallando, la regla del equipo es: **escribir en la bitácora qué se hizo, qué se vio (con la salida pegada) y qué se supone**, y marcarlo así con las cuatro etiquetas. Una falla documentada se convierte en una fila de este manual; una falla ignorada reaparece en la demostración.

Cómo aprende una planta de sus fallas

Los equipos de planta mantienen **procedimientos operativos** de falla (qué hacer cuando suena cada alarma), **registros** de eventos y un **ciclo de aprendizaje**: cada falla grave termina en un cambio de diseño o de procedimiento. La bitácora de Aether es la versión de laboratorio de ese ciclo.

Para practicar: casos de diagnóstico

1. La página muestra un recuadro vacío en lugar del video del Jetson, pero `curl localhost:8001/health` responde bien. ¿Qué dos pruebas siguen, en qué orden?
2. `curl` desde el Jetson da *timeout* hacia la laptop pero el servidor corre. ¿Qué tres causas revisar?

3. Lumina no despierta y la consola de voz dice "MICRÓFONO ACTIVO". ¿Qué se mira primero?
4. La celda marca DETENIDA y "Cámara apagada" aunque el agente corre. ¿Qué verifica el estado?
5. `torch.cuda.is_available()` dio `False` tras instalar `ultralytics`. ¿Cuál es la primera hipótesis y cómo se comprueba?
6. Un A4988 se quemó. Nombrar tres causas que la hoja de armado enumera.

Capítulo 26: Hacia dónde puede crecer Aether

Lo que está pendiente según los propios documentos, lo que no conviene hacer todavía, las tecnologías que habría que aprender y un criterio para decidir qué va primero.

Un proyecto se mide por lo que sabe que le falta

Un proyecto sano es capaz de listar sus huecos con la misma precisión con que lista sus logros. Este capítulo reúne los huecos que el propio repositorio declara (`docs/TODO.md`, la ADR 0012, los criterios de aceptación y la bitácora), separa los que **están bloqueados por hardware** de los que **se pueden avanzar ya**, y propone, como **opinión** (no evidencia), un orden de trabajo.

Lo que no se puede avanzar sin hardware o decisiones

Según `docs/TODO.md` (sección "bloqueado por hardware"), y con las decisiones del 24 de octubre ya aplicadas:

- **Confirmar la interfaz real de la celda**: el equipo decidió GPIO + relevadores hacia el LOGO; faltan los **detalles físicos** (pines, placa de relevadores, modelo de LOGO), que figuran en los pendientes del libro.
- **El programa del ESP32**: la decisión es PlatformIO; el **contrato con el Jetson**, el **protocolo del motor-rueda** y el programa mismo no existen.
- **La cámara del robot (USB o CSI)**: la cámara USB ya funciona en el Jetson; si el diseño final la mantiene, falta definirlo.
- **Pruebas de temporización del pistón** contra la velocidad real de la banda: necesitan pistón, banda y piezas juntos.
- **Flasheo y comprobación de versiones del Jetson**: ya hechos (capítulo 9); queda medir con carga real.

Lo que sí se puede avanzar ya

1. **El criterio real de PASS/FAIL por defectos** (`backend/main.py`, función `_make_line_vision_processor`). Hoy distingue piezas registradas de otras cosas, pero **no inspecciona calidad**. Se puede desarrollar y probar con la cámara que ya existe.

2. **Más datos para el modelo de piezas**: el repositorio conserva 19 fotos; un mAP50 de 0.995 con tan pocas fotos es **optimista**. Más fotos, con distintas luces y fondos, y fotos de **objetos que no son pieza** para medir el FAIL de verdad.
3. **Persistir los** `event_id` de las observaciones en disco, para que un reinicio de la laptop no repita eventos (capítulo 16).
4. **Cerrar la deuda de limpieza**: las dos pruebas fallidas de Lumina y las 243 observaciones de estilo (capítulo 22).
5. **Medir con voz y ruido reales** (verificaciones V12 y V13) y decidir si se instala Whisper (capítulo 15).
6. **Medir el agente del Jetson con buena luz y con el modo de energía anotado**, y decidir **TensorRT con datos**.
7. **La Raspberry Pi 4 real**: sistema, medir el modelo ONNX con su versión de OpenCV, cámara V4L2 y, solo al final, GPIO con LED y multímetro (capítulo 8).
8. **Autenticación de los agentes**, si el proyecto sale del laboratorio.
9. **Arranque automático** (systemd) para la celda y, cuando resulte estable, el agente del Jetson.

Lo que se descartó a propósito

La ADR 0012 lista lo que **no** se adoptó y por qué, una lección en sí misma: **TensorRT** (no se puede sostener ni documentar sin medir en el Jetson), **systemd** (otra pieza que mantener; `tmux` es el procedimiento actual), **WebRTC y RTSP** (más complejos que MJPEG), **MQTT** como canal de los agentes (HTTP ya está probado), **ROS 2**, **Docker**, **Tailscale** y una **aplicación nativa**. Y, de la arquitectura anterior, el **aprendizaje por demostración** y los **brazos robóticos**. Un proyecto de ~240 horas no puede permitirse todo; elegir qué **no** hacer es una decisión de ingeniería.

ROS 2 y SLAM, sin miedo

Dos siglas que cualquier jurado pregunta:

- **ROS 2** (*Robot Operating System 2*) es un conjunto de librerías y convenciones para robots: estándares de mensajes, herramientas de navegación y simulación. Aether lo descartó por **costo de aprendizaje** para el tamaño del equipo, no por defecto técnico.
- **SLAM** (*Simultaneous Localization and Mapping*, "localización y mapeo simultáneos") es la técnica con que un robot construye un mapa mientras se ubica en él, normalmente con **lidar** o cámaras. El proyecto lo **difirió**: el robot navega por

waypoints con **ArUco** como verificación; un SLAM completo excede el tiempo disponible y no es indispensable para el ciclo del laboratorio.

Tecnologías que habría que aprender, por área

Área	Tecnología	Para qué
Firmware	**PlatformIO**, librerías de motores a pasos, comunicación serie	El programa del ESP32
Visión	**TensorRT**, entrenamiento con más datos, métricas por clase	Rendimiento y FAIL real
Datos	**PostgreSQL**, índices	Escalar el inventario
Comunicaciones	**MQTT** con un broker, tópicos de telemetría	La Opción C (estado en vivo)
Operación	**systemd**, **Docker**, **monitoreo**	Arranque y reproducibilidad
Seguridad	Autenticación, gestor de secretos, **IEC 62443**	Salir del laboratorio
Calidad	**Integración continua**, cobertura	Pruebas en cada cambio
Navegación	**Odometría con IMU**, **SLAM**, lidar	Autonomía en espacios no marcados

Qué pedirle a un asistente de programación

El proyecto trabaja con un asistente de IA y tiene **reglas escritas** para él (`aether_context_docs.md`, sección 7): no inventar archivos ni dependencias, no afirmar que algo funciona sin comprobarlo, no modificar varios módulos sin necesidad, explicar antes de un cambio estructural y **esperar confirmación** cuando sea arquitectónicamente significativo. Un buen encargo concreto: "*Implementa el criterio PASS/FAIL con tolerancias por pieza y escribe la prueba que falla con el código viejo*"; uno malo: "*Mejora la visión*".

Ejercicio de priorización (opinión)

Un criterio útil: **primero lo que reduce el mayor riesgo para la próxima demostración, después lo que lo habilita todo, después lo que embellece.** Con ese criterio, un orden razonable de las nueve tareas de arriba sería: **1)** pruebas de seguridad física y paro (lo que más daña si falla), **2)** FPS del Jetson y datos de la Pi (lo que habilita la demostración), **3)** criterio de PASS/FAIL y más fotos (lo que hace creíble la celda), **4)** firmware del ESP32 (lo que habilita al robot), y solo después autenticación, systemd y TensorRT. Es **una opinión, no una decisión**: el orden real lo fija el equipo según la fecha de la demostración.

Cómo crecen los productos reales

Un producto como Aether se vuelve sistema comercial agregando **versiones** estables, **certificaciones** de seguridad, **un servicio en la nube opcional** para varias sedes y un **soporte** que pueda reproducir fallas con registros. El propio mensaje comercial del proyecto, "*el robot revisó el Anaquel 4 y encontró 23 unidades*", recuerda que el valor está en la **confianza del dato**, no en la complejidad técnica.

Preguntas de repaso: el futuro

1. Citar tres tareas que están bloqueadas por hardware y tres que no.
2. ¿Por qué un mAP50 de 0.995 con 19 fotos debe tomarse con cautela?
3. ¿Qué es SLAM y por qué se difirió?
4. ¿Por qué tiene sentido listar lo que se descartó?
5. ¿Qué criterio de prioridad se propone y qué tarea saldría primera?
6. Redactar un encargo concreto para un asistente de programación sobre el criterio PASS/FAIL.

Capítulo 27: Lo que enseñan las empresas reales

Cuatro historias de robots de almacén y de tienda, qué las hizo funcionar o fracasar y qué le dicen a un proyecto como Aether.

> **Nota de método.** Los datos de este capítulo son **conocimiento general** sobre hechos públicos (no se verificaron durante la escritura del libro, que no tuvo acceso a fuentes externas). Las cifras son **aproximadas**: conviene confirmarlas en la fuente original antes de citarlas ante un jurado. Lo que aquí vale es la lección, no el número.

Por qué mirar a otras empresas

Un jurado casi siempre pregunta: "*¿esto ya existe? ¿por qué sería distinto?*" Responder bien exige conocer dos o tres casos reales y poder decir **qué hicieron, qué les funcionó y qué les falló**, sin exagerar la comparación. Aether es un prototipo universitario; las empresas de abajo son compañías con cientos de ingenieros. La comparación honesta no es de tamaño, sino de **decisiones**.

Amazon y Kiva: comprar la idea

Hacia 2012, **Amazon adquirió Kiva Systems** (una empresa de robots que llevaban **estantes completos** hasta el operario, en vez de que el operario caminara hasta el estante) por una suma del orden de varios cientos de millones de dólares. La lección: el valor no estaba en un robot "inteligente", sino en **rediseñar el proceso** para que el robot hiciera lo que mejor sabe (transportar) y el humano lo que mejor sabe (tomar piezas). Un paralelo con Aether: el carrito **no manipula**; transporta fixtures, y la inspección la hace una cámara fija. El proyecto aprendió algo parecido al **abandonar los brazos robóticos**: simplificar el proceso redujo el riesgo.

Walmart y los robots de anaquel que se terminaron

Hacia 2020, **Walmart puso fin a un programa de robots que escaneaban los estantes** de sus tiendas (con la empresa Bossa Nova), tras años de pruebas. Las razones públicas más repetidas: el robot detectaba faltantes, pero **la tienda no los resolvía más rápido** y había alternativas más baratas (cámaras fijas, empleados con dispositivos). La lección para

Aether, que es un robot de inventario: **detectar no es resolver.** Un dato de inventario solo vale si alguien lo usa. De ahí la insistencia del proyecto en un **inventario declarado con historial y confianza**, y no en un conteo "mágico".

Simbe: el que sí escaló

Simbe Robotics sigue operando con un robot de inventario para tiendas. A diferencia de otros, su producto se vende como **un servicio de datos** (cuánto hay, dónde y qué falta) integrado a la operación de la tienda, no como un robot. La lección: lo que escala es la **utilidad del dato**, no el movimiento del robot. Aether lo anticipa con su mensaje comercial: "*el robot revisó el Anaquel 4 y encontró 23 unidades del producto X*".

Zebra y Fetch Robotics: el almacén se consolida

En 2021 **Zebra Technologies** (empresa de códigos de barras, impresoras y escáneres) compró **Fetch Robotics**, fabricante de **AMR** (robots móviles autónomos) para almacén, por una cifra de unos cientos de millones de dólares. La lección: los **códigos y lectores** y los **robots** se juntan porque resuelven el mismo problema (saber dónde está cada cosa). Aether, con su **enfoque híbrido** (detección + código de barras/QR), ya está en esa intersección.

Qué dicen las historias juntas

Lección	Cómo se refleja en Aether
El valor está en **rediseñar el proceso**, no en el robot	Se abandonó el brazo por banda + pistón y se separó transporte de inspección
Detectar no es resolver	Inventario declarado con historial, estados de cobertura y retiro explícito
Lo que escala es **el dato confiable**	Observaciones con confianza, `event_id`, bitácora auditable
Los **códigos** y los **robots** convergen	YOLO + QR + ArUco en un solo pipeline
La **seguridad** pesa tanto como la inteligencia	Paro físico en el diseño, nada se mueve sin `gpiod`, honestidad sobre lo no certificado
Muchos fracasos son de **expectativa**	Principio de honestidad de producto y etiquetas de certeza

Qué podría preguntar un jurado

Algunas preguntas probables y la clase de respuesta honesta que el proyecto puede dar:

1. **"¿Qué lo hace distinto de un robot comercial?"** — No pretende serlo: es un prototipo que demuestra integración completa con hardware real y un método de trabajo honesto; su diferenciador es la **trazabilidad y la confianza del dato**.
2. **"¿Qué parte funciona de verdad?"** — La percepción, el inventario, el panel y Lumina en laptop, y el agente en el Jetson (29 FPS medidos); la celda y el carrito están por probar en hardware real.
3. **"¿Cómo garantizan la seguridad?"** — Con un paro físico de contacto normalmente cerrado que corta la potencia independientemente del software; hoy es diseño del robot, y el paro de la celda **está pendiente**.
4. **"¿Qué pasa si falla la red?"** — La celda decide y actúa sola; los agentes acumulan datos en una cola acotada y los entregan cuando vuelve.
5. **"¿Cuánto acierta la visión?"** — Un mAP50 de 0.995 con pocas fotos; el criterio de calidad por defectos no existe todavía.

El Apéndice B desarrolla estas respuestas con las afirmaciones permitidas y las prohibidas.

Preguntas de repaso: casos reales

1. ¿Qué rediseñó Kiva en el proceso de un almacén y qué paralelo tiene con el carrito de Aether?
2. ¿Por qué se terminó el programa de robots de anaquel de Walmart y qué enseña a un robot de inventario?
3. ¿Qué diferencia hay entre vender un robot y vender un servicio de datos?
4. ¿Por qué se juntan los fabricantes de lectores de códigos y los de robots móviles?
5. Dar dos preguntas de jurado y la respuesta honesta a cada una.

Apéndice A: Hoja de arranque y apagado

El orden exacto para encender, comprobar y apagar el sistema, máquina por máquina. Cada comando dice dónde se ejecuta. Lo que viene de documentos del equipo y no se ha corrido en el hardware real queda marcado.

> **Estado de esta hoja.** Los pasos de la laptop vienen de `docs/guia_arranque.md` e `iniciar_aether.ps1`. Los del Jetson, de `COMANDOS_LINUX.md` y los scripts `.sh`; los del agente se corrieron una vez en el Jetson (prueba reportada el 23 de octubre). Los de la **Pi 4** y del **carrito no se han corrido** en hardware real. El prefijo de cada comando indica la máquina: `[Laptop · PowerShell]`, `[Jetson · SSH]`, `[Pi 4 · SSH]`.

Antes de empezar, una sola vez

1. **Clave de la celda** en `backend\.env` de la laptop: `CELDA_HMI_CLAVE=<clave>` (sin ella, Arranque y Paro no responden; la clave no se pega en el chat ni en el repositorio).
2. **Regla de firewall** en perfil **Privado** (`[Laptop · PowerShell, como administrador]`), ver capítulo 10.
3. **Direcciones de los agentes** en `backend\.env` (por ejemplo `AETHER_PERCEPTION_AGENT_URL=http://<ip-jetson>:8001`).
4. **Voz** (opcional): instalar `requirements-voice.txt`, descargar el modelo de Vosk y calibrar con `probar_voz.py` (capítulo 15).
5. **Energía de la laptop**: "Nunca" suspender mientras esté conectada.

Arranque, en este orden

1. La laptop (el servidor central)

`[Laptop · PowerShell]`, desde la raíz del repositorio:

```
.\iniciar_aether.ps1
```

Hace las comprobaciones, imprime las IP de la laptop y arranca el servidor en esa ventana; si la voz está lista, la abre en **su propia ventana**. Opciones: `-SoloComprobar` (no arranca nada), `-SinVoz`, `-Jetson` (arranca también el agente por SSH). Respaldo manual:

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000
```

2. El agente del Jetson

`[Jetson · SSH]`, la primera vez copiar la configuración y editar la IP de la laptop (`AETHER_LAPTOP_URL`):

```
cp ~/aether/scripts/jetson/aether-agent.env.example ~/.aether-agent.env && nano ~/.aether-agent.env
```

Cada vez:

```
bash ~/aether/scripts/jetson/iniciar-agente.sh
```

Salida esperada: `Agente en marcha (tmux 'agente'...)` y `{"status":"ok","error":null}`. Si dice que `gpiod` **está instalado, no se sigue**.

3. La celda en la Raspberry Pi 4 (sin probar en hardware real)

`[Pi 4 · SSH]`, dentro de `tmux`:

```
cd ~/aether/backend && ~/celda-venv/bin/python -m uvicorn celda_agent:app --host 0.0.0.0 --port 8000
```

La página: `http://<ip-pi>:8000/celda.html`. Mientras `python3-libgpiod` **no** esté instalado, las piezas se anotan y **nada se mueve** (capítulos 7 y 8).

4. El carrito (diseño; sin armar según el repositorio)

Orden de la hoja de armado: **conectar el XT60** y **encender el KCD1** (lógica y Jetson); el **relé** cierra solo si el **paro** está suelto. Antes de cualquier movimiento, **ruedas en el aire** y comprobar el paro (capítulo 4). **No hay firmware del ESP32 todavía**: este paso es de diseño.

Comprobar que todo vive

Qué	Comando	Esperado
Laptop	`[Jetson · SSH]` `curl -s -m 3 http://<ip-laptop>:8000/health`	`{"status":"ok"}`

Qué	Comando	Esperado
Agente	`[Jetson · SSH] curl -s localhost:8001/health`	`{"status":"ok","error":null}`
Estado del agente	`[Jetson · SSH] curl -s localhost:8001/status`	`camera.fps_camera`, `laptop.reachable: true`
Medición de 15 s	`[Jetson · SSH] python3 ~/aether/scripts/probar_agente.py http://localhost:8001 --segundos 15`	FPS de cámara y entrega, ms de inferencia, CPU y RAM
Celda	`[Pi 4 · SSH] curl -s localhost:8000/health`	`{"status":"ok","error":null}`
Todo, desde la laptop	`[Laptop · PowerShell] .\iniciar_aether.ps1 -SoloComprobar`	"Todo en orden" (o la lista de lo que falta)

Ver la página de Lumina dentro del Jetson (en una terminal de la sesión **VNC**): `bash ~/aether/scripts/ver-aether.sh robot.html`.

Operación diaria de la voz

En la ventana del servicio de voz: `p` pausa y reanuda (pausar **suelta** el micrófono), `w` despierta a Lumina sin hablar y `q` sale. Para arrancar pausado: `AETHER_VOICE_START=off`.

Guardar y apagar sin perder nada

En orden inverso al arranque:

1. **Carrito** (diseño): pulsar el paro, **apagar el KCD1** y **desconectar el XT60**.
2. **Pi 4**: detener `celda_agent` (`Ctrl+C` dentro de `tmux`) y apagar con `sudo shutdown now`, `[Pi 4 · SSH]`. Un apagado limpio evita corromper la tarjeta microSD (regla general).
3. **Jetson**: `[Jetson · SSH]` `bash ~/aether/scripts/jetson/detener-agente.sh` y después `sudo shutdown now`.
4. **Laptop**: en la ventana de voz, `q`; en la del servidor, `Ctrl+C`.
5. **Registro**: anotar en la bitácora lo que se hizo y se vio, con la salida pegada.
6. **Git**: `[Laptop · PowerShell]` `git status --short` y `git diff --stat` antes de cualquier `push`; **nunca** subir `.env`, tokens, `aether.db`, pesos de modelos ni fotos del dataset.

Qué se pierde al reiniciar

Las sesiones de `tmux` y los procesos de los agentes; **no** los archivos, los entornos virtuales ni las librerías. Hay que volver a lanzar `iniciar-agente.sh` en el Jetson y el agente de la celda en la Pi. La base de datos (`backend/aether.db`) vive en la laptop y se conserva.

Si algo no conecta

Seguir el orden del capítulo 10 (`iniciar_aether.ps1 -SoloComprobar` → `curl` a la laptop → `curl` al agente → `probar_agente.py`) y el catálogo del capítulo 25.

Apéndice B: Lista para la defensa

Qué se puede afirmar con respaldo, qué hay que matizar, qué conviene no decir y qué revisar antes de presentar. Cada línea remite al capítulo donde está su evidencia.

Se puede afirmar (está comprobado)

- **El código de percepción híbrida corre y está probado**: YOLO + lectura de códigos + ArUco, con pruebas automáticas (capítulo 14). La suite del servidor dio **345 pruebas aprobadas y 2 fallidas** el 24 de octubre (las dos fallidas son de Lumina y ya fallaban antes) (capítulo 22).
- **El agente de percepción corrió en el Jetson real**: **29.0 FPS** de cámara y **29.1** de entrega a 640 × 480, inferencia de **68 ms**, reportado por Mario el 23 de octubre (capítulo 9).
- **PyTorch con GPU funciona en el Jetson** (`torch.cuda.is_available() = True`, CUDA 12.6) (capítulo 9).
- **El inventario separa observaciones crudas de inventario declarado**, con historial que nunca se sobrescribe y retiro por decisión humana, no automático (capítulo 17).
- **La entrega de datos es idempotente y resistente a fallas de red** (identificador por evento, cola acotada, reintento con espera creciente) (capítulo 16).
- **El audio de voz nunca se guarda ni sale de la laptop**, y una prueba lo verifica leyendo el código (capítulo 15).
- **Ningún botón de la pantalla de la celda puede mover un pin** mientras `gpiod` no esté instalado; el equipo lo comprobó leyendo el código (capítulo 7).
- **El dominio público es estático y no expone el backend** (las rutas del laboratorio responden 404 allí) (capítulo 20).
- **La contraseña de la celda no está en el código** (viene de `CELDA_HMI_CLAVE`) (capítulo 21).
- **Hay un método**: decisiones por escrito (12 ADR), bitácora con salidas reales, matriz de trazabilidad y etiquetas de certeza (capítulo 23).
- **La línea cambió de brazos a banda + pistón por una decisión documentada** (ADR 0010, 16 de septiembre) (capítulo 1).

Hay que matizarlo

No decir así	Decir así	Capítulo
"El robot navega solo"	"El robot está **diseñado** para navegar por waypoints; el firmware del ESP32 todavía no existe"	3, 5
"La celda inspecciona la calidad"	"La celda distingue piezas registradas de objetos que no lo son; **no inspecciona defectos todavía**"	7
"El modelo acierta el 99.5 %"	"El mAP50 es 0.995 con **pocas fotos**; no se probó con objetos que no son pieza"	7, 14
"La voz de Lumina funciona en cualquier ruido"	"La palabra de activación se midió con **voz sintética**; con murmullo acierta mucho menos; con voz real **no se ha medido**"	15
"La celda ya está probada en la Raspberry"	"Está **construida y probada en laptop**; la Pi real falta"	7, 8
"El carrito está armado"	"Existe un **diseño detallado** (hoja de armado rev. 3.8); no hay evidencia de que esté armado"	3
"Tiene paro de emergencia"	"El robot tiene un **diseño** de paro con contacto NC que corta la potencia; el de la celda **no está documentado**"	4, 7
"MQTT conecta todo"	"MQTT está **decidido para telemetría** pero **sin implementar**; hoy se usa HTTP y WebSocket"	18
"La seguridad está resuelta"	"Hay capas de software y reglas físicas; los agentes **no tienen autenticación** (riesgo aceptado de laboratorio); Cloudflare está pendiente"	20, 21
"Los datos son en tiempo real"	"Es una **captura continua de esta cámara**, no inventario en tiempo real de todo el piso"	17

Evitar decirlo

- **"Conteo real de inventario"**: solo existe "conteo visible detectado".
- **"El sistema aprendió la tarea"** o "learning" para el pistón: es lógica programada de punto fijo (principio 4.2).
- **"Evasión inteligente de obstáculos"**: es detección reactiva.
- **"ArUco es el sistema de navegación"**: es verificación de posición.
- **"Cobertura completa = vimos todo"**: `COMPLETE` significa que el procedimiento terminó.
- **"Es un producto certificado"**: es un prototipo de laboratorio.
- Cualquier cifra de rendimiento **sin decir en qué equipo y en qué modo** se midió.

- Cualquier afirmación sobre los motores, el peso o la batería **sin decir que es estimación**, mientras sigan **PENDIENTES**.

Preguntas probables y respuestas con respaldo

1. **¿Qué lo hace distinto?** La trazabilidad y la honestidad del dato; la arquitectura de "datos chicos" con agentes independientes; el método documentado.
2. **¿Qué pasa si falla la red?** La celda decide y actúa sola; los agentes acumulan datos y los entregan al volver.
3. **¿Qué pasa si se cuelga el Jetson con el robot en movimiento?** El diseño: el watchdog del ESP32 detiene las ruedas en 1 s, y el paro físico corta la potencia sin software. **Ambos son diseño, no probado.**
4. **¿Por qué banda y pistón y no brazos?** Reduce el riesgo de integración física con el mismo requisito de hardware real (ADR 0010).
5. **¿Qué falta?** Lo listado en la sección de pendientes del final del libro, con su responsable.

Antes de presentar

1. Correr `iniciar_aether.ps1 -SoloComprobar` (`[Laptop · PowerShell]`) y revisar que todo esté en verde.
2. Medir los **FPS del agente** con `probar_agente.py` anotando el modo de energía (`sudo nvpmodel -q`).
3. **Probar la voz con el micrófono y el ruido del lugar** (grabar y comparar) y tener la tecla `w` lista.
4. **Confirmar la clave de ElevenLabs** (válida y con créditos) y tener el plan B de la voz del navegador.
5. **Medir el video en el hardware y la red del escenario**: lo medido hoy fue en una laptop.
6. **Rotar** cualquier secreto que se haya pegado en un chat.
7. Si la celda va a mover algo: **lista de la sección "Antes de energizar"** del capítulo 8, **paro físico** cableado y a la mano.
8. Revisar `git status` y `git diff --stat`: nada de claves, tokens ni bases de datos.
9. Ensayar una caída: apagar el WiFi de la laptop y ver que la celda sigue.
10. Llevar la **hoja de pendientes** del final del libro: es la prueba de que el equipo sabe lo que falta.

Glosario de siglas y términos

*Cada término se explica la primera vez que aparece; aquí está todo junto. La columna «Cap.» dice dónde se explica. *

Término	Qué significa	Cap.
A4988	Driver (circuito de potencia) para motores a pasos: recibe STEP y DIR y entrega hasta 1.0 A por bobina.	5
ADR	Architecture Decision Record: documento corto que registra una decisión de arquitectura, sus alternativas y su estado.	3, 7
AGV / AMR	Vehículo guiado automáticamente / robot móvil autónomo: robots que mueven carga en almacenes.	3, 6
Amperio (A)	Unidad de corriente eléctrica: cuánta carga pasa por segundo.	2
Amperios-hora (Ah)	Capacidad de una batería: corriente por tiempo que puede entregar.	4
API / REST	Interfaz de un programa para que otro lo use; REST es el estilo de usarla con HTTP, con una dirección por recurso.	11, 18
ArUco	Cuadro blanco y negro con patrón único que una cámara usa para saber dónde está y hacia dónde mira.	6
AWG	American Wire Gauge: número de calibre de un cable; menor número = cable más grueso.	4
Banda transportadora	Cinta que lleva las piezas por la celda.	3, 7
Bash	La shell (intérprete de comandos) de Linux; se usa en el Jetson y la Pi.	12
BCM	Numeración lógica de los pines GPIO de la Raspberry (por el chip de Broadcom), distinta del número físico en el conector.	8
BNO055	Módulo IMU (acelerómetro, giroscopio y magnetómetro) con dirección I^2C 0x28.	6
Bornera KF301	Conector atornillable para cables, soldado a la placa.	4
Broker	Servidor intermediario de MQTT que reparte los mensajes publicados a quien se suscribió.	18
Bus	Conjunto de cables compartidos por varios dispositivos (I^2C, RS485).	5, 6
CC / CA (DC / AC)	Corriente continua (siempre en el mismo sentido) y corriente alterna (cambia de sentido; la de la pared).	2

Término	Qué significa	Cap.
Centro de gravedad	Punto donde parece concentrarse todo el peso; cuanto más alto, más fácil volcar.	3
CIDR	Notación como 192.168.100.0/24 para describir un grupo de direcciones de una subred.	10
Cliente / servidor	El cliente pide; el servidor se queda esperando a que le pidan.	Prólogo, 10
Cookie	Texto pequeño que el servidor entrega y el navegador devuelve para recordar una sesión.	11
Cortocircuito	Unión accidental entre dos puntos de distinto voltaje que deja pasar una corriente enorme.	4
CSP	Content Security Policy: la política con que el navegador decide qué recursos puede cargar una página.	9
CUDA	Herramientas de NVIDIA para que los programas usen la GPU.	9
cuDNN	Biblioteca de NVIDIA con operaciones de redes neuronales sobre CUDA.	9
Declared Inventory	Inventario declarado: resumen agrupado de las observaciones, con historial y confianza.	17
DHCP	Servicio de una red que asigna direcciones IP a los equipos que se conectan.	10
Diferencial (robot)	Robot con dos motores y una llanta por motor; gira cambiando la velocidad relativa de las llantas.	3, 5
DNS / mDNS	Directorio que traduce nombres a IP; mDNS lo hace en red local con nombres .local sin servidor central.	10
Docker	Herramienta para empaquetar un programa con su entorno exacto en un contenedor.	13, 26
Driver	Circuito que obedece una señal débil y entrega la corriente grande que necesita un motor.	2, 5
ECU	Engine Control Unit: computadora de motor de un coche; es la pieza que viaja en cada fixture.	3, 7
Electroválvula	Válvula que abre o cierra el aire cuando recibe corriente; mueve el pistón.	7, 8
Encoder	Sensor que cuenta cuánto gira un eje.	3, 5
Endpoint	Dirección del servidor que responde a un tipo de petición, como GET /health.	13
ESP32	Microcontrolador con Wi-Fi y Bluetooth; en Aether es el controlador de bajo nivel del robot.	5

Término	Qué significa	Cap.
FastAPI / Uvicorn	Librería que convierte funciones de Python en rutas web / programa que escucha el puerto y atiende las peticiones.	13
Final de carrera	Interruptor que se activa cuando una pieza móvil llega a su límite; da el cero (home).	3, 5, 6
Firewall	El portero de una computadora: decide qué conexiones entrantes deja pasar.	10
Fixture	Base que sostiene y posiciona cada pieza sobre la banda y dentro del carrito.	3, 7
FPS	Frames per second: cuadros por segundo que entrega una cámara.	6, 9
Fusible	Hilo calibrado que se funde si pasa más corriente de la permitida y corta el circuito.	2, 4
Git / GitHub	Programa que guarda cada versión del código / servicio en línea donde se guarda una copia.	23
GND	Ground: tierra o referencia de 0 V contra la que se miden los voltajes.	2, 4
GPIO	General Purpose Input/Output: pin que puede leer o encender una señal.	2, 8
GPU	Graphics Processing Unit: chip con miles de núcleos pequeños; acelera redes neuronales.	9
HMI	Human-Machine Interface: pantalla con la que un operador maneja una máquina.	7
Home	Posición cero de un mecanismo, que se busca con un final de carrera.	3, 5
HSTS	Encabezado que obliga al navegador a usar solo HTTPS con un dominio durante un tiempo.	20
HTTP / HTTPS	Protocolo de la web; HTTPS es la versión cifrada.	11
Idempotencia	Propiedad de una operación que, repetida, produce el mismo efecto que una sola vez.	16
IEC 62443	Familia de normas de ciberseguridad para sistemas de control industriales.	21
IMU	Inertial Measurement Unit: unidad que mide giro y aceleración.	6
Infrarrojo industrial	Sensor de 3 hilos (+24 V, 0 V y señal) que detecta presencia; puede ser PNP o NPN.	2, 7
IoU	Intersection over Union: medida de cuánto coinciden dos cajas; 1.0 es idéntica.	14

Término	Qué significa	Cap.
ISA-101 / ISA-18.2	Normas sobre cómo diseñar pantallas de operador y cómo administrar alarmas en planta.	19
I²C	Bus de dos cables (SDA y SCL) donde varios dispositivos comparten línea, cada uno con su dirección.	6
JetPack / L4T	Sistema y controladores de NVIDIA para Jetson (Linux for Tegra).	9
JSON	Formato de texto para intercambiar datos estructurados entre programas.	13, 16
KCD1	Interruptor de encendido de la rama de lógica del carrito.	4
LAY37	Modelo del botón de paro de emergencia del carrito (contacto normalmente cerrado).	4
LOGO	Mini controlador lógico programable; en la celda mueve la banda.	7
Lumina	Nombre del asistente de Aether: cara animada con voz, cámara y modelo de lenguaje local.	15
mAP50	Nota de qué tan bien un modelo de detección localiza y acierta objetos; 1.0 es perfecto.	7, 14
MAX485	Chip que traduce el serial del ESP32 a RS485.	5
Microcontrolador	Chip que ejecuta un solo programa sin sistema operativo, con pines que controla con precisión.	5
Micropaso	Subdivisión de un paso de un motor a pasos; 1/16 da 3200 pulsos por vuelta.	5
MJPEG	Video enviado como una secuencia interminable de fotos JPEG por una respuesta HTTP.	11
Modbus TCP	Protocolo industrial para leer y escribir registros de un PLC por Ethernet (puerto 502).	7
MQTT	Protocolo ligero de publicar y suscribirse a tópicos, con un broker; decidido para telemetría, sin implementar.	18
Multímetro	Instrumento para medir voltaje, corriente y continuidad.	2
NEMA 17	Tamaño estándar (42 × 42 mm) de motor a pasos; aquí el 17HS4401.	3, 5
Newton-metro (N·m)	Unidad de torque: fuerza que hace girar.	2
NO / NC	Contacto normalmente abierto / normalmente cerrado.	4
Observation	Registro crudo de una detección: hora, clase, código leído, confianza y marcador ArUco.	16, 17
Odometría	Estimar la posición sumando cuánto giró cada rueda.	5

Término	Qué significa	Cap.
Ohmio (Ω) y ley de Ohm	Unidad de resistencia; V = I × R.	2
Ollama	Programa que corre modelos de lenguaje en la propia computadora.	15
ONNX	Formato neutral para guardar modelos de redes neuronales.	8
Optoacoplador	Chip que pasa una señal con luz para separar eléctricamente dos circuitos.	2, 8
Outbox (buzón de salida)	Cola local con reintento desde la que un agente entrega eventos a la laptop.	16
p95	Valor que el 95 % de las mediciones no supera.	24
PlatformIO	Extensión de Visual Studio Code para programar microcontroladores; el equipo la eligió para el ESP32.	5
PLC	Programmable Logic Controller: controlador industrial robusto para tableros.	7
PNP / NPN	Tipos de salida de un sensor de 3 hilos: entrega +24 V o conecta a 0 V al detectar.	2
Potencia (W)	Trabajo por segundo; P = V × I.	2
PowerShell	La shell de Windows; se usa en la laptop.	12
Puerto	Número que identifica a qué programa de una computadora va un mensaje de red.	10
PWM	Pulse-Width Modulation: encender y apagar muy rápido y variar la proporción para controlar potencia.	5
pytest	Herramienta de pruebas automáticas de Python.	22
QR / código de barras	Códigos que una cámara lee; identifican cada fixture.	6
QTR-8A	Barra de 8 sensores infrarrojos que ven el piso (seguidor de línea, respaldo).	6
Relé / relevador	Interruptor que se cierra al energizar una bobina; separa el mundo de 3.3 V del de potencia.	2, 4, 8
ROS 2 / SLAM	Conjunto de librerías para robots / técnica para construir un mapa mientras se localiza el robot.	26
RS485	Estándar de comunicación de dos cables trenzados (A y B), robusto para entornos con ruido.	5
ruff	Herramienta que revisa estilo y errores comunes del código Python.	22

Término	Qué significa	Cap.
SCADA	Sistema de supervisión industrial que muestra y controla máquinas desde una pantalla.	11
SDA / SCL	Cables de datos y de reloj del bus I^2C.	6
Servo	Motor pequeño que gira a un ángulo indicado por una señal PWM (MG90S).	5
SQLite / SQLAlchemy	Base de datos en un solo archivo / librería que describe las tablas como clases de Python.	17
SSH	Terminal remota cifrada para manejar el Jetson o la Pi desde la laptop.	10
STEP / DIR / ENABLE	Señales de un driver de pasos: un pulso por paso, el sentido y el activar/soltar.	5
Subred	Grupo de direcciones IP que se ven directamente entre sí.	10
systemd	Gestor de servicios de Linux que arranca y vigila programas; descartado por ahora.	12
TCP / UDP	Protocolos de red: TCP establece una llamada fiable; UDP manda mensajes sin confirmación.	10
TensorRT	Compilador de NVIDIA que acelera modelos en el Jetson; no se usa todavía.	9, 26
Tierra común	Unir los GND de todas las placas para que compartan referencia.	2, 4
Time-of-Flight (ToF)	Medir distancia por el tiempo que tarda un pulso de luz en rebotar.	6
tmux	Programa que mantiene vivas las sesiones de terminal aunque se corte la conexión.	12
TR8	Husillo (tornillo de potencia) de 8 mm.	3, 5
USB UVC	Estándar de cámaras USB que Linux reconoce sin controladores extra.	6
V4L2	Video4Linux2: controlador estándar de cámaras en Linux.	6
VAD	Voice Activity Detector: decide si en el audio hay voz o solo ruido.	15
venv	Entorno virtual de Python: carpeta aislada con sus propias librerías.	13
VL53L0X	Sensor de distancia de tiempo de vuelo.	6
VNC	Escritorio remoto; en el Jetson solo sirve para mirar la pantalla.	9, 10
Voltio (V)	Unidad de voltaje: la «presión» eléctrica.	2
Vosk / Whisper	Reconocedores de voz locales: Vosk ligero con vocabulario limitado; Whisper más preciso y pesado.	15

Término	Qué significa	Cap.
Vref	Voltaje de referencia de un A4988 que fija la corriente máxima por bobina.	4
Watchdog	Perro guardián: temporizador que detiene el robot si el programa deja de mandar su latido.	4, 5
Waypoint	Punto de ruta por el que debe pasar el robot.	3, 6
WebSocket	Conexión web abierta por la que ambos lados pueden mandarse mensajes en vivo.	11
XL4005	Módulo reductor de voltaje (12 → 5 V).	4
XSHUT	Pin de apagado de un VL53L0X; sirve para darles direcciones distintas.	6
XT60	Conector amarillo de potencia para baterías.	4
YOLO	You Only Look Once: red neuronal que detecta objetos en una imagen de una sola pasada.	14

Pendientes de confirmar con el equipo

Todo lo que el libro dejó marcado como PENDIENTE (P-01, P-02, …) está aquí, y solo aquí. Cada línea dice qué falta, con quién se confirma y en qué capítulos aparece. Mientras no se confirme, el libro lo trata como «no se sabe».

Código	Qué falta	Con quién	Capítulos
P-01	Llegada e instalación de los motores ya decididos: los dos motores-rueda M0601 (FIT1042 y FIT1038) y los tres NEMA 17 de la torre. Modelo y documentación exactos de cada uno.	David, Isaac	3, 5
P-02	Peso total real del robot, reparto por llanta (hoy: máximo estimado de 40 kg y unos 17 kg por llanta motriz), altura del centro de gravedad y pendiente máxima del piso.	David, Isaac	2, 3
P-03	Diámetro real de las llantas motrices y de las rodajas.	David, Isaac	2
P-04	Estado real del armado del carrito y si la hoja de armado (revisión 3.8) es el diseño que se construye; material, impresora y relleno de las piezas impresas.	Isaac, David	3
P-05	Modelo y capacidad reales de la batería (la hoja de armado dice BR-1224, 12 V, 24 Ah).	David	3, 4
P-06	Paro de emergencia del robot: decisión abierta sobre si el ESP32 debe enterarse de que se pulsó (hoy solo corta el relé); modelo, cableado y ubicación definitivos.	David, Mario	4, 5
P-07	Modelo exacto de la placa ESP32 que se tiene en mano (la hoja supone un DevKitC de 38 pines).	Mario	3, 5
P-08	Protocolo de mensajes RS485 del motor-rueda M0601, según la hoja técnica del fabricante.	Mario, David	5
P-09	Paso real (avance por vuelta) del husillo TR8: define todo el programa del elevador.	David, Isaac	5
P-10	Sensores definitivos del robot: si se comprará un lidar y si los VL53L0X (4), el BNO055 y la QTR-8A quedan confirmados.	Mario, David	6
P-11	Tamaño impreso, colocación y distancia de lectura de los marcadores ArUco.	Mario, Isaac	6
P-12	Detalles físicos del cableado de la celda por GPIO: pines reales, tipo de placa de relevadores (activa en alto o en bajo, aislamiento, diodo, alimentación) y marca y modelo del LOGO.	Uriel, David, Pedro	7, 8

Código	Qué falta	Con quién	Capítulos
P-13	Marca, modelo y tipo (PNP o NPN) de los dos infrarrojos de la celda y sus bornes.	Uriel, David	2
P-14	Pistón de scrap: tipo, electroválvula, voltaje de la bobina, presión y tiempos reales de pulso y regreso.	Pedro	7
P-15	Banda: motor, velocidad real, altura real de la cinta y si corre sola o la arranca el LOGO.	Isaac, David, Pedro	3, 7
P-16	Paro de emergencia físico de la celda y arranque y paro físicos independientes de la pantalla.	David, Pedro	4, 7, 8, 21
P-17	Alcance de la HMI de la celda: pantalla propia, quién la opera, qué muestra.	Emmanuel	7, 19
P-18	Pruebas de la Raspberry Pi 4 en hardware real: sistema, RAM, versión de OpenCV con el modelo ONNX, CPU y cámara.	Mario, Uriel	7, 8
P-19	Alimentación del Jetson dentro del robot y cómo se evita el doble camino de 5 V hacia el ESP32 (convertidor y USB).	David	4, 9
P-20	Si el robot (o el Jetson) tendrá micrófono y bocina propios.	Mario	6, 9, 15
P-21	Dimensiones y material oficiales de los fixtures y quién los diseña y fabrica.	Isaac	7
P-22	Mediciones y umbrales pendientes: FPS del Jetson con buena luz, cámara en su lugar y modo de energía anotado; umbrales numéricos de los criterios de aceptación; pruebas de los tres estados de cobertura.	Mario	6, 9, 14, 17, 22
P-23	Contrato de mensajes entre el Jetson y el ESP32: mensajes, latido, tiempo de frenado y qué hace el ESP32 al pulsarse el paro.	Mario	5
P-24	Criterio real de PASS/FAIL por defectos de la pieza (hoy solo reconoce piezas registradas).	Mario	7
P-25	Arranque automático de la celda al encender la Pi (systemd u otro mecanismo).	Mario, Uriel	8, 12
P-26	Diagnósticos abiertos del Jetson: causa de los cortes de SSH, de `snap-confine`, propiedad de `libcusparseLt` y SSH sin contraseña.	Mario	9, 10
P-27	Pruebas de voz con el micrófono, la voz y el ruido reales del lugar (verificaciones V5 a V13), y decisión sobre instalar Whisper.	Mario	15, 22
P-28	Confirmar que el secreto de la integración de música, que se pegó una vez en un chat, ya fue rotado.	Mario	21

Código	Qué falta	Con quién	Capítulos
P-29	Pasos manuales de Cloudflare: alta del dominio, servidores de nombres, SSL Full (strict), HSTS, encabezados, Bot Fight, límite de peticiones, DMARC y DKIM.	Mario	20, 21
P-30	Ejecutar la lista del laboratorio en el router y la PC (sin reenvío de puertos, sin UPnP, firewall solo privado).	Mario	21
P-31	Medir el video de presentación y la sincronía en el hardware y la red del escenario, y el reconocimiento de «Lumina» con el micrófono real.	Mario	24
P-32	Aceptación formal de la ADR 0012 y publicación (`push`) de la rama `jetson-agente` .	Mario	1, 9